



ESCUELA DE INGENIERÍA DE FUENLABRADA

GRADO EN INGENIERÍA AERQESPACIAL
EN AERONAVEGACIÓN

TRABAJO FIN DE GRADO

DESARROLLO DE UN ALGORITMO DE VISIÓN
PARA LA NAVEGACIÓN RELATIVA AL TERRENO DURANTE ALUNIZAJES

Autor: SERGIO GONZÁLEZ GIL

Tutor: HODEI URRUTXUA CEREIJO

Curso académico 2025-2026

Resumen

Aterrizar en un punto concreto de la Luna es, todavía hoy, un problema abierto. Una nave que desciende solo con su unidad inercial acumula deriva, y sin un sistema de posicionamiento global que la corrija termina cayendo dentro de una elipse de dispersión de varios kilómetros. La respuesta que ha adoptado la industria es la navegación relativa al terreno: mirar el suelo con una cámara y compararlo con un mapa guardado a bordo. La variante más exigente de esa idea consiste en no comparar imágenes, sino reconocer cráteres concretos y usarlos como puntos de referencia, igual que un sensor de estrellas reconoce constelaciones. Es el método que llevó a la sonda japonesa SLIM a menos de diez metros de su objetivo en enero de 2024.

Este trabajo desarrolla y valida por completo un sistema de ese tipo. Se construye primero un entorno sintético de superficie lunar en Blender, con relieve procedural y un catálogo de cráteres de verdad conocida, que permite generar secuencias de cámara con la trayectoria real anotada fotograma a fotograma. Sobre ese entorno se afina un detector de objetos YOLO hasta alcanzar una cobertura del 99,4 % de los cráteres visibles en condiciones nominales. Las detecciones, expresadas únicamente en píxeles, alimentan un motor de emparejamiento geométrico escrito en MATLAB que no recibe ninguna información previa de posición ni de altitud: agrupa las detecciones en tripletas, calcula para cada una un descriptor de cuatro componentes invariante a la escala y a la rotación, y lo busca en una tabla precalculada a partir de un mapa embarcado de 258 cráteres. La hipótesis ganadora se verifica exigiendo coherencia simultánea de posición y de diámetro, y de la transformación de semejanza resultante se despejan la posición, la orientación y la altitud.

El sistema se somete a tres ensayos de dificultad creciente. En vuelo de crucero resuelve el 97,7 % de los fotogramas con un error mediano de posición de 0,150 km. Ejecutado en directo sobre vídeo, con la red neuronal corriendo en el bucle, mantiene la misma precisión a 4,4 Hz. En un descenso con la cámara girando dos ciclos completos de $\pm 180^\circ$ en guiñada, estima la orientación con un error mediano de $0,112^\circ$ y la altitud con 0,950 km, y funciona hasta que la altitud baja de unos 118 km, punto en el que el campo de visión deja de abarcar suficientes cráteres del mapa. Ese límite se caracteriza de forma cuantitativa y se demuestra, mediante un barrido con tres mapas de densidad creciente, que no se resuelve simplemente añadiendo más cráteres al catálogo: los mapas más densos producen más soluciones, pero menos fiables.

Palabras clave: navegación relativa al terreno, detección de cráteres, aprendizaje profundo, YOLO, descriptores invariantes, aterrizaje lunar de precisión, navegación óptica autónoma.

Abstract

Landing on a specific spot on the Moon remains an open problem. A spacecraft descending on inertial measurements alone accumulates drift, and with no global positioning system to correct it, it ends up somewhere inside a dispersion ellipse several kilometres wide. The industry answer is terrain relative navigation: look at the ground with a camera and compare it against a map carried on board. The most demanding version of that idea does not compare images at all, but recognises individual craters and uses them as reference points, much as a star tracker recognises constellations. That is the method that placed the Japanese SLIM probe within ten metres of its target in January 2024.

This work develops and fully validates such a system. A synthetic lunar surface is first built in Blender, with procedural relief and a ground-truth crater catalogue, which allows camera sequences to be rendered with the true trajectory annotated frame by frame. A YOLO object detector is then fine-tuned on that environment until it recovers 99,4 % of the visible craters under nominal conditions. The detections, expressed purely in pixels, feed a geometric matching engine written in MATLAB that receives no prior estimate of position or altitude: it groups detections into triplets, computes for each one a four-component descriptor invariant to scale and rotation, and looks it up in a table precomputed from an on-board map of 258 craters. The winning hypothesis is verified by requiring simultaneous consistency in position and diameter, and position, attitude and altitude are recovered from the resulting similarity transform.

The system is tested in three increasingly demanding scenarios. In cruise flight it solves 97,7 % of the frames with a median position error of 0,150 km. Running live on video, with the neural network in the loop, it holds the same accuracy at 4,4 Hz. In a descent with the camera performing two full $\pm 180^\circ$ yaw cycles, it estimates attitude with a median error of $0,112^\circ$ and altitude with 0,950 km, and keeps working until altitude drops below roughly 118 km, the point at which the field of view no longer contains enough mapped craters. That limit is characterised quantitatively, and a sweep over three maps of increasing density shows it cannot be removed simply by adding more craters to the catalogue: denser maps yield more solutions, but less reliable ones.

Keywords: terrain relative navigation, crater detection, deep learning, YOLO, invariant descriptors, precision lunar landing, autonomous optical navigation.

Agradecimientos

A mi tutor, Hodei Urrutxua, por aceptar un trabajo que empezó siendo una idea vaga sobre reconocer cráteres y por dedicarle un tiempo que no siempre estaba en su horario. Sus correcciones son la razón de que esta memoria se parezca a un trabajo de ingeniería y no a un cuaderno de laboratorio.

A mis padres, por estos cuatro años de paciencia, y por no preguntar demasiado cuando la respuesta era que seguía sin salir. Nada de lo que hay aquí existiría sin el apoyo, callado y constante, que me han dado desde mucho antes de que empezara la carrera.

A mis amigos y a mis compañeros de la Escuela, con quienes he compartido prácticas, vísperas de examen y más de una tarde entera peleándome con un error que resultó ser una tontería. En especial a vosotros, Carlos, Joaquín, Marcos y Mario, se hace mucho más llevadero en compañía.

A mi gran amigo Alejandro, con quien he compartido tardes y tardes de cafés charlando de cosas filosóficas y mucha ciencia.

A mis dos compañeros de trabajo, Raúl y Sebas, que han trascendido lo laboral hasta llegar a ser grandes amigos, muchas gracias por todo y por llevar el terreno a la realidad.

A dos amigos con los que he compartido muchos momentos, anécdotas, partidas, chismoseos... Por acompañarme en muchas noches y momentos felices, Álvaro e Iker.

A Ana, mi pesesita pes, por haber aguantado este último año de cerca, que es el que peor se lleva. Por escucharme explicar cráteres y matrices de semejanza con una paciencia que no le correspondía, por sacarme de casa cuando llevaba demasiadas horas delante de la pantalla y por no dejar que las semanas malas parecieran más largas de lo que eran. Esto también es suyo.

A todos ellos, gracias.

Índice general

1. Introducción	1
1.1. Motivación	1
1.1.1. ¿Por qué mirar el suelo?	2
1.1.2. ¿Por qué cráteres, y por qué este trabajo?	2
1.2. Navegación relativa al terreno	3
1.2.1. Correlación de imagen con un mapa ortorectificado	3
1.2.2. Navegación por puntos característicos	4
1.2.3. Navegación por cráteres	4
1.2.4. Correlación de relieve mediante altimetría	5
1.2.5. Odometría visual y velocimetría	5
1.2.6. La capa de estimación	5
1.2.7. ¿Qué ha volado de verdad?	5
1.3. Detección e identificación de cráteres	6
1.3.1. Familias de detección	6
1.3.2. El problema de la caja y la elipse	7
1.3.3. Identificación contra el catálogo	8
1.3.4. Catálogos y datos disponibles	9
1.4. Objetivos	9
1.5. Planificación y gestión del trabajo	10
1.5.1. Paquetes de trabajo	10
1.5.2. Hitos	10
1.5.3. Método de trabajo y tabla de aprendizajes	11
1.6. Relación con las asignaturas del grado	11
1.6.1. Habilidades adquiridas fuera del temario	12
1.7. Declaración sobre el uso de inteligencia artificial	12
1.8. Estructura del trabajo	13
2. Generación del entorno lunar sintético	15
2.1. ¿Qué necesita el sistema?	15
2.2. ¿Por qué un entorno sintético y por qué Blender?	16
2.3. Unidades y sistemas de referencia	17
2.4. Generación del terreno	17
2.4.1. Ruido fractal	17
2.4.2. Tamaño de los cráteres	19
2.4.3. Forma de un cráter	20
2.4.4. Mares, crestas de arruga y albedo	21
2.4.5. ¿De dónde salen los parámetros?	22
2.5. Montaje de la escena en Blender	22
2.5.1. La malla del terreno	23
2.5.2. Geometría y bump	23
2.5.3. Material, iluminación y render	23

2.6.	Modelo de cámara	24
2.6.1.	Cámara estenopeica	24
2.6.2.	Huella y resolución	24
2.6.3.	La cámara del mapa	25
2.7.	Trayectorias	25
2.7.1.	Secuencia de cruceo	25
2.7.2.	Secuencia de descenso con rotación	26
2.8.	Limitaciones del entorno	27
3.	Detección de cráteres	29
3.1.	¿Por qué un detector de una sola pasada?	29
3.2.	¿Cómo funciona la red?	30
3.2.1.	Troncal, cuello y cabeza	30
3.2.2.	La función de pérdida	31
3.2.3.	Supresión de no máximos	31
3.3.	Cadena de entrenamiento	31
3.4.	Etapas 1: imágenes reales de cráteres	32
3.5.	Etapas 2: ajuste fino sobre los renders	32
3.5.1.	La partición del conjunto, que no es obvia	33
3.5.2.	Hiperparámetros	33
3.5.3.	¿Cuánto aporta esta etapa?	34
3.6.	Etiquetado automático de los renders	34
3.6.1.	Proyección del catálogo	35
3.6.2.	¿Qué cráteres se etiquetan?	35
3.6.3.	Comprobación de las etiquetas	36
3.7.	Un fallo que no era del modelo	36
3.8.	Punto de operación	36
3.9.	Etapas 3: reentrenamiento para la rotación	37
3.9.1.	¿Qué se midió?	37
3.9.2.	¿Por qué ocurre?	38
3.9.3.	La corrección	38
3.9.4.	Un efecto secundario que resultó revelador	38
3.10.	Validación del detector	39
3.10.1.	Métricas	39
3.10.2.	Conjuntos de datos y protocolo	39
4.	Posicionamiento relativo al terreno	41
4.1.	El problema: estimar el vector de estados	41
4.2.	El mapa embarcado y la tabla de tripletas	42
4.2.1.	¿Qué cráteres van al mapa?	42
4.2.2.	¿Por qué tripletas?	42
4.2.3.	El descriptor invariante	43
4.2.4.	Construcción y poda de la tabla	44
4.3.	Resolución de un fotograma	45
4.3.1.	Generación de hipótesis	45
4.3.2.	Ajuste de la transformación de semejanza	45
4.3.3.	De la semejanza a la pose	46
4.3.4.	Verificación de la hipótesis	47
4.3.5.	El algoritmo completo	47
4.4.	Validación del posicionamiento	49
4.4.1.	Magnitudes de error y criterio de acierto	49
4.4.2.	Protocolo de los ensayos	49

4.4.3.	Comprobaciones de consistencia	50
4.4.4.	Implementación	50
5.	Simulación y validación del sistema de navegación	51
5.1.	Escenarios de validación	51
5.1.1.	Condiciones de simulación	51
5.1.2.	¿Qué recibe el sistema y qué no?	53
5.1.3.	Lo que ve la cámara	53
5.2.	¿Cómo se mide?	53
5.3.	Rendimiento del detector	55
5.3.1.	El ajuste fino sobre el terreno sintético	55
5.3.2.	El coste real del fallo de filtrado	56
5.3.3.	El punto de operación	56
5.3.4.	Robustez frente a la rotación	57
5.4.	Ajuste del emparejador	58
5.5.	Ensayo 1: navegación en diferido	59
5.6.	Ensayo 2: navegación en tiempo real	60
5.7.	Ensayo 3: descenso con rotación en guiñada	61
5.8.	El muro de altitud y la densidad del mapa	62
5.9.	Repetibilidad	63
5.10.	Síntesis	63
6.	Conclusiones y trabajos futuros	65
6.1.	Conclusiones	65
6.2.	Trabajos futuros	67
6.2.1.	Detección de elipses orientadas	67
6.2.2.	Incorporación de un filtro de navegación	67
6.2.3.	Mapa jerárquico por altitud	67
6.2.4.	Validación sobre imágenes lunares reales	68
	Bibliografía	69
	Anexos	73
A.	Repositorio de código	75
A.1.	Organización del repositorio	75
A.2.	Índice de ficheros	75
A.3.	El puente entre MATLAB y Python	76
B.	Tablas de resultados completas	79
B.1.	Barrido conjunto de umbral de mapa y número de detecciones	79
B.2.	Robustez del detector frente al giro de la imagen	80
B.3.	Barrido del umbral de confianza	80
B.4.	Cobertura por franja de altitud	80
C.	Parámetros del sistema	83
C.1.	Cámara y escena	83
C.2.	Terreno y catálogo	83
C.3.	Detector	84
C.4.	Emparejamiento	84
C.5.	Verificación y filtros físicos	84
C.6.	Continuidad entre fotogramas	84

D. Guía de reproducción	87
D.1. Entorno	87
D.2. Orden de ejecución	87
D.3. Salidas esperadas	88
D.4. Comprobación de que la reproducción es correcta	88
E. Deducciones	89
E.1. El ajuste de la transformación de semejanza	89
E.2. El inflado de la caja envolvente bajo rotación	90

Índice de figuras

1.1.	Principio de la navegación relativa al terreno	3
1.2.	Caja alineada frente a elipse orientada	8
2.1.	Arquitectura del sistema	16
2.2.	Cadena de generación del entorno sintético	18
2.3.	Perfiles radiales de cráter	21
2.4.	Geometría de proyección	25
2.5.	Trayectoria de crucero	26
3.1.	Arquitectura del detector	30
3.2.	Curvas de entrenamiento del detector	34
3.3.	Detecciones sobre un fotograma	37
4.1.	El mapa embarcado	43
4.2.	El descriptor de tripleta	44
4.3.	Algoritmo de resolución de un fotograma	48
5.1.	Perfiles de los escenarios de validación	52
5.2.	Secuencia de fotogramas con las detecciones	54
5.3.	Vista tridimensional de la trayectoria	54
5.4.	Distribución del error de posición	55
5.5.	Robustez del detector frente al giro de la imagen	57
5.6.	Trayectoria estimada y dispersión del error	59
5.7.	Error de posición a lo largo del vuelo de crucero	60
5.8.	Cobertura y error frente a la altitud	61
5.9.	Cobertura frente a altitud con tres mapas de distinta densidad	62

Índice de tablas

1.1. Misiones con navegación relativa al terreno	6
1.2. Paquetes de trabajo	10
1.3. Tabla de aprendizajes	11
2.1. Herramientas consideradas	16
2.2. Capas del modelo de terreno	18
3.1. Cadena de entrenamiento del detector	32
3.2. Conjuntos públicos empleados	32
3.3. Partición del conjunto de renders	33
3.4. Hiperparámetros del ajuste fino	33
3.5. Efecto del ajuste fino	34
3.6. Efecto del tamaño de lote sobre la pérdida de fotogramas	36
3.7. Robustez a la rotación	38
5.1. Escenarios de validación	51
5.2. Comparación de detectores	56
5.3. Efecto del fallo de filtrado sobre la percepción útil	56
5.4. Barrido del umbral de confianza	57
5.5. Barrido del número de detecciones por fotograma	58
5.6. Los tres mapas de referencia	58
5.7. Resultados del ensayo en diferido	59
5.8. Resultados del ensayo de descenso	61
5.9. Efecto de la densidad del mapa	62
5.10. Síntesis de los tres ensayos	64

Capítulo 1

Introducción

1.1. Motivación

El 20 de enero de 2024 la sonda japonesa SLIM se posó sobre la Luna a unos 55 m del punto al que apuntaba. Tres días antes, en cualquier manual de la disciplina, un aterrizaje lunar «preciso» significaba caer en algún lugar dentro de una elipse de varios kilómetros. La diferencia entre ambas cifras no está en el motor ni en la estructura del vehículo: está en cómo la nave averigua dónde se encuentra (JAXA, 2024).

Ese es el problema del que trata este trabajo, y hay que plantearlo desde el principio sin adornos. Una nave que desciende sobre la Luna lleva a bordo una unidad de medida inercial (IMU): un conjunto de acelerómetros y giróscopos que mide, muchas veces por segundo, la aceleración y la velocidad de giro del vehículo. Integrando esas medidas en el tiempo se obtiene la velocidad, y volviendo a integrar, la posición. Sobre el papel, con eso bastaría. En la práctica no basta, porque toda medida tiene un pequeño error y la integración lo acumula: un sesgo minúsculo en un acelerómetro se convierte, tras diez o quince minutos de descenso, en un error de posición de kilómetros. Es lo que se conoce como *deriva inercial*, y no es un defecto de fabricación que se pueda eliminar comprando un sensor mejor: es una consecuencia matemática de integrar dos veces una señal ruidosa.

En la Tierra este problema está resuelto desde hace décadas y ni siquiera lo percibimos. Un avión o un teléfono móvil corrigen continuamente su deriva inercial con las señales de los satélites GNSS, que le dicen dónde está en coordenadas absolutas varias veces por segundo. En la Luna no hay GNSS. Tampoco hay balizas, ni radiofaros, ni ninguna infraestructura de navegación instalada sobre el terreno. Y la ayuda desde la Tierra, que durante décadas ha sido el método estándar, tiene dos limitaciones independientes que la descartan para las fases críticas. La primera es que la determinación de órbita desde tierra no es instantánea: las estaciones no miden la posición de la nave, sino su velocidad en la línea de visión y su distancia, de modo que la componente transversal solo se despeja acumulando un arco de seguimiento de una o varias órbitas y ajustando después un modelo dinámico a todas las medidas a la vez. Entre la toma de datos, el procesado en tierra y la subida del vector de estado corregido transcurren horas, mientras que un descenso propulsado dura minutos. La segunda es el propio retardo del enlace: la señal tarda 1,28 s en llegar a la Luna y otro tanto en volver, y con 2,56 s de ida y vuelta es imposible cerrar un lazo de control en el que las decisiones se toman en fracciones de segundo.

La consecuencia práctica de todo lo anterior es que, hasta hace muy poco, elegir un lugar de aterrizaje era en realidad elegir una *región*. Las misiones Apollo aterrizaron en mares basálticos amplios y llanos precisamente porque no se podía garantizar caer en un punto concreto. Y esto choca frontalmente con lo que la exploración lunar quiere hacer ahora. Los objetivos científicos y operativos del programa Artemis y de las misiones asociadas están, casi todos, en sitios pequeños y rodeados de terreno hostil: cráteres en permanente sombra cerca del polo sur donde hay hielo, bordes de cráter iluminados de forma casi continua que son los únicos emplazamientos viables para una base energéticamente autónoma, tubos de lava, y en

el futuro las inmediaciones de infraestructura ya desplegada. Aterrizar «en la zona» ya no vale. Hay que aterrizar *ahí*.

1.1.1. ¿Por qué mirar el suelo?

La única referencia absoluta disponible en las cercanías de la Luna es la propia Luna. De esa observación evidente nace la *navegación relativa al terreno* (en la literatura, *Terrain Relative Navigation* o TRN): el vehículo mira el suelo con un sensor, compara lo que ve con un mapa que lleva almacenado, y de esa comparación deduce en qué punto de ese mapa se encuentra. El resultado, una posición absoluta en coordenadas del cuerpo celeste, se inyecta en el filtro de navegación y corta de raíz la deriva acumulada por la IMU.

Es exactamente lo que hacía un navegante con un sextante y una carta, y también lo que hicieron los comandantes del Apollo: el Módulo Lunar navegaba por propagación inercial, y el astronauta identificaba accidentes del terreno que había memorizado de las fotografías de reconocimiento, corrigiendo el punto de toma con una retícula grabada en la ventana. Con ese procedimiento el Apollo 12 se posó a menos de 200 m de la sonda Surveyor 3 en 1969 (Lunar Reconnaissance Orbiter Camera, 2009). Es un dato para tener presente durante todo este trabajo: el listón lo puso un ser humano mirando por una ventana, y la primera máquina que lo igualó de forma autónoma no llegó hasta 2021, con el sistema de visión del aterrizador de Perseverance en Marte (Johnson et al., 2023).

1.1.2. ¿Por qué cráteres, y por qué este trabajo?

Dentro de la TRN hay varias maneras de comparar lo que se ve con lo que se lleva guardado, y todas se revisan en la Sección 1.2. La que ha volado con más éxito compara la imagen de la cámara con un mosaico del terreno, píxel contra píxel, y funcionó lo bastante bien como para dejar a Perseverance a 5 m de su objetivo.

La navegación por cráteres propone algo distinto: en lugar de comparar brillos, comparar *geometría*. Un cráter es una circunferencia sobre el terreno, y su centro y su diámetro son magnitudes físicas que siguen siendo las mismas aunque cambie la hora del día. Ese cambio de moneda arrastra consecuencias sobre el tamaño del mapa, sobre la sensibilidad a la iluminación y sobre qué hay que calcular a bordo, y todas ellas se examinan en la Sección 1.2.3.

Ese planteamiento dejó de ser teórico en enero de 2024. SLIM es, hasta la fecha, la única misión que ha volado el emparejamiento de cráteres como método principal de posicionamiento absoluto: llevaba mapas de cráteres contruidos con imágenes de los orbitadores Kaguya y LRO, los comparaba con lo que veían sus cámaras de navegación, y tenía menos de cinco segundos para resolver cada fijación (Ishida et al., 2025). JAXA evaluó la precisión de posición antes de la maniobra final en unos 10 m o menos (JAXA, 2024). Hay que separar bien dos cosas que la prensa mezcló: el aterrizador acabó volcado porque perdió una tobera del motor principal a 50 m de altura. La navegación funcionó.

Al mismo tiempo, la pieza que hasta hace poco era el cuello de botella de este enfoque — detectar cráteres de forma fiable en una imagen real — ha cambiado por completo. La detección automática de cráteres pasó en pocos años de algoritmos de bordes y sombras, frágiles frente a la iluminación, a redes neuronales convolucionales que alcanzan coberturas superiores al 90 % (Silburt et al., 2019). Hay, por tanto, una convergencia clara: un método de navegación cuya viabilidad operativa acaba de demostrarse en vuelo, y una tecnología de percepción que acaba de volverse suficientemente buena.

Este trabajo se sitúa en esa convergencia, y su motivación última es de carácter demostrativo. No pretende superar a SLIM ni proponer un algoritmo mejor que el de un equipo de una agencia espacial. Pretende algo más modesto y, para un Trabajo Fin de Grado, más útil: construir de principio a fin (terreno sintético, detector entrenado, mapa embarcado, motor de emparejamiento y demostración en tiempo real) una cadena de navegación por

cráteres completa y honesta, medida contra una verdad de referencia conocida, y llevarla deliberadamente hasta el punto en el que falla, para poder decir con números por qué falla. Un sistema de navegación que solo se enseña funcionando no enseña gran cosa. Uno del que se conoce el borde del sobre de operación, sí.

1.2. Navegación relativa al terreno

Antes de revisar las técnicas conviene fijar el vocabulario, porque hay tres cosas que se confunden con frecuencia en la literatura divulgativa e incluso en algunas notas de prensa de fabricantes. La *navegación relativa al terreno* responde a la pregunta «¿dónde estoy?» en coordenadas absolutas del cuerpo celeste, y para responderla necesita un mapa previo. La *odometría visual* responde a «¿cuánto me he movido desde hace un instante?», no necesita mapa, y por tanto no elimina la deriva: solo la ralentiza. Y la *detección de peligros* (*Hazard Detection and Avoidance*, HDA) responde a «¿dónde puedo posarme sin volcar?», que es una pregunta sobre la seguridad del terreno y no sobre la posición del vehículo. Las tres suelen convivir a bordo de la misma nave y se apoyan mutuamente, pero son problemas distintos. Este trabajo aborda la primera.

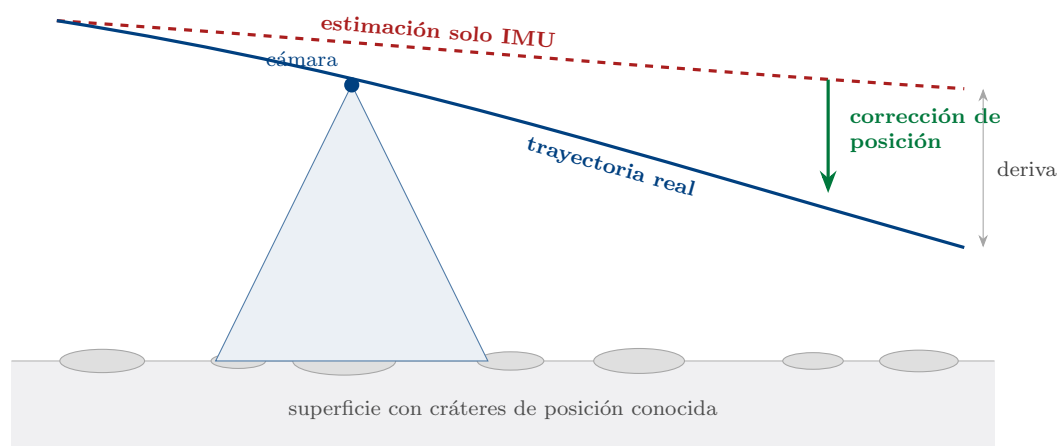


Figura 1.1: Principio de la navegación relativa al terreno. La estimación puramente inercial se separa progresivamente de la trayectoria real. Cada vez que la cámara reconoce el terreno que tiene debajo y lo localiza en el mapa embarcado, el filtro de navegación recibe una posición absoluta y la deriva se anula.

Las familias que siguen se distinguen por *qué* se compara: intensidades de imagen, puntos característicos, cráteres, relieve o movimiento aparente. La capa que fusiona todo eso con la IMU (el filtro de navegación) se trata aparte en la Sección 1.2.6, porque es un problema de estimación y no de percepción, y mezclarlo con lo anterior es una fuente habitual de confusión.

1.2.1. Correlación de imagen con un mapa ortorectificado

Es la aproximación más directa y la que mejor ha funcionado en vuelo. A bordo se lleva un mosaico de imágenes del terreno tomado previamente desde un orbitador y corregido geométricamente para que se comporte como un mapa plano (*ortorectificado*). Durante el descenso se recorta una ventana pequeña de la imagen de la cámara y se desliza sobre la región del mapa donde el filtro cree estar, buscando la posición en la que ambas coinciden mejor. La medida de coincidencia es una correlación estadística entre intensidades de píxel; el máximo de esa correlación da el desplazamiento, y de ahí sale la posición.

El sistema de visión del aterrizador de Mars 2020 es el ejemplo canónico. Lleva un mosaico construido a partir de imágenes de la cámara CTX del orbitador MRO, localiza del orden

de 15 puntos de referencia por imagen y cierra la solución con la IMU (Cheng et al., 2021; Johnson et al., 2023). El resultado fue un aterrizaje a 5 m del objetivo el 18 de febrero de 2021, el primero de la historia con navegación relativa al terreno operativa en un aterrizaje propulsado (Chen et al., 2021).

Sus dos limitaciones son estructurales. Una es el volumen del mapa: decenas o cientos de megabytes de memoria no volátil para una única zona preparada a medida, lo que obliga a rehacer el trabajo cartográfico en cada misión. La otra es la dependencia de la iluminación: la correlación compara brillos, y el brillo de una ladera depende del ángulo solar. Si el mapa se tomó con el Sol a 45° y el descenso ocurre con el Sol a 10° , las sombras son otras y la correlación se degrada. En la práctica esto se traduce en ventanas de aterrizaje restringidas.

1.2.2. Navegación por puntos característicos

En lugar de comparar la imagen entera, aquí se extraen de ella unos pocos puntos singulares (esquinas, manchas, texturas repetibles) y se describe cada uno con un vector numérico que intenta ser el mismo aunque cambien la escala, el giro o el punto de vista. Los descriptores clásicos de visión por computador, SIFT y sus sucesores, hacen exactamente esto (Lowe, 2004). A bordo se guarda una base de datos de esos descriptores con sus coordenadas conocidas; en vuelo se extraen los de la imagen actual y se buscan sus parejas.

La demostración más impresionante de esta familia es el *Natural Feature Tracking* de OSIRIS-REx, que en octubre de 2020 guió la nave hasta la superficie del asteroide Bennu comparando lo que veía con parches de terreno renderizados a partir de un modelo digital construido durante casi dos años de reconocimiento previo. La nave llegó a menos de un metro del punto objetivo, con una diferencia de 3,5 cm respecto a la posición predicha de contacto (Norman et al., 2022). Es el récord absoluto de precisión, con la salvedad importante de que la gravedad de Bennu es del orden de 10^{-5} veces la terrestre y la aproximación fue lentísima; no es directamente comparable con un descenso lunar propulsado.

La limitación de la familia es que sus puntos característicos son *arbitrarios*: no significan nada, son solo configuraciones de píxeles que resultan ser estables. Eso los hace muy generales, pero también los hace vulnerables a lo mismo que la correlación, porque un descriptor construido sobre gradientes de intensidad cambia cuando cambia la iluminación.

1.2.3. Navegación por cráteres

Es el caso particular de la familia anterior en el que el punto característico deja de ser arbitrario y pasa a ser un objeto con significado: un cráter de impacto. La diferencia parece cosmética y no lo es. Un cráter tiene una propiedad de la que carece una esquina cualquiera: es una circunferencia sobre el terreno, con un centro y un diámetro que son magnitudes físicas medibles. Eso permite razonar geoméricamente sobre él, en lugar de limitarse a comparar apariencias.

Las ventajas se siguen de ahí. El mapa se reduce a una lista de coordenadas y diámetros, minúscula comparada con un mosaico de imagen. La descripción es invariante a la iluminación una vez detectado el cráter, porque la geometría no depende del Sol. Y los cráteres están, en la Luna, por todas partes y a todas las escalas, lo que da referencias tanto en órbita alta como cerca del suelo.

Las desventajas también. Hace falta un detector fiable, y detectar cráteres *sí* depende de la iluminación aunque su geometría no lo haga. Hace falta resolver la identificación, que es un problema combinatorio y nada trivial. Y hace falta que en el campo de visión quepan suficientes cráteres catalogados, condición que deja de cumplirse a baja altura y que en este trabajo se caracteriza cuantitativamente.

El estado del arte de esta familia se cubre en la Sección 1.3, junto con la detección, porque en la práctica ambos problemas son inseparables.

1.2.4. Correlación de relieve mediante altimetría

Las tres familias anteriores usan una cámara pasiva. Esta usa un sensor activo (un altímetro o un LIDAR) que mide directamente la distancia al suelo. Con un LIDAR de barrido se obtiene un perfil o una nube de puntos tridimensional del terreno sobrevolado, y esa forma se compara con un modelo digital de elevaciones almacenado a bordo, buscando el desplazamiento que mejor las alinea (Johnson and Ivanov, 2010).

Su gran virtud es que el relieve no depende de la iluminación en absoluto: funciona de noche, con el Sol rasante o dentro de un cráter en sombra permanente, que es precisamente donde están los objetivos científicos del polo sur lunar. Sus inconvenientes son el consumo, la masa y el coste del instrumento, y el hecho de que necesita relieve que correlar: sobre una llanura basáltica lisa, un perfil de alturas es igual en todas partes y la solución es ambigua. El programa ALHAT de la NASA y el LIDAR LEIA previsto para el aterrizador europeo Argonaut son los exponentes de esta línea (Agencia Espacial Europea, 2026; Carson et al., 2014).

1.2.5. Odometría visual y velocimetría

La última familia es de naturaleza distinta porque *no lleva mapa*. Se compara la imagen actual con la anterior, se mide cuánto se han desplazado los detalles del terreno entre ambas y, conocida la altura, se deduce la velocidad del vehículo. No da posición absoluta: da un incremento. Por eso no elimina la deriva, aunque la reduce mucho respecto a la integración inercial pura.

El sistema DIMES de los róveres Spirit y Opportunity es el ejemplo histórico: midió la velocidad horizontal durante el descenso para que el sistema de retrocohetes pudiera compensar el viento marciano (Johnson et al., 2007). Su papel natural es de complemento y no de sustituto: da continuidad entre dos fijaciones absolutas de TRN.

1.2.6. La capa de estimación

Ninguna de las familias anteriores se usa sola. Todas producen medidas ruidosas y esporádicas que hay que combinar con la propagación inercial, y de eso se encarga el filtro de navegación. El caballo de batalla es el filtro de Kalman extendido y, en las implementaciones modernas, su variante en error (*error-state*), que propaga el error de la estimación en lugar de la estimación misma y se comporta mucho mejor cuando hay ángulos de actitud de por medio. Cuando el problema es muy no lineal o la incertidumbre inicial es enorme se recurre a filtros de partículas, y cuando se puede permitir procesar a posteriori, a optimización sobre grafos de restricciones.

La distinción importante para este trabajo es la siguiente: el filtro *suaviza y arrastra* la estimación en el tiempo, y por tanto puede tapar fallos puntuales del sistema de percepción. Un sistema que produzca soluciones correctas el 70 % de las veces puede parecer impecable después de pasar por un buen filtro. Como el objetivo aquí es medir la calidad de la percepción y del emparejamiento, y no la del filtro, en este trabajo **no se ha implementado ningún filtro de navegación**: cada fotograma se resuelve de forma independiente, sin usar el resultado del anterior. Es una decisión tomada a sabiendas: empeora las cifras y hace visible lo que se quiere medir. Se discute de nuevo en las conclusiones.

1.2.7. ¿Qué ha volado de verdad?

La Tabla 1.1 resume las misiones que han empleado alguna forma de navegación relativa al terreno. Merece la pena leerla con atención porque, más allá de los nombres, cuenta una historia: la técnica pasa de ser un experimento a ser un requisito de misión en apenas cinco

años, y las dos misiones comerciales que fallaron en 2024 y 2025 lo hicieron por problemas en sus sensores de distancia, no por el concepto.

Tabla 1.1: Misiones que han empleado navegación relativa al terreno o técnicas directamente relacionadas. La precisión indicada es la distancia al punto objetivo cuando se ha publicado.

Misión	Año	Sensor	Método	Precisión
Apollo 12	1969	Ojo humano	Designación manual del punto de toma	<200 m
MER (DICES)	2004	Cámara	Flujo óptico, solo velocidad	—
Chang'e-3	2013	Cámara + LIDAR	Detección de peligros en dos fases	n.p.
Hayabusa2	2019	Cámara	Seguimiento de marcador artificial	~1 m
OSIRIS-REx	2020	NavCam	Puntos característicos naturales	<1 m
Perseverance	2021	Cámara + IMU	Correlación con mosaico CTX	5 m
SLIM	2024	NavCam	Emparejamiento de cráteres	~10 m
IM-1 Odysseus	2024	LIDAR Doppler	Telémetros propios inoperativos	~1,5 km
Blue Ghost M1	2025	Cámaras	TRN visual + detección de peligros	<100 m
IM-2 Athena	2025	Telémetro láser	Medida degradada	~250 m
Argonaut (previsto)	2030	LIDAR LEIA	Relieve + detección de peligros	50 m (obj.)

De esta tabla salen dos lecturas. La primera es que la única misión que ha volado emparejamiento de cráteres como método principal es SLIM, y lo hizo hace poco más de dos años: es una técnica madura en el papel pero con un único precedente en vuelo, lo que la convierte en un objeto de estudio especialmente oportuno. Y la otra, que Apollo 12, con un ser humano y una retícula de plexiglás, sigue estando por delante de la mitad de la tabla.

1.3. Detección e identificación de cráteres

Usar cráteres como referencias exige resolver dos problemas encadenados que conviene no mezclar. El primero es la *detección*: encontrar en la imagen dónde hay un cráter y qué tamaño tiene. El segundo es la *identificación*: decidir cuál de los cráteres del catálogo es cada uno de los detectados. Son problemas de naturaleza distinta (el primero es de percepción, el segundo combinatorio y geométrico) y en la literatura los abordan comunidades diferentes, con métricas que no son comparables entre sí.

Antes de entrar en las técnicas hay que entender por qué detectar un cráter es difícil, porque no es evidente. Un cráter no es un objeto con textura propia, como una cara o un coche: es una forma del relieve, un cuenco, que solo se hace visible por cómo lo ilumina el Sol. De ahí se derivan cuatro dificultades que atraviesan todos los métodos. La apariencia depende radicalmente del ángulo solar, hasta el punto de que con el Sol casi cenital un cráter poco profundo desaparece. Los cráteres antiguos tienen el borde erosionado por impactos posteriores y son ambiguos incluso para un geólogo humano, lo que hace que los propios catálogos manuales discrepen entre sí. Los cráteres se solapan y se anidan, compartiendo bordes. Y coexisten en la misma imagen tamaños que difieren en tres órdenes de magnitud.

1.3.1. Familias de detección

Los métodos *basados en forma* atacan el problema desde la geometría pura: detectan bordes en la imagen con un operador tipo Canny y buscan después conjuntos de puntos de borde que se ajusten a una circunferencia o una elipse, normalmente con una transformada de Hough. Son transparentes, no necesitan datos de entrenamiento y dan directamente los parámetros geométricos que la navegación quiere. Fallan cuando el borde está incompleto, que es la situación habitual.

Los métodos *basados en iluminación* hacen justo lo contrario: en lugar de luchar contra el efecto del Sol, lo aprovechan. Un cráter iluminado oblicuamente produce una firma muy característica, un par de regiones contiguas clara y oscura orientadas según la dirección solar,

que se puede buscar por correlación con una plantilla. Son rápidos y robustos mientras la geometría de iluminación sea la esperada, y se derrumban cuando no lo es.

Los métodos *basados en relieve* eliminan el problema de raíz trabajando sobre un modelo digital de elevaciones en lugar de sobre una imagen. Un cráter en un mapa de alturas es una depresión, y eso no depende del Sol en absoluto. El precio es que hace falta el modelo digital, cuya resolución es mucho peor que la de las imágenes, y que la nave no lleva a bordo un altímetro capaz de generarlo en tiempo real.

Los métodos de *aprendizaje automático clásico* extraen del entorno de cada punto un conjunto de características diseñadas a mano y las pasan a un clasificador entrenado, tipo AdaBoost o máquina de vectores soporte. Mejoraron a los anteriores, pero generalizan mal fuera de la región donde se entrenaron, porque las características siguen siendo una elección humana.

Los métodos de *aprendizaje profundo* son los que han cambiado el panorama, y se dividen en dos ramas según lo que producen. La primera son las redes de segmentación, que etiquetan cada píxel con la probabilidad de pertenecer al borde de un cráter. El trabajo de referencia es DeepMoon (Silburt et al., 2019), una red en forma de U entrenada sobre el modelo digital lunar de LOLA y Kaguya, deliberadamente sobre relieve y no sobre imagen para eliminar la dependencia solar, que alcanzó una cobertura del 92 % frente a los catálogos humanos, casi duplicó el número de cráteres conocidos en su rango, y demostró transferirse a Mercurio sin reentrenar, prueba de que había aprendido «forma de cráter» y no «textura lunar». Su inconveniente para navegación es que la salida es una máscara de píxeles, no una lista de objetos, y convertir una en otra exige un posprocesamiento que es en sí mismo una fuente de error.

La segunda rama son los detectores de objetos, que devuelven directamente una lista de cajas. Los de una sola pasada, la familia YOLO, procesan la imagen completa en una única evaluación de la red, lo que los hace lo bastante rápidos para funcionar en tiempo real y lo bastante ligeros para plantearse embarcarlos. Los trabajos recientes sobre imágenes lunares reportan precisiones medias del orden del 96 % sobre conjuntos de prueba tomados casi en vertical (Tang et al., 2025). Es la familia que se emplea en este trabajo, y las razones de la elección se detallan en el Capítulo 3.

Esas cifras piden contexto, eso sí, porque es fácil que induzcan a error. Cuando se evalúan detectores sobre imágenes lunares *reales tomadas en oblicuo*, como las de la cámara de aterrizaje de Chang'e-5, el rendimiento cae drásticamente: el mejor resultado publicado sobre ese conjunto es un F1 del 63,1 % (Rodda et al., 2024). La diferencia entre ese 63,1 % y el 96 % anterior no mide la calidad de los algoritmos, sino la dificultad de los conjuntos de prueba. Es un aviso que este trabajo tiene muy presente, porque opera sobre terreno sintético.

1.3.2. El problema de la caja y la elipse

Hay una limitación de los detectores de objetos que merece mención aparte, porque reaparecerá en los resultados y en las líneas de trabajo futuro. Un cráter es una circunferencia sobre el terreno. Bajo proyección perspectiva desde una cámara con actitud arbitraria, esa circunferencia se proyecta en la imagen como una *elipse orientada*; esto no es una aproximación, es un resultado exacto de la geometría proyectiva. Sin embargo, un detector estándar tipo YOLO devuelve una caja alineada con los ejes de la imagen, definida por cuatro números: centro, anchura y altura. Una elipse necesita cinco, porque hace falta además el ángulo.

Las consecuencias son tres. Se pierde la orientación. Se sesga el diámetro estimado, y ese sesgo se propaga directamente a la altitud, que es precisamente lo que este trabajo quiere medir. Y, sobre todo, se impide usar los métodos de identificación teóricamente más sólidos, que trabajan con la matriz de la cónica completa. El efecto es pequeño en visión vertical y crece con la oblicuidad, lo que lo convierte en un problema justamente en la fase

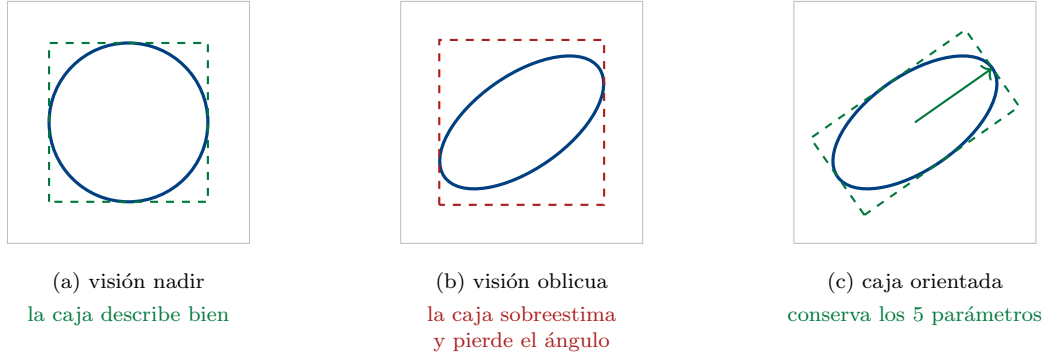


Figura 1.2: El problema de la representación de salida del detector. En visión nadir (a) el cráter se proyecta como una circunferencia y una caja alineada con los ejes lo describe sin pérdida. En visión oblicua (b) se proyecta como una elipse orientada: la caja envolvente conserva el centro pero sobreestima el tamaño y descarta por completo el ángulo. Una detección orientada (c) conservaría los cinco parámetros que la elipse necesita.

de descenso. Este trabajo opera en visión nadir y por tanto no sufre el caso peor, pero sí observa una manifestación medible del fenómeno, que se documenta en el Capítulo 5. La búsqueda bibliográfica realizada no ha encontrado ningún trabajo que aplique detección con cajas orientadas a cráteres, y solo un trabajo académico que aplique regresión directa de elipses a navegación lunar (Dong et al., 2021); es, por tanto, una línea de mejora concreta y poco explorada, y se recoge como tal en el Capítulo 6.

1.3.3. Identificación contra el catálogo

Detectar no basta. Cinco manchas circulares en una imagen no dicen nada por sí solas, porque todos los cráteres se parecen: la identidad no está en el cráter individual sino en el *patrón* que forman varios vecinos. Es literalmente el mismo principio de un sensor de estrellas, que no reconoce estrellas sueltas sino configuraciones.

La estrategia general consiste en tomar grupos de cráteres (lo habitual son tripletas) y calcular a partir de ellos magnitudes que no cambien al cambiar el punto de vista. Esas magnitudes forman un descriptor que se precalcula para todas las tripletas del catálogo y se guarda en una tabla ordenada. En vuelo se calcula el descriptor de la triplete observada y se busca en esa tabla, igual que se busca una palabra en un diccionario. Existen tres niveles de rigor. Los descriptores de semejanza usan ángulos y razones de longitudes, que solo son invariantes bajo escala y rotación, y por tanto valen en visión aproximadamente vertical con campo estrecho; es la línea de Hanak y colaboradores, cuyo trabajo canónico identificó cráteres en imágenes reales del Apollo 16 con un 89 % de aciertos cuando la imagen contenía cinco o más cráteres catalogados, y, dato más importante, rechazó correctamente el 100 % de las identificaciones erróneas mediante un test estadístico de falso emparejamiento (Hanak et al., 2009). Los descriptores mixtos combinan razones geométricas con información fotométrica de la región (Jin and Shao, 2024). Y en el nivel superior están los invariantes proyectivos de cónicas.

Ese último merece un párrafo propio porque es la referencia teórica del campo. Christian et al. (2021) presentan el primer tratamiento matemáticamente riguroso del problema: modelan cada cráter como una cónica y la superficie lunar como una cuádrica, y demuestran que para una tríada de cráteres existen exactamente siete invariantes proyectivos algebraicamente independientes, y que no hay más. Es un resultado de completitud, no una heurística. Su método verifica además la hipótesis reprojectando el contorno elíptico esperado y comparándolo con el observado, lo que le permite trabajar con solo tres cráteres donde otros

métodos necesitan cinco. La conexión con la Sección 1.3.2 es directa e importante: ese método exige la elipse completa de cada cráter, de modo que la limitación del detector de cajas no es un detalle estético, sino lo que impide emplear la formulación más sólida disponible.

Queda distinguir, por último, dos regímenes de operación con requisitos muy distintos. En el modo *lost in space* el sistema no tiene ninguna idea previa de dónde está y debe buscar en el catálogo global, lo que exige un descriptor globalmente discriminativo, una indexación eficiente y, sobre todo, un test de validación estadística, porque la ambigüedad crece con el tamaño del catálogo. En el modo de seguimiento se dispone de una estimación previa acotada y basta con buscar entre los cráteres que caen dentro del campo de visión proyectado, lo que es órdenes de magnitud más barato. Una misión real adquiere una vez en modo *lost in space* y después mantiene el seguimiento, reservando el primero como recuperación ante fallo. Este trabajo opera **siempre en modo *lost in space***, resolviendo cada fotograma desde cero sin usar el anterior. Es la condición más exigente posible y, de nuevo, es una elección consciente para que las cifras midan el método y no la inercia del seguimiento.

1.3.4. Catálogos y datos disponibles

Todo lo anterior descansa sobre la existencia de mapas de cráteres de referencia. Para la Luna, el catálogo global de Robbins (2019) contiene del orden de 1,3 millones de cráteres y es aproximadamente completo por encima de 1 km a 2 km de diámetro; está construido sobre imágenes de la cámara WAC del orbitador LRO y topografía del altímetro LOLA. Para los cráteres grandes existe el catálogo de Head et al. (2010), derivado directamente de altimetría LOLA. En cuanto a los datos primarios, la cámara WAC de LRO da cobertura global a 100 m por píxel y su cámara de ángulo estrecho llega a 0,5 m por píxel sobre franjas limitadas; el modelo digital de elevaciones que combina LOLA con la cámara de Kaguya alcanza unos 59 m por píxel entre las latitudes $\pm 60^\circ$ (Barker et al., 2016).

Este trabajo no emplea ninguno de esos catálogos, sino un terreno lunar sintético generado procedualmente, del que se conocen exactamente las posiciones, los diámetros y las profundidades de todos los cráteres. La razón es metodológica y se argumenta en el Capítulo 2: disponer de una verdad de referencia perfecta, sin la incertidumbre de un catálogo construido a mano, es lo que permite atribuir cada error observado a su causa real en lugar de a una discrepancia del catálogo.

1.4. Objetivos

Objetivo principal

Diseñar, implementar y validar experimentalmente un sistema completo de navegación óptica relativa al terreno que, partiendo únicamente de las imágenes de una cámara embarcada y de un mapa de cráteres almacenado en memoria, y sin recibir ninguna estimación previa de dónde se encuentra ni a qué altura vuela, sea capaz de determinar de forma autónoma la posición, la orientación y la altitud del vehículo, y caracterizar cuantitativamente tanto su precisión como las condiciones en las que deja de funcionar.

Objetivos específicos

Los objetivos específicos se plantearon desde el principio como hitos verificables, cada uno de los cuales debía quedar cerrado con una medida y no con una impresión. Son los siguientes:

1. Construir un entorno lunar sintético con verdad de referencia conocida, que permita renderizar secuencias de cámara sabiendo exactamente dónde estaba el vehículo y qué cráteres tenía delante en cada fotograma.

2. Obtener un detector de cráteres basado en aprendizaje profundo capaz de recuperar la práctica totalidad de los cráteres útiles de cada imagen, y determinar experimentalmente su punto de operación.
3. Diseñar un mapa embarcado y un motor de emparejamiento geométrico que identifique los cráteres observados en ese mapa sin conocimiento previo de la posición, y que rechace las hipótesis incorrectas en lugar de aceptarlas silenciosamente.
4. Demostrar la navegación completa sobre una trayectoria de crucero, primero en diferido y después en tiempo real sobre vídeo con la red neuronal funcionando dentro del bucle.
5. Extender el sistema a un descenso con rotación de la cámara en guiñada, llevarlo deliberadamente hasta el punto de fallo, y explicar con datos dónde está ese límite y por qué está ahí.

Los cinco se han cumplido. El quinto pide una aclaración, porque su enunciado suena extraño leído de corrido: el objetivo no era evitar el fallo, sino provocarlo de forma controlada y medir a qué altitud aparece.

1.5. Planificación y gestión del trabajo

1.5.1. Paquetes de trabajo

El trabajo se organizó en cinco paquetes, resumidos en la Tabla 1.2. La relación entre paquetes e hitos no es uno a uno, y hay que decirlo explícitamente, porque es una fuente habitual de confusión: hay hitos que cierran un paquete completo (H2 cierra PT2), hitos que requieren la convergencia de dos paquetes (H3 necesita PT3 terminado y la primera tarea de PT4), y paquetes que producen varios hitos sucesivos (PT4 produce H3, H4 y H5).

Tabla 1.2: Paquetes de trabajo, tareas e hito asociado.

Paquete	Tareas	Hito
PT1 Entorno sintético	T1.1 Generación procedural del terreno y del catálogo de cráteres	H1
	T1.2 Definición de cámaras, trayectorias y renderizado	
	T1.3 Exportación de etiquetas y de la trayectoria de referencia	
PT2 Detector de cráteres	T2.1 Preparación y ampliación del conjunto de entrenamiento	H2
	T2.2 Entrenamiento, afinado y reentrenamiento con rotación	
	T2.3 Caracterización del punto de operación	
PT3 Mapa y emparejamiento	T3.1 Diseño del descriptor invariante y de la tabla de tripletas	H3
	T3.2 Resolución de un fotograma y verificación de hipótesis	
	T3.3 Validación cruzada entre las implementaciones Python y MATLAB	
PT4 Demostradores	T4.1 Navegación en diferido sobre la secuencia de crucero	H3
	T4.2 Navegación en tiempo real sobre vídeo	H4
	T4.3 Descenso con rotación en guiñada	H5
PT5 Análisis	T5.1 Caracterización del límite de altitud	H6
	T5.2 Redacción de la memoria	

1.5.2. Hitos

H1 Entorno sintético operativo. Terreno lunar con relieve y textura, catálogo de cráteres de verdad conocida y secuencias renderizadas con trayectoria anotada fotograma a fotograma.

H2 Detector caracterizado. Cobertura del 99,4% de los cráteres útiles en condiciones nominales, con el punto de operación fijado experimentalmente.

H3 Navegación completa en diferido. Posición, orientación y altitud resueltas sobre la secuencia de cruce con error mediano inferior a 200 m.

H4 Demostrador en tiempo real. El sistema completo, con la red neuronal en el bucle, funcionando sobre vídeo y representando la solución en directo.

H5 Descenso con rotación caracterizado. Navegación durante un descenso con giro de la cámara, con el límite de altitud cuantificado y explicado.

H6 Memoria depositada.

1.5.3. Método de trabajo y tabla de aprendizajes

El trabajo se abordó de forma iterativa: cada decisión de diseño se planteó como una hipótesis y se resolvió con un experimento medido, no con un juicio. Esa disciplina tuvo una consecuencia que merece la pena documentar, y es que buena parte de lo que se intentó no funcionó. La Tabla 1.3 recoge las catorce decisiones técnicas principales del trabajo con su resultado.

Tabla 1.3: Decisiones técnicas ensayadas a lo largo del trabajo y su resultado. Una decisión se considera exitosa si sobrevivió a su validación experimental y forma parte del sistema final.

#	Decisión o hipótesis ensayada	Resultado
1	Afinar un detector preentrenado sobre los renders sintéticos	Éxito
2	Atribuir la pérdida de detecciones al límite de detecciones por imagen	Fallo
3	Atribuir la pérdida de tiempo del filtrado de solapamientos por tamaño de lote	Éxito
4	Etiquetar los vértices de la tripleta por su lado opuesto (orden canónico)	Éxito
5	Generar las tripletas del mapa por combinatoria exhaustiva	Fallo
6	Generarlas recorriendo cada cráter como el menor de la tripleta	Éxito
7	Podar las tripletas que no caben en un fotograma	Éxito
8	Verificar la hipótesis solo por proximidad de los cráteres	Fallo
9	Verificarla solo por coherencia relativa entre diámetros	Fallo
10	Acotar el factor mediano de escala entre observación y mapa	Fallo
11	Combinar ancla absoluta de diámetro, coherencia relativa y rango de altitud	Éxito
12	Reentrenar el detector con aumento de datos por rotación	Éxito
13	Densificar el mapa para rebajar el límite de altitud	Fallo
14	Ejecutar el sistema completo en tiempo real con la red en el bucle	Éxito
Tasa de éxito: 8 de 14		57,1 %

Ese 57,1% es, probablemente, el número más honesto de toda la memoria, y hay que leerlo bien. Las seis decisiones fallidas no fueron tiempo perdido: cuatro de ellas (las número 8, 9, 10 y 13) revelaron una propiedad del problema que no era evidente al empezar y que condiciona todo el diseño del sistema. Se enuncia aquí porque es el hilo conductor de los capítulos siguientes: **un emparejador geométrico no falla con un error, falla con una respuesta plausible y equivocada**. Cuando una restricción se relaja, el sistema no deja de dar solución: da una solución bonita, coherente consigo misma y falsa. Por eso cada relajación tuvo que compensarse con una restricción independiente, y por eso la verificación acabó necesitando tres condiciones simultáneas en lugar de una.

1.6. Relación con las asignaturas del grado

Este trabajo no aplica una asignatura concreta, sino que apoya cada bloque en una distinta, lo que refleja bastante bien el carácter transversal de la aeronavegación. Las contribuciones principales son las siguientes.

- **Sistemas de navegación y navegación aérea.** Aportan el marco conceptual completo: qué es una solución de navegación, por qué un sistema inercial deriva, qué significa una fijación de posición y por qué conviene combinar un sensor de posición absoluta con uno de posición relativa. Todo el planteamiento del Capítulo 1 descansa sobre ello.
- **Guiado, navegación y control.** Aporta la separación entre las capas de percepción y de estimación discutida en la Sección 1.2.6, y el criterio para decidir deliberadamente no implementar un filtro.
- **Mecánica clásica y del vuelo.** Aporta la comprensión del perfil de descenso y de las escalas de tiempo y de altitud realistas que se modelan en el entorno sintético.
- **Ampliación de matemáticas y álgebra lineal.** Es la base del motor de emparejamiento: transformaciones de semejanza, mínimos cuadrados, y la construcción de invariantes geométricos. La solución del ajuste se plantea, de hecho, en aritmética compleja.
- **Estadística.** Aporta el tratamiento de los resultados: medianas frente a medias en distribuciones con valores atípicos, percentiles, y el diseño de los barridos de parámetros.
- **Programación.** No aporta los lenguajes concretos que se han usado aquí, sino algo más duradero: la forma de enfrentarse a un problema. Descomponerlo en piezas que se puedan resolver y comprobar por separado, razonar sobre el coste de una solución antes de escribirla, y depurar de forma sistemática en lugar de a base de tanteo. Esa manera de pensar es la que se ha aplicado en todo el desarrollo, con independencia del entorno concreto en cada bloque.
- **Tratamiento de señales e imagen.** Aporta los fundamentos de formación de imagen, muestreo, proyección de cámara y los efectos de submuestreo que se encontraron al representar el terreno.

1.6.1. Habilidades adquiridas fuera del temario

Al margen de lo anterior, el trabajo obligó a adquirir cinco competencias que no forman parte del plan de estudios y que merecen mención por su peso real en el desarrollo. La primera son los propios lenguajes: Python para toda la parte de aprendizaje automático y MATLAB para el motor de navegación, ninguno de los dos impartido en el grado, además de hacerlos hablar entre sí, ya que la demostración en tiempo real necesita que MATLAB invoque la red neuronal escrita en Python y reciba sus resultados fotograma a fotograma. La segunda es el modelado y la generación procedural en Blender, incluido su interfaz de programación, empleado para construir el terreno, animar las cámaras y exportar la verdad de referencia. La tercera es el entrenamiento y afinado de redes neuronales de detección de objetos, con todo lo que conlleva de preparación de conjuntos de datos, aumento de datos y diagnóstico de un comportamiento anómalo dentro de una biblioteca de terceros. La cuarta es el diseño de experimentos: formular una hipótesis falsable, decidir de antemano qué mediría el éxito, y aceptar el resultado cuando contradice la intuición, algo que ocurrió repetidamente según muestra la Tabla 1.3. Y la quinta es la elaboración de documentación técnica con $\text{\LaTeX}$ y el control de versiones del código.

1.7. Declaración sobre el uso de inteligencia artificial

En la realización de este trabajo se han empleado asistentes conversacionales basados en modelos de lenguaje de gran tamaño. Se declara aquí en qué tareas se han usado y en cuáles

no, para que el alcance de esa ayuda quede explícito y pueda valorarse la autoría del trabajo.

El asistente se ha utilizado en tres tareas. La primera es la **redacción y revisión de esta memoria**: se ha empleado para dar forma a borradores a partir de guiones y notas propias, para revisar la claridad y la corrección gramatical del texto y para uniformar la terminología entre capítulos. El contenido técnico de cada párrafo, las cifras, las decisiones que se describen y la interpretación de los resultados son del autor; el texto resultante ha sido leído, corregido y reescrito por él, que responde de su contenido. La segunda es la **asistencia en programación**: depuración de errores, vectorización de bucles que resultaban inviables por tiempo de cálculo, y sugerencias de reorganización del código ya escrito. La tercera es la **discusión de alternativas de diseño**, en el mismo papel en el que se consulta documentación o se pregunta a un compañero: contrastar si un descriptor invariante a semejanza necesita tres o cuatro cráteres, o si conviene una malla desplazada frente a un modificador de desplazamiento.

No se ha empleado para las decisiones que constituyen el trabajo. La formulación del problema, la elección de navegar por emparejamiento de tripletas de cráteres, el diseño del descriptor invariante de cuatro componentes, la estrategia de entrenamiento en tres etapas del detector, el diseño de los ensayos y la determinación de los parámetros de operación son propios. En particular, no se ha seguido una práctica de *vibe coding*: no hay en el trabajo código aceptado sin leer ni ejecutado sin verificar. Todos los programas han sido comprendidos, ejecutados y validados por el autor, y varios de los errores que se documentan en esta memoria se detectaron precisamente al no dar por buena una salida que parecía razonable. Son ejemplos el aplanamiento de la superficie por un guardado de imagen que escribía archivos en blanco sin emitir ningún error (Sección 2.5.1), el error de escala del 31,6 % causado por un encuadre del mapa de referencia que no coincidía con el terreno (Sección 2.6.3), el vaciado silencioso de detecciones por el límite de tiempo de la supresión de no máximos (Sección 3.7) y el sesgo sistemático de 2,7 km en altitud por no referir el mapa a su cota media. En los cuatro casos el código se ejecutaba sin errores y devolvía un resultado plausible; lo que los descubrió fue contrastar ese resultado contra una magnitud conocida de antemano.

Se ha optado además por no delegar en el asistente ninguna comprobación numérica: todas las cifras de esta memoria proceden de los ficheros de resultados que generan los programas, y el Anexo D detalla la secuencia exacta que hay que ejecutar para reproducirlas.

1.8. Estructura del trabajo

El Capítulo 1 plantea el problema de la navegación autónoma sin sistema de posicionamiento global, revisa por familias las técnicas de navegación relativa al terreno y de detección e identificación de cráteres, y fija los objetivos y la planificación del trabajo. El Capítulo 2 describe la generación del entorno lunar sintético: los requisitos que impone la metodología, la justificación de las herramientas empleadas, el modelo procedimental de superficie, el modelo de cámara, las trayectorias y el procedimiento de autoetiquetado que produce la verdad de referencia. El Capítulo 3 trata la detección de cráteres: la elección de la familia de detectores, el entrenamiento en tres etapas propias, la determinación del punto de operación y el protocolo con el que se valida. El Capítulo 4 desarrolla el posicionamiento relativo al terreno: el diseño del mapa embarcado, el descriptor invariante de tripletas, el ajuste de la transformación de semejanza, la verificación de las hipótesis y su validación. El Capítulo 5 presenta y analiza los resultados de los tres ensayos realizados (navegación en diferido, navegación en tiempo real y descenso con rotación), cuantifica la precisión alcanzada y caracteriza el límite de altitud en el que el sistema deja de resolver. El Capítulo 6 recoge las conclusiones del trabajo y propone las líneas de continuación, entre ellas la sustitución de la detección por cajas alineadas con los ejes por una detección de elipses orientadas. Los

anexos recogen el código desarrollado y las tablas de resultados completas.

Capítulo 2

Generación del entorno lunar sintético

Este capítulo explica cómo se ha construido el terreno lunar sobre el que se prueba el sistema y cómo se obtienen de él las imágenes y la trayectoria real. El entorno no es el objetivo del trabajo, sino la herramienta que lo hace posible: el detector del Capítulo 3 y el algoritmo de posicionamiento del Capítulo 4 necesitan imágenes de las que se sepa exactamente qué contienen y desde dónde se han tomado, y eso es lo que aporta este bloque.

2.1. ¿Qué necesita el sistema?

Antes de decidir cómo generar las imágenes hubo que concretar qué tenían que aportar. Son cinco cosas:

1. **Un catálogo de cráteres exacto**, con la posición y el diámetro de cada uno. El mapa embarcado se construye a partir de él, y cualquier error del catálogo pasaría directamente a la posición estimada.
2. **La trayectoria real de la cámara** en cada fotograma. Sin ella no se puede calcular el error del sistema.
3. **Muchas imágenes etiquetadas**, para reentrenar el detector con imágenes como las que va a ver. Etiquetar a mano cientos de imágenes con cuatrocientos cráteres cada una no es viable.
4. **Control sobre las condiciones**: altitud, iluminación y orientación de la cámara, para poder estudiar cada una por separado.
5. **Reproducibilidad**. Todo el terreno sale de una semilla fija, así que ejecutando los mismos scripts se obtiene exactamente el mismo resultado.

La Figura 2.1 muestra cómo encaja el entorno en el sistema completo. Blender genera las imágenes, YOLO detecta los cráteres de cada una y entrega sus posiciones en píxeles, y MATLAB las compara con el mapa embarcado y calcula la posición, la guiñada y la altitud. La trayectoria real solo se usa al final, para medir el error.

Hay dos decisiones de diseño que afectan a todo el trabajo. La primera es que al sistema no se le da la altitud: MATLAB solo recibe coordenadas en píxeles y tiene que deducir la altitud a partir de la escala del emparejamiento. La segunda es que cada fotograma se resuelve por separado, sin filtro y sin usar lo que se calculó en el fotograma anterior, que es el caso que la Sección 1.3.3 llamaba *lost in space*. Un sistema real usaría un filtro y obtendría mejores cifras, pero así cada error que se mide se debe al método y no a la estimación anterior.

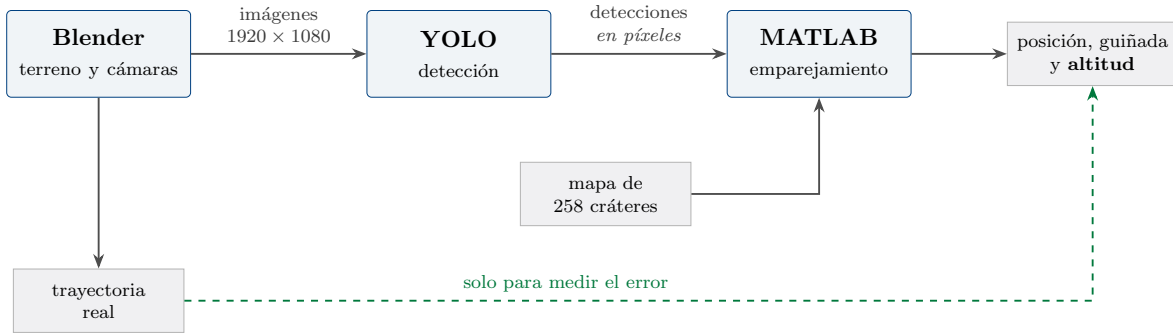


Figura 2.1: Arquitectura del sistema. El flujo principal va de izquierda a derecha y no se realimenta: cada fotograma se resuelve por separado. La trayectoria real (en verde) no entra nunca en el cálculo; solo se usa al final para medir el error.

2.2. ¿Por qué un entorno sintético y por qué Blender?

Existían datos reales que se podrían haber usado: el mosaico global de la cámara de gran angular del *Lunar Reconnaissance Orbiter* (LRO), a 100 m por píxel, los modelos de elevación del altímetro LOLA y catálogos como el de Robbins (2019), con más de un millón de cráteres. Se descartaron por dos razones. La primera es que no existe ninguna secuencia pública de descenso lunar con imágenes y la trayectoria real anotada, así que no habría forma de medir el error del sistema. La segunda es que los catálogos reales tienen errores de posición del orden del kilómetro en los cráteres pequeños, y ese error se sumaría al del método sin poder separarlos. En un terreno generado, en cambio, el catálogo es exacto, porque es el mismo que se ha usado para construirlo.

Esto tiene un coste. Un render no es una fotografía: no tiene ruido de sensor ni desenfoque por movimiento, y la superficie se ilumina como un difusor ideal, cuando el regolito real devuelve más luz hacia la dirección de la que viene. Por eso los resultados del Capítulo 5 miden el método en condiciones controladas y no la precisión que tendría el sistema en vuelo. Probarlo con imágenes reales es la primera línea de trabajo futuro (Capítulo 6).

Para generar el entorno se valoraron las cuatro opciones de la Tabla 2.1.

Tabla 2.1: Herramientas consideradas para generar el entorno.

Herramienta	A favor	En contra
Unreal Engine o Unity	Tiempo real y editor cómodo	Iluminación aproximada y curvas de tono cinematográficas; la red podría aprender detalles que una cámara real no tiene
PANGU (Univ. de Dundee)	Es el simulador que usa la industria para este problema	Licencia comercial, sin acceso para un TFG
Trazador de rayos propio	Control total	Programarlo sería un trabajo tan grande como este
Blender	Trazado de rayos físico (Cycles), API de Python y software libre	Render lento

Se eligió Blender. Su motor Cycles calcula las sombras resolviendo el transporte de la luz, y las sombras que proyectan los bordes de los cráteres son justo lo que el detector aprende a reconocer. Además, toda la escena se puede construir desde Python con su API, **bpy**: el terreno, el material, el sol, las cámaras y la animación salen de scripts, sin ningún paso a mano, y por eso la generación es reproducible. Al ser libre, cualquiera puede repetirla. El inconveniente es el tiempo de render: los 700 fotogramas de la secuencia de crucero tardan varias horas en un ordenador doméstico, y eso limita el tamaño del conjunto de entrenamiento.

2.3. Unidades y sistemas de referencia

En toda la escena se usa la equivalencia

$$1 \text{ unidad de Blender} = 1 \text{ km.} \quad (2.1)$$

Se trabaja en kilómetros y no en metros por un problema de precisión. Blender guarda las coordenadas en coma flotante de precisión simple, con unos siete dígitos significativos. En metros, con un terreno de 840 km y la cámara a 170 km, las coordenadas tienen seis cifras y se pierde la diferencia entre dos profundidades cercanas; en las primeras pruebas eso se veía como una retícula oscura y picos sobre la superficie. En kilómetros todo el escenario cabe entre 0 y 300 unidades y el problema desaparece.

Se usan cuatro sistemas de referencia:

- **Mundo de Blender** (x, y, z) , con el origen en el centro del terreno, x hacia el este, y hacia el norte y z hacia arriba. En él se sitúan la cámara y los cráteres.
- **Mapa** (x_m, y_m) , con el origen en la esquina suroeste del terreno, de 0 a 840 km en x_m y de 0 a 630 km en y_m . Es el del catálogo y el mapa embarcado. Se pasa al del mundo con una traslación: $x = x_m - 420$, $y = y_m - 315$.
- **Cámara**, con el origen en el centro óptico. Siguiendo el convenio de Blender, la cámara mira hacia $-z$, y x e y apuntan a la derecha y hacia arriba de la imagen.
- **Imagen** (u, v) , normalizado entre 0 y 1 en los dos ejes y con v medido desde arriba. Es el que usa YOLO en sus etiquetas.

Entre el sistema del mapa, en el que y crece hacia el norte, y el de la imagen, en el que v crece hacia abajo, hay un cambio de mano: pasar de uno a otro es una reflexión, no un giro. Si no se tiene en cuenta, la guiñada estimada sale con el signo cambiado, y el error no se nota en los fotogramas con guiñada cercana a 0° o 180° . El Capítulo 4 lo trata en detalle.

2.4. Generación del terreno

El terreno se genera con un script propio, `terreno_lunar.py`, que solo usa NUMPY y no depende de Blender, así que se pudo probar y depurar sin abrirlo. A partir de una semilla fija produce el mapa de alturas **H**, el mapa de albedo **A** y el catálogo con todos los cráteres. Después, tres scripts de Blender montan la escena, animan la cámara y generan las etiquetas (Figura 2.2).

La superficie se construye sumando siete capas, siempre en el mismo orden (Tabla 2.2). El orden importa por lo mismo que en la Luna: primero existe el relieve, después la lava rellena las cuencas y luego caen los meteoritos, los más recientes encima de los más antiguos.

2.4.1. Ruido fractal

Las capas 1, 6 y 7 usan ruido fractal, o movimiento browniano fraccionario (fBm), que es la forma habitual de fabricar un relieve con aspecto natural. El motivo de usarlo es que el espectro de potencia de una superficie planetaria decae aproximadamente como una potencia de la frecuencia espacial (Shepard et al., 2001), y el fBm reproduce ese comportamiento por construcción. Se sortean alturas al azar en los nodos de una rejilla gruesa y se interpolan entre ellos, lo que da ondulaciones grandes. Después se repite con una rejilla el doble de fina y la mitad de amplitud, se suma, y así varias veces. Cada repetición es una *octava*:

$$F(x, y) = \frac{1}{\sum_{k=0}^{N-1} g^k} \sum_{k=0}^{N-1} g^k V_k(\lambda^k x, \lambda^k y), \quad (2.2)$$

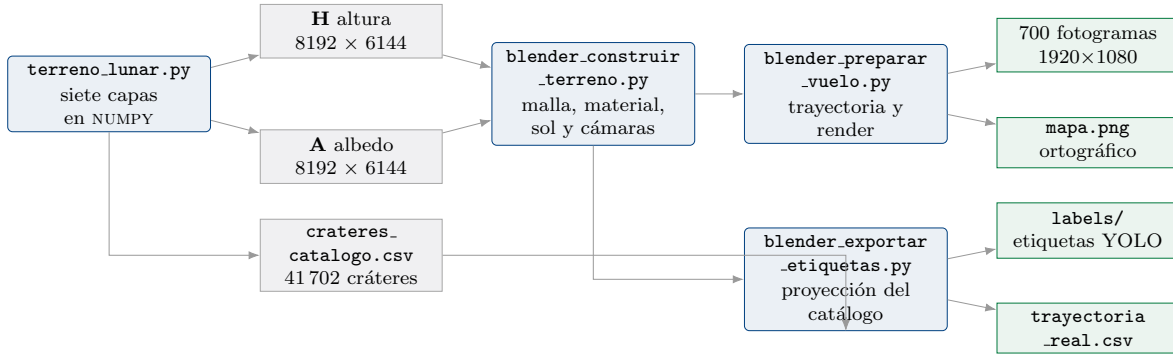


Figura 2.2: Cadena de generación del entorno sintético. El generador de superficie es código NUMPY independiente de Blender y produce dos matrices y un catálogo; los tres *scripts* de Blender montan la escena, definen la trayectoria y proyectan el catálogo para obtener las etiquetas. Todo lo que sale a la derecha, salvo los fotogramas, es verdad de referencia exacta: no hay ningún etiquetado manual en la cadena.

Tabla 2.2: Capas del modelo de terreno, en el orden en que se aplican. La amplitud es el relieve vertical que aporta cada capa.

#	Capa	Modelo y parámetros	Amplitud
1	Topografía regional	fBm, 3 celdas base (210 km), 7 octavas, lacunaridad 2, ganancia 0,5	2200 m
2	Mares basálticos	12 cuencas elípticas de semieje 102–189 km con el borde deformado; hundimiento y aplanamiento	≈ -680 m
3	Crestas de arruga	4–7 polilíneas, perfil $(1 - u^2)^2$, solo dentro de los mares	120–320 m
4	Población de cráteres	400 000 sorteos, ley de potencias $N(> D) \propto D^{-2,1}$, $D \in [0,28; 45]$ km	según D
5	Cráteres de referencia	180 cráteres frescos, $D \in [16; 32]$ km, separación mínima 34 km	según D
5b	Lluvia final	180 000 cráteres de $D \in [0,28; 2,5]$ km, estampados después de los de referencia	según D
6	Regolito	dos fBm finos, 192 y 768 celdas base	130 y 35 m
7	Albedo	base 0,42; mares $-0,27$; dos fBm; halos y rayos de eyecta	—

donde V_k es el ruido interpolado de la octava k , λ la *lacunaridad* (cuánto se multiplica la frecuencia de una octava a la siguiente) y g la *ganancia* (cuánto se multiplica la amplitud). Con $\lambda = 2$ y $g = 0,5$ los rasgos grandes son altos y los pequeños son bajos. El espectro resultante decae como f^{-2} , dentro del intervalo que Shepard et al. (2001) miden en terrenos naturales; si todas las octavas pesaran lo mismo, el terreno parecería ruido. El denominador normaliza la suma. En la capa 1 la rejilla más gruesa tiene 3×4 celdas sobre los 630×840 km del terreno, es decir, celdas de 210 km, y la séptima octava llega a celdas de 3,3 km.

Dentro de cada celda no se interpola linealmente, sino con la función *smoothstep*,

$$s(t) = t^2(3 - 2t), \quad t \in [0, 1], \quad (2.3)$$

que tiene derivada nula en los extremos. Con interpolación lineal la pendiente cambia de golpe en el borde de cada celda, y con el sol rasante la rejilla se veía en el render como una malla de líneas rectas. Esto se repite en todo el capítulo: con iluminación rasante lo que se ve no es tanto la altura como la pendiente, así que cualquier salto de pendiente aparece en la imagen.

2.4.2. Tamaño de los cráteres

En la Luna hay muchísimos cráteres pequeños y muy pocos grandes, en una proporción que, en un intervalo amplio de diámetros, se aproxima bien por una ley de potencias (Neukum et al., 2001; Robbins, 2019),

$$N(> D) \propto D^{-b}, \quad (2.4)$$

con $b \approx 2$ para la Luna. Se ha usado $b = 2,1$, lo que quiere decir que hay $2^{2,1} \approx 4,3$ veces más cráteres de más de 5 km que de más de 10 km. Para sortear diámetros con esta distribución se usa su función de distribución acumulada, que entre $D_{\min}$ y $D_{\max}$ es

$$P(D \leq d) = \frac{D_{\min}^{-b} - d^{-b}}{D_{\min}^{-b} - D_{\max}^{-b}}. \quad (2.5)$$

Despejando d en función de un número aleatorio uniforme u entre 0 y 1 se obtiene la expresión que usa el generador:

$$D(u) = \left[D_{\min}^{-b} - u \left(D_{\min}^{-b} - D_{\max}^{-b} \right) \right]^{-1/b}. \quad (2.6)$$

Con $D_{\min} = 0,28$ km, $D_{\max} = 45$ km y 400 000 sorteos sobre $529\,200$ km² sale una densidad de 0,75 cráteres por km². La mayoría son diminutos: la mediana es de 0,39 km y solo el 11 % supera los 0,8 km. Los que miden menos de 1,2 píxeles de radio no se estampan.

A cada cráter se le sortea además una edad a entre 0 y 1, y se estampan de más viejo a más joven, de forma que los recientes tapan en parte a los antiguos. De la edad sale la *frescura* ϕ , que indica lo bien conservado que está:

$$\phi = \text{clip}(1 - a + \varepsilon, 0,05, 1), \quad \varepsilon \sim \mathcal{N}(0, 0,12^2). \quad (2.7)$$

El término ε añade algo de dispersión, porque en la realidad lo conservado que está un cráter no depende solo de su edad.

En el catálogo solo se guardan los cráteres de $D \geq 0,8$ km. A 130 km de altura uno más pequeño tendría una caja de menos de 12 píxeles, y el detector no podría aprenderlo. El catálogo final tiene 41 702 cráteres, 180 de ellos de referencia.

2.4.3. Forma de un cráter

Cada cráter se añade al mapa de alturas como un perfil con simetría de revolución. Sea r la distancia al centro dividida por el radio del borde, D el diámetro en kilómetros y ϕ la frescura.

La profundidad sale de las relaciones que midió Pike (1977) sobre cráteres lunares:

$$d_0(D) = \begin{cases} 0,196 D^{1,010} & \text{si } D < D_t \text{ (simples),} \\ 1,044 D^{0,301} & \text{si } D \geq D_t \text{ (complejos),} \end{cases} \quad (2.8)$$

con d_0 y D en kilómetros y $D_t = 18$ km el diámetro a partir del cual un cráter deja de ser simple y pasa a ser complejo (en la Luna ocurre entre 15 y 20 km). En los cráteres simples la profundidad es más o menos la quinta parte del diámetro; en los complejos crece mucho más despacio, porque las paredes se derrumban y el fondo se ensancha. Al principio el generador usaba $d/D = 0,2$ para todos los tamaños, y un cráter de 20 km salía con 4 km de profundidad, el doble de lo real; sus sombras eran manchas negras sin ningún detalle. Con la Ecuación (2.8) esa profundidad baja a 2,6 km.

La frescura reduce la profundidad y la altura del borde:

$$d = d_0 (0,25 + 0,75 \phi), \quad h_b = 0,032 D (0,15 + 0,85 \phi). \quad (2.9)$$

La altura del borde recién formado, el 3,2 % del diámetro, está dentro del intervalo habitual en cráteres de impacto (Melosh, 1989). El borde se desgasta más deprisa de lo que se rellena el cuenco, así que un cráter muy viejo conserva una depresión poco profunda pero casi no tiene borde. Como con sol rasante un cráter se ve sobre todo por la sombra de su borde, los cráteres viejos son difíciles de detectar, igual que en la realidad.

El interior de un **cráter simple** es un cuenco parabólico:

$$z(r) = -d + (d + h_b) r^2, \quad r \leq 1. \quad (2.10)$$

Un **cráter complejo** tiene el fondo plano, paredes con terrazas y, si mide más de 25 km, un pico central. Con $r_f = 0,40 + 0,10 (1 - \phi)$ el radio del fondo (los cráteres viejos están más rellenos) y

$$t = \text{clip}\left(\frac{r - r_f}{1 - r_f}, 0, 1\right), \quad s_5(t) = t^3 (6t^2 - 15t + 10), \quad (2.11)$$

el perfil es

$$z(r) = \underbrace{-d + (d + h_b) s_5(t)}_{\text{fondo y pared}} \underbrace{- 0,05 d \text{ clip}\left(1 - \frac{r}{r_f}, 0, 1\right)^2}_{\text{concavidad}} + \underbrace{0,010 d \phi \sin(4\pi t)}_{\text{terrazas}} + \underbrace{0,30 d \phi s(u)}_{\text{pico central}}, \quad (2.12)$$

con $u = \text{clip}(1 - r/0,22, 0, 1)$. Las terrazas solo se añaden si $\phi > 0,45$, y el pico si además $D > 25$ km. La transición entre el fondo y la pared usa s_5 , la *smootherstep*, que además de la primera derivada anula también la segunda. Con la *smoothstep* normal quedaba un salto de curvatura en $r = r_f$ que en el render se veía como un círculo perfecto en el borde del fondo.

Fuera del borde se añade el **manto de eyecta**, el material expulsado en el impacto, cuyo espesor decae con la distancia siguiendo una ley de potencias (Melosh, 1989):

$$z(r) = h_b r^{-3,8}, \quad r > 1. \quad (2.13)$$

Se corta en $r = 2,2$ con una rampa suave; cortarlo de golpe dejaba un escalón de decenas de metros que se veía como una red de circunferencias finas por todo el mapa. Además, la eyecta real no es lisa sino que forma estrías radiales, así que su altura y el contorno del borde se modulan con varias ondulaciones en el ángulo. Sin eso, cada cráter quedaba rodeado de un anillo uniforme que se notaba artificial.

La Figura 2.3 dibuja los tres casos típicos con las mismas ecuaciones y los mismos valores que usa el generador.

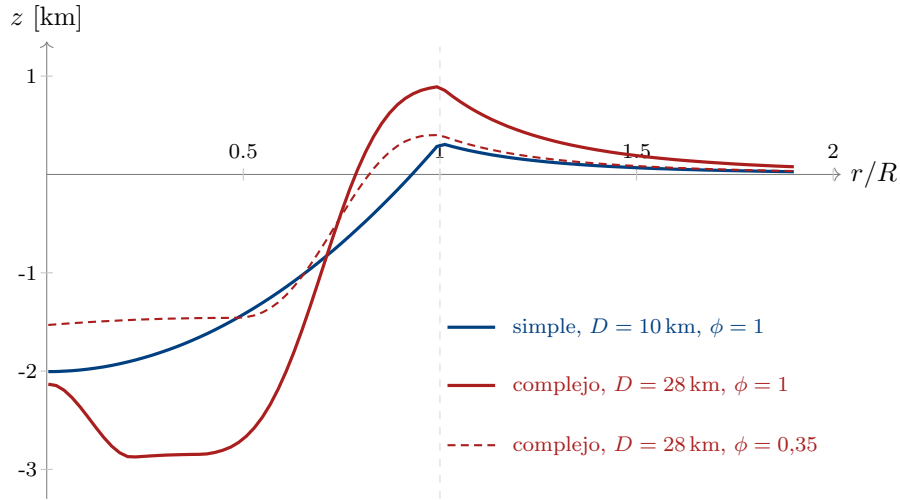


Figura 2.3: Perfiles radiales generados por la Ecuaciones (2.10), (2.12) y (2.13), dibujados con los mismos valores que emplea el generador. El eje horizontal es la distancia al centro normalizada por el radio del borde. Obsérvese el fondo plano y el pico central del cráter complejo, el borde elevado en $r/R = 1$ y la caída $r^{-3,8}$ del manto de eyecta. La degradación por edad ($\phi = 0,35$) reduce la profundidad a la mitad y achata el borde, que es el efecto que hace que los cráteres viejos sean poco detectables con iluminación rasante.

2.4.4. Mares, crestas de arruga y albedo

Los mares son las llanuras oscuras de la Luna: cuencas que se inundaron de lava, que al enfriarse dejó un suelo bajo y liso. Se modelan como 12 elipses con semiejes de 102 a 189 km. Para que el borde no sea una elipse perfecta, la distancia al centro se deforma con dos ondulaciones:

$$\rho = \frac{\sqrt{\xi^2 + \eta^2}}{1 + 0,16 \sin(3\theta + \varphi_3) + 0,09 \sin(7\theta + \varphi_7)}, \quad (2.14)$$

donde (ξ, η) son las coordenadas locales divididas por los semiejes y θ el ángulo. Con ρ se construye una máscara $\mu = s((1, 10 - \rho)/0,30)$, que vale 1 dentro del mar, 0 fuera, y pasa suavemente de uno a otro en el borde. La altura se modifica en tres pasos seguidos, que juntos dan

$$h' = (h - 900\mu)(1 - 0,8\mu) - 500\mu \quad (\text{en metros}). \quad (2.15)$$

El paso del medio es el importante: no solo hunde el terreno, sino que lo aplana al 20 %, que es lo que hace la lava al rellenar la cuenca. En el centro de un mar, con $\mu = 1$, queda $h' = 0,2h - 680$, así que el fondo acaba de media 680 m por debajo de las tierras altas. Además, dentro de los mares se elimina el 80 % de los cráteres, porque su superficie es más joven y ha recibido menos impactos.

Dentro de los mares también hay **crestas de arruga**: lomas largas y sinuosas que se formaron al contraerse la lava. Se generan como líneas quebradas de cinco a diez tramos con giros aleatorios, y sobre cada tramo se levanta el perfil

$$z = A(1 - u^2)^2 \mu(x, y), \quad u = \text{clip}\left(\frac{\delta}{w}, 0, 1\right), \quad (2.16)$$

con δ la distancia al tramo, w su semianchura y A entre 120 y 320 m. Al multiplicar por la máscara μ , las crestas solo aparecen donde hay mar.

Por último, el **albedo** indica cuánta luz refleja cada punto. El albedo normal medido por el altímetro LOLA a 1064 nm es de unos 0,33 en las tierras altas y de unos 0,15 en los mares (Lucey et al., 2014). Conviene no confundir ese valor con el albedo geométrico del disco lunar completo en el visible, que es de 0,12: son magnitudes distintas, y la segunda promedia toda la

esfera. El generador parte de 0,42 en las tierras altas, algo por encima del valor medido, para aumentar el contraste con los mares, a los que resta $0,27\mu$ y deja en torno a 0,15. Encima se añaden dos capas de fBm de poca amplitud y, alrededor de los cráteres frescos ($\phi > 0,90$ y $D > 6$ km), un halo claro y rayos: las estrías brillantes de material reciente. Los rayos se generan con

$$R(\theta) = \text{clip}\left(1,6 \sum_{k \in \mathcal{K}} \frac{\sin(k\theta + \varphi_k)}{\sqrt{k}}, 0, 1\right)^2, \quad (2.17)$$

sumando tres ondulaciones $\mathcal{K}$ de frecuencia alta, sorteadas entre 6 y 25. El recorte y el cuadrado hacen que salgan estrechos y discontinuos, como los reales. En una primera versión los rayos empezaban en el centro del cráter y cada uno aparecía con una estrella dibujada dentro; se corrigió haciendo que empiecen en el borde, que es donde nace la eyecta.

El mapa de alturas tiene 8192×6144 muestras, 102 m por muestra. Es del orden de lo que resuelve la cámara, entre 72 y 102 m por píxel según la altura (Sección 2.6), así que más resolución no se vería en las imágenes y solo gastaría memoria. Aun así, cada matriz ocupa 192 MiB, y el ruido fractal se calcula por bandas de 1024 filas para no agotar la memoria del ordenador.

2.4.5. ¿De dónde salen los parámetros?

Los valores que aparecen en la Tabla 2.2 y en las ecuaciones anteriores son de tres clases distintas, y conviene separarlas.

Unos pocos están **tomados de la literatura**: las relaciones profundidad-diámetro y el diámetro de transición entre cráteres simples y complejos (Pike, 1977), el exponente de la distribución de tamaños (Neukum et al., 2001; Robbins, 2019), la altura del borde y la caída del manto de eyecta (Melosh, 1989), la forma del espectro de potencia del relieve (Shepard et al., 2001), los valores de albedo (Lucey et al., 2014) y la función de transferencia sRGB (International Electrotechnical Commission, 1999).

Otros están **deducidos de una restricción del propio trabajo**, y se pueden recalcular a partir de ella. La resolución del mapa de alturas, 102 m por muestra, sale de igualar la resolución de la cámara; el diámetro mínimo del catálogo, 0,8 km, de exigir una caja de al menos 12 píxeles a 130 km de altura; el número de cráteres de fondo, de mantener la densidad de 0,75 cráteres por km^2 ; la intensidad del sol, de la Ecuación (2.21); y los parámetros de la trayectoria, de la búsqueda descrita en la Sección 2.7.

El resto son **parámetros de forma elegidos para este trabajo**: la elevación solar, el umbral de frescura, el número y el tamaño de los cráteres de referencia, los coeficientes de degradación, el radio del fondo de los cráteres complejos, la amplitud de las terrazas y de las crestas, o la profundidad de los mares. Se ajustaron hasta que el renderizado tuviera el aspecto y la estadística de una superficie lunar, se fijaron antes de empezar los ensayos y no se han modificado después. Todos están recogidos, con la semilla del generador, en el Anexo C.

Esta última clase no compromete las medidas del Capítulo 5. El terreno solo tiene que cumplir dos condiciones: ser lo bastante exigente para que la prueba sea representativa, y ser exactamente conocido. Lo segundo lo garantiza la forma de construirlo, no su realismo, porque el catálogo con el que se mide el error es el mismo que se ha usado para generar el terreno. Cambiar la elevación del sol o la profundidad de los cráteres cambiaría lo bien que detecta la red, pero no la validez de la medida.

2.5. Montaje de la escena en Blender

El script `blender_construir_terreno.py` llama al generador y monta la escena: la malla del terreno, su material, el sol y las dos cámaras.

2.5.1. La malla del terreno

Lo habitual para llevar un mapa de alturas a Blender es crear un plano subdividido y aplicarle un modificador *Displace* que lea las alturas de una imagen. Es lo que se hizo al principio, y falló sin dar ningún error: el render salía completamente plano, sin un solo cráter. Ese método depende de tres cosas que tienen que estar bien a la vez (la imagen guardada, las coordenadas de textura y el modificador), y si falla cualquiera el resultado es un plano liso.

La causa resultó ser el guardado de la imagen. La función de Blender que se estaba usando escribía los archivos en blanco sin avisar. Se descubrió porque los dos PNG ocupaban exactamente 110 KiB cada uno, cuando una imagen de 4096×3072 a 16 bits debería ocupar entre 10 y 20 MiB; un PNG en blanco comprime a casi nada.

Se hicieron dos cambios. Los PNG pasaron a escribirse a mano, con **zlib** y **struct**, que para una imagen en escala de grises es sencillo. Y, sobre todo, se quitó el modificador: la malla se construye poniendo directamente en la coordenada z de cada vértice la altura del mapa,

$$z_{ij} = \frac{H(f_j, c_i)}{1000}, \quad (2.18)$$

donde f_j y c_i son la fila y la columna del mapa de alturas que corresponden al vértice, y el factor $1/1000$ pasa de metros a kilómetros. Así no hay nada que encajar, y el script avisa si el relieve de la malla sale de menos de 10 m.

La malla tiene 3000×2250 vértices, unos 6,75 millones, uno cada 280 m. Se crea con **foreach_set**, que copia los datos de NUMPY directamente; la función habitual, **from_pydata**, necesita listas de Python y con este tamaño tardaba varios minutos y ocupaba casi 1 GB.

2.5.2. Geometría y bump

Con un vértice cada 280 m la malla no puede representar detalles más pequeños. Por eso el relieve se separa en dos partes:

$$H = \underbrace{\bar{H}}_{\text{geometría}} + \underbrace{(H - \bar{H})}_{\text{bump}}, \quad (2.19)$$

donde $\bar{H}$ es el mapa de alturas promediado en bloques del tamaño de una celda de la malla. $\bar{H}$ va a los vértices, y el resto, $H - \bar{H}$, se guarda como **detalle.png** y se conecta a un nodo *Bump* del material, que cambia cómo se ilumina la superficie sin mover la geometría. La suma de las dos partes es exactamente el mapa original. Se promedia en lugar de tomar una muestra de cada bloque porque, si no, el ruido fino aparece como *moiré*.

La limitación es que el bump cambia el sombreado pero no proyecta sombras. A las altitudes de trabajo, con 72 a 102 m por píxel, no se nota; en un descenso hasta pocos kilómetros sí se notaría.

2.5.3. Material, iluminación y render

El regolito se modela como un material difuso rugoso (Oren-Nayar, rugosidad 0,9), sin brillo especular, porque una superficie de polvo sin atmósfera no tiene reflejos.

La textura de albedo se carga con el perfil de color **Non-Color**. Los valores del generador son reflectancias lineales, y si la textura se carga como sRGB, que es lo que Blender hace por defecto, les aplica la función de transferencia inversa de esa norma (International Electrotechnical Commission, 1999) y los vuelve a linealizar:

$$\left(\frac{0,42 + 0,055}{1,055} \right)^{2,4} \approx 0,147. \quad (2.20)$$

Todo el render salía casi tres veces más oscuro, sin ninguna pista de por qué.

La luz es un sol con su tamaño angular real, $0,53^\circ$, a 28° sobre el horizonte y con azimut 315° . La elevación baja es la decisión más importante de la iluminación. Con el sol alto no hay sombras y los cráteres solo se distinguen por el color; con el sol rasante cada cráter proyecta una sombra que depende de su profundidad, y eso es lo que el detector aprende a reconocer. Además, es la condición de la mayoría de las imágenes orbitales reales con las que se hizo la primera etapa del entrenamiento. No hay luz ambiente: el fondo es negro y las sombras solo reciben la luz que rebota en el propio terreno.

El render se hace con Cycles a 64 muestras por píxel y con reducción de ruido. Con menos muestras aparece ruido dentro de las sombras, que el detector podría aprender; con más, los 700 fotogramas tardarían demasiado. En la gestión de color se cambia la curva AgX, que Blender aplica por defecto y que comprime las luces para dar un aspecto cinematográfico, por la vista **Standard**, para que el valor de cada píxel dependa directamente de la luz calculada. Con esta vista la intensidad del sol se ajusta para que las tierras altas queden en un gris medio. Para una superficie lambertiana,

$$L = \frac{S \cos \theta_i \alpha_{\text{alb}}}{\pi}, \quad (2.21)$$

y con $S = 4$, $\cos \theta_i = \cos 62^\circ = 0,469$ y albedo $0,41$ sale $L \approx 0,25$, unos $0,54$ en sRGB, lo que deja margen para que las paredes iluminadas de frente no se saturen.

2.6. Modelo de cámara

2.6.1. Cámara estenopeica

La cámara de la nave se modela como una cámara estenopeica ideal, sin distorsión. Un punto $\mathbf{P}$ expresado en el sistema de la cámara se proyecta en la imagen según

$$\begin{pmatrix} u_{\text{px}} \\ v_{\text{px}} \end{pmatrix} = \begin{pmatrix} c_x \\ c_y \end{pmatrix} + \frac{f}{-P_z} \begin{pmatrix} P_x \\ -P_y \end{pmatrix}, \quad (2.22)$$

donde $(c_x, c_y) = (W/2, H/2)$ es el centro de la imagen. El signo de P_z se debe a que la cámara mira hacia $-z$, y el de P_y al cambio de mano entre la cámara y la imagen (Sección 2.3).

La distancia focal en píxeles depende del ancho de la imagen W y del campo de visión horizontal α :

$$f = \frac{W}{2 \tan(\alpha/2)}. \quad (2.23)$$

Con $W = 1920$ y $\alpha = 60^\circ$ sale $f = 1663$ píxeles. En Blender se introduce como longitud focal sobre un sensor de 36 mm , $31,18 \text{ mm}$, que es la misma geometría expresada en otra unidad.

2.6.2. Huella y resolución

Mirando hacia abajo desde una altura h , la imagen cubre en el suelo

$$L_x = 2h \tan(\alpha/2) = 1,155 h, \quad L_y = L_x \frac{H}{W} = 0,650 h, \quad (2.24)$$

y cada píxel cubre

$$\text{GSD} = \frac{L_x}{W} = \frac{h}{f}. \quad (2.25)$$

A 170 km la huella es de $196 \times 110 \text{ km}$ y cada píxel cubre 102 m ; a 120 km , $139 \times 78 \text{ km}$ y 72 m .

Estas dos ecuaciones explican un resultado importante del Capítulo 5. Al subir, el área vista crece con h^2 , así que entran más cráteres en la imagen, pero cada uno se ve más pequeño, porque su tamaño en píxeles baja con h . A partir de cierta altura los cráteres del mapa son demasiado pequeños para detectarlos y el sistema deja de funcionar (Sección 5.8).

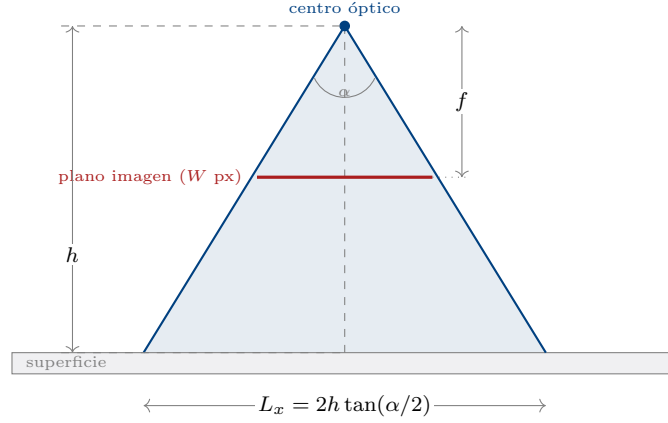


Figura 2.4: Geometría de la cámara estenopeica en visión nadir. La distancia focal en píxeles f y el campo de visión horizontal α están ligados por la Ecuación (2.23); la huella sobre el suelo y la resolución sobre el terreno dependen linealmente de la altura (Ecuaciones (2.24) y (2.25)). Esta dependencia lineal es la que permite despejar la altitud a partir del factor de escala de la semejanza, y también la causa del muro de altitud: el área observada crece con h^2 pero el tamaño aparente de los cráteres decrece con h .

2.6.3. La cámara del mapa

El mapa embarcado se obtiene renderizando el terreno con una segunda cámara, ortográfica, colocada sobre el centro del terreno con `ortho_scale` = 840, de forma que la imagen abarca exactamente los 840×630 km del terreno. Se renderiza a 4096×3072 píxeles, 205 m por píxel. Se usa una cámara ortográfica porque en ella la escala es la misma en toda la imagen, que es lo que se espera de un mapa.

Esta cámara corrigió un error importante. La primera versión del mapa se renderizó con la cámara normal, en formato 16:9, y solo el 76 % del ancho de la imagen era terreno; a cada lado sobraban 44 km de fondo negro. MATLAB pasaba de píxeles a kilómetros con

$$k = \frac{\text{ancho del terreno en km}}{\text{ancho de la imagen en px}}, \quad (2.26)$$

suponiendo que todo el ancho de la imagen era terreno, y por eso todas las distancias salían un 31,6 % más cortas: la altitud mediana estimada era de 91 km cuando la real era de 120 km. Lo difícil de este error es que no se notaba, porque la trayectoria tenía la forma correcta y los emparejamientos eran buenos. Solo apareció al comparar con la altitud real exportada de Blender. Con la cámara ortográfica

$$k = \frac{840}{W_{\text{mapa}}} = \frac{630}{H_{\text{mapa}}} = 0,205 \text{ km/px}, \quad (2.27)$$

y las dos divisiones tienen que dar lo mismo, lo que sirve de comprobación.

2.7. Trayectorias

2.7.1. Secuencia de crucero

La secuencia principal tiene $N = 700$ fotogramas. Con $t = k/(N - 1)$, que va de 0 a 1, y la *smoothstep* $s(t)$ de la Ecuación (2.3), la cámara sigue

$$x(t) = x_0 + (x_1 - x_0) t, \quad (2.28)$$

$$y(t) = y_c + A_y \sin(2\pi n_c t), \quad (2.29)$$

$$h(t) = h_0 + (h_1 - h_0) s(t), \quad (2.30)$$

con $x_0 = 100$ km, $x_1 = 740$ km, $y_c = 350$ km, $A_y = 130$ km, $n_c = 2$, $h_0 = 170$ km y $h_1 = 120$ km. La cámara mira siempre hacia abajo y sin guiñada. La Figura 2.5 muestra el recorrido.

La altura baja siguiendo la *smoothstep* para que el descenso empiece y termine con velocidad vertical nula. Los fotogramas clave se interpolan linealmente, porque la interpolación que Blender usa por defecto se pasa del valor objetivo en las curvas del serpenteo.

Los valores se eligieron buscando el recorrido más largo que cumpliera dos condiciones: que en ningún fotograma haya menos de tres cráteres de referencia, que es el mínimo que necesita el algoritmo del Capítulo 4, y que la huella no se salga nunca del terreno, porque un fotograma con fondo negro no sirve ni para navegar ni para entrenar. De los 180 cráteres de referencia del generador se ven de media 6,3 por fotograma, con un mínimo de 3. Contando los 258 del mapa embarcado (Sección 4.2) la media sube a 8,1, con un mínimo de 3 y un máximo de 16. Hay 7 fotogramas, el 1,0 %, con solo tres cráteres del mapa. Con tres se puede plantear una hipótesis, pero no comprobarla con un cuarto cráter como exige la verificación de la Sección 4.3.4, así que esos fotogramas no se pueden resolver, haga lo que haga el detector.

El recorrido mide 1263 km y la cámara avanza 1,81 km entre fotogramas, un 1,3 % del ancho de la imagen. Así los fotogramas seguidos se solapan casi por completo y la trayectoria es continua, pero hay suficiente variedad para que el conjunto de entrenamiento no sea repetitivo.

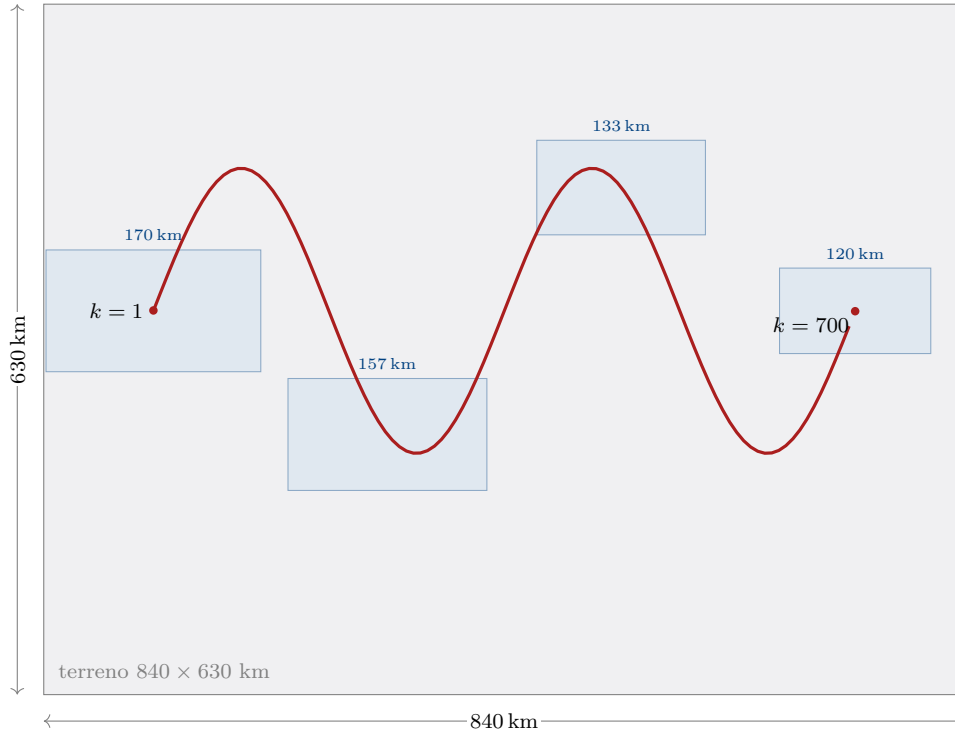


Figura 2.5: Planta de la trayectoria de cruceo definida por las Ecuaciones (2.28) a (2.30): avance uniforme en x , serpenteo sinusoidal de dos ciclos en y y descenso suavizado de 170 km a 120 km. En azul, la huella de la cámara en cuatro instantes; se encoge de 196×110 a 139×78 km conforme la nave desciende. El recorrido sobre el suelo es de 1263 km.

2.7.2. Secuencia de descenso con rotación

La segunda secuencia se usa en el tercer ensayo del Capítulo 5. Baja desde 170 km hasta el límite inferior de funcionamiento y va girando la cámara en guiñada, para probar las dos cosas que la secuencia de cruceo no incluye: un rango de altitud amplio y una orientación distinta de cero. Se genera igual que la de cruceo, añadiendo un giro alrededor del eje z de

la cámara que crece linealmente con el fotograma. Al girar la imagen, las cajas del detector, que están alineadas con los ejes de la imagen, cambian de tamaño, y por eso fue necesario reentrenar el detector con giros (Capítulo 3).

Las etiquetas de todos estos fotogramas y la trayectoria real se generan automáticamente proyectando el catálogo, como se explica en la Sección 3.6.

2.8. Limitaciones del entorno

El entorno tiene estas limitaciones, que hay que tener en cuenta al leer los resultados:

- La superficie se ilumina como un difusor de Oren–Nayar. El regolito real devuelve más luz hacia la fuente (el efecto de oposición), y un modelo de Hapke sería más fiel.
- No se simula el sensor: no hay ruido, ni diferencias entre píxeles, ni desenfoque por movimiento, que a velocidad orbital no es despreciable.
- El terreno tiene la estadística de la Luna, pero no es la Luna: el detector no ha visto ninguna formación real.
- El relieve más fino va en el bump y no proyecta sombras (Sección 2.5.2).
- El mapa usado en los ensayos se generó con un umbral de diámetro de 10 km. Para reproducir el trabajo hay que fijar ese valor; el Anexo C recoge todos los parámetros.

Capítulo 3

Detección de cráteres

Este capítulo describe el detector: qué familia de métodos se eligió y por qué, cómo funciona la red por dentro, cómo se construyeron los conjuntos de entrenamiento, cómo se entrenó en tres etapas encadenadas y cómo se mide. La salida del detector es una lista de cráteres en píxeles, y es lo único que recibe el algoritmo de posicionamiento del Capítulo 4.

3.1. ¿Por qué un detector de una sola pasada?

De las familias revisadas en la Sección 1.3.1, hay cuatro que se usan habitualmente para encontrar cráteres, y conviene decir por qué se descartan tres.

Los **métodos clásicos** basados en la transformada de Hough o en el ajuste de elipses sobre bordes no necesitan entrenamiento, pero dependen de que el contorno del cráter aparezca como un borde cerrado. Con iluminación rasante eso no ocurre: se ve un arco iluminado y un arco en sombra, y el resto del contorno no está. Fallan justo en la condición en la que este trabajo opera.

Los **detectores de dos pasadas**, como Faster R-CNN, proponen primero regiones candidatas y las clasifican después. Son algo más precisos en objetos difíciles, pero cuestan bastante más tiempo por imagen, y aquí hay que procesar cientos de objetos por fotograma a cadencia de vídeo.

Las **redes de segmentación**, como la que emplea DeepMoon (Silburt et al., 2019), devuelven una máscara de píxeles. Eso obliga a un posprocesado para agrupar píxeles en objetos y estimar centro y radio, y ese posprocesado introduce sus propios parámetros y sus propios errores.

Los **detectores de elipses**, como Ellipse R-CNN (Dong et al., 2021), son en teoría lo ideal, porque un cráter visto de forma oblicua es una elipse y no un círculo. Se descartaron por disponibilidad de implementaciones maduras y por coste de cálculo, y la limitación que esto impone se discute en la Sección 1.3.2 y reaparece en el Capítulo 5.

Se eligió un **detector de objetos de una sola pasada**, en concreto YOLO (Redmon et al., 2016), por tres razones:

1. **La salida es directamente lo que hace falta.** Entrega una lista de cajas con centro, tamaño y confianza. El centro de la caja es el centro del cráter y el lado es su diámetro aparente, que son las dos magnitudes que necesita el emparejador. No hay posprocesado entre medias.
2. **La velocidad.** Una sola pasada por la red y una supresión de no máximos. Es lo que permite el ensayo en tiempo real del Capítulo 5.
3. **La madurez de la implementación.** La biblioteca Ultralytics (Jocher et al., 2023) está mantenida, documentada y es de uso extendido en trabajos de detección de cráteres publicados (Tang et al., 2025), lo que hace que los resultados sean comparables con los de otros.

Dentro de la familia se usa la versión 8 y, de sus cinco tamaños, el más pequeño, **YOLOv8n**, con 3,2 millones de parámetros. El motivo es que el problema tiene una sola clase y objetos de forma muy repetitiva; un modelo mayor aportaría capacidad que no hace falta y costaría tiempo de inferencia, que aquí sí importa.

3.2. ¿Cómo funciona la red?

La idea de YOLO es tratar la detección como un problema de regresión resuelto de una vez sobre toda la imagen, en lugar de examinar regiones candidatas una por una (Redmon et al., 2016). La imagen entra una sola vez y la red predice, para cada posición de una rejilla, si allí hay un objeto y qué caja lo envuelve. La Figura 3.1 resume la estructura.

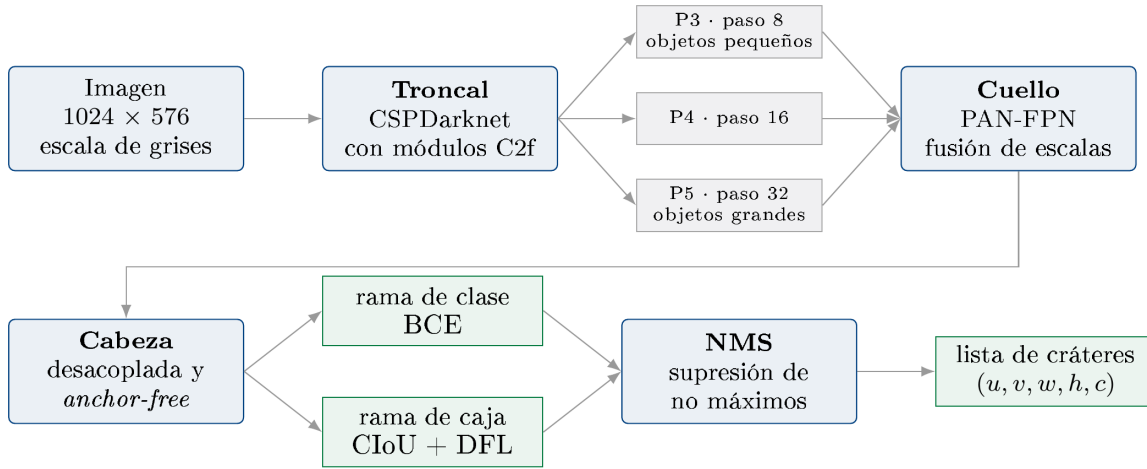


Figura 3.1: Arquitectura de YOLOv8. La imagen entra una sola vez y recorre el troncal, que produce mapas de características a tres escalas; el cuello los fusiona para que los objetos pequeños conserven detalle y los grandes conserven contexto; y la cabeza predice, en cada posición y sin cajas de referencia predefinidas, una puntuación de clase y una caja. La supresión de no máximos elimina las detecciones repetidas: es el último bloque, y no la red, el que causó el fallo de la Sección 3.7.

3.2.1. Troncal, cuello y cabeza

El **troncal** (*backbone*) es una red convolucional de tipo CSPDarknet con módulos C2f, que extrae características cada vez más abstractas y a la vez reduce la resolución. De él salen mapas a tres escalas, con pasos de 8, 16 y 32 píxeles respecto a la imagen de entrada. Esto importa aquí más de lo habitual: los cráteres del mapa embarcado ocupan del orden de 145 píxeles a las alturas de trabajo, mientras que los pequeños ocupan seis o siete, y una única escala no puede atender bien a ambos.

El **cuello** (*neck*) es una estructura de tipo PAN-FPN que mezcla las tres escalas en los dos sentidos: baja la información semántica de las capas profundas hacia las de alta resolución y sube el detalle fino hacia las profundas. El resultado es que los objetos pequeños se predicen con un mapa que conserva resolución pero ya sabe qué es un cráter.

La **cabeza** (*head*) de YOLOv8 tiene dos propiedades que conviene nombrar porque la diferencian de versiones anteriores (Jocher et al., 2023):

- Es **anchor-free**: no parte de un conjunto de cajas de referencia con formas predefinidas, sino que predice directamente la distancia de cada posición a los cuatro lados de la caja. Para este problema viene bien, porque no hay que elegir a mano unas proporciones de caja que representen a los cráteres.

- Es **desacoplada**: la clasificación y la regresión de la caja se calculan en ramas separadas, en lugar de compartir la última capa.

3.2.2. La función de pérdida

Durante el entrenamiento, cada predicción se compara con las etiquetas mediante una pérdida con tres términos (Jocher et al., 2023):

$$\mathcal{L} = \lambda_{\text{cls}} \mathcal{L}_{\text{BCE}} + \lambda_{\text{box}} \mathcal{L}_{\text{CIoU}} + \lambda_{\text{dfl}} \mathcal{L}_{\text{DFL}}. \quad (3.1)$$

El primero, $\mathcal{L}_{\text{BCE}}$, es una entropía cruzada binaria sobre la puntuación de clase. Con una sola clase se reduce a decidir si en esa posición hay un cráter o no.

El segundo mide lo bien que encaja la caja predicha con la verdadera mediante la IoU completa (CIoU), que amplía la intersección sobre unión de la Ecuación (3.8) con dos términos adicionales: la distancia entre los centros y la diferencia de proporción entre los lados. La IoU simple tiene un problema como función de pérdida: si dos cajas no se solapan, vale cero y su gradiente no indica hacia dónde mover la caja. El término de distancia entre centros lo resuelve.

El tercero, la pérdida focal de distribución (DFL), es consecuencia de que la cabeza no predice cuatro números sino cuatro *distribuciones discretas* sobre las distancias a los lados, de las que después se toma la esperanza. La DFL empuja la probabilidad hacia los dos valores discretos que rodean a la distancia verdadera. La ventaja práctica es que el borde de un cráter no es una línea nítida sino una transición de sombra, y una distribución representa esa incertidumbre mejor que un valor único.

La asignación de qué predicción se hace responsable de qué objeto la resuelve un asignador alineado con la tarea (*Task-Aligned Assigner*), que elige para cada objeto las posiciones cuya puntuación de clase y cuya IoU son buenas a la vez, en lugar de asignarlas solo por solapamiento geométrico.

3.2.3. Supresión de no máximos

La red predice muchas cajas solapadas para el mismo objeto. El último paso las reduce a una: se ordenan por confianza y, empezando por la mejor, se eliminan las que se solapan con ella por encima de un umbral de IoU. Este paso no es parte de la red, es un algoritmo aparte que se ejecuta después, y esa distinción resultó importante en este trabajo: el fallo de la Sección 3.7 está ahí y no en la red.

3.3. Cadena de entrenamiento

El detector se ha entrenado en tres etapas propias, encadenadas: cada una parte de los pesos que dejó la anterior y le añade una especialización. Lo único que se toma de terceros son los pesos iniciales de la primera.

Entrenar una red convolucional desde pesos aleatorios exige decenas de miles de imágenes etiquetadas y un presupuesto de cálculo del que este trabajo no dispone. La práctica estándar es partir de una red ya entrenada sobre un conjunto general de imágenes, que llega sabiendo reconocer bordes, gradientes y texturas, y reaprovechar esas representaciones de bajo nivel, que son en buena medida independientes del problema concreto (Yosinski et al., 2014). Aquí se parte del modelo *nano* de YOLOv8, distribuido públicamente por sus autores. A partir de ahí, todo el entrenamiento es propio.

Tabla 3.1: Cadena de entrenamiento del detector. La etapa 0 no es un entrenamiento sino el punto de partida público; las tres siguientes son propias y cada una arranca de los pesos de la anterior. Las dos últimas producen dos modelos distintos porque el crucero y el descenso tienen requisitos incompatibles.

	Parte de	Datos de entrenamiento	Resultado
0	—	<i>Pesos públicos de YOLOv8n, imágenes generales</i>	<i>No se entrena</i>
1	Etapa 0	Conjuntos públicos de cráteres marcianos y lunares	Detector genérico
2	Etapa 1	Renders del terreno sintético, sin rotación	Detector de crucero
3	Etapa 2	Los mismos renders, con aumento por rotación	Detector de descenso

3.4. Etapa 1: imágenes reales de cráteres

Esta etapa se realizó en una fase temprana del trabajo. Su objetivo era enseñar a la red qué es un cráter, usando imágenes reales de superficie planetaria.

No existía un único conjunto de datos adecuado, así que se ensamblaron tres conjuntos públicos distribuidos en Kaggle, que se recogen en la Tabla 3.2. El primero mezcla cráteres marcianos y lunares; el segundo distingue cráteres de rocas, de modo que hubo que quedarse solo con la clase de cráteres; el tercero es una versión en formato YOLO de un subconjunto del catálogo LU3M6TGT.

Tabla 3.2: Conjuntos de datos públicos empleados en la etapa 1. Los tres se descargan con el script `descargar_datasets.py`, disponible en el repositorio del Anexo A.

Identificador en Kaggle	Contenido	Formato original
lincolnzh/martianlunar-crater-detection-dataset	Cráteres marcianos y lunares	YOLO, con particiones train , valid y test
bhargavlc/lunar-craters-and-boulders	Cráteres y rocas lunares	Carpeta plana, etiquetas junto a las imágenes
riccardolagrassa/lu3m6tgt	Subconjunto del catálogo LU3M6TGT	YOLO, con particiones train y val

Los tres traían criterios de etiquetado y estructuras de carpetas diferentes, de modo que hubo que reetiquetarlos a una sola clase, unificar su organización y repartirlos. El conjunto resultante ronda las 9000 imágenes. El entrenamiento se hizo con 50 épocas, imágenes de 640 píxeles y lotes de 8.

Esta etapa dejó un problema que no se detectó hasta mucho después y que conviene contar, porque es el mismo error que se evita de forma deliberada en la etapa siguiente. Al unir los conjuntos, la partición resultante mandaba miles de imágenes a entrenamiento y dejaba un conjunto de validación de 33 **imágenes**, que además no seguía la misma distribución que el de entrenamiento. Con un conjunto de validación así, las métricas que la biblioteca informa durante el entrenamiento no miden nada útil: ni representan lo aprendido ni permiten decidir cuándo parar. El modelo servía, porque la etapa 2 lo reajusta, pero sus cifras de validación no eran interpretables. La lección se aplicó en la etapa 2.

3.5. Etapa 2: ajuste fino sobre los renders

Un detector entrenado con fotografías orbitales reales no funciona sobre renderizados. La estadística de la imagen es distinta: el ruido, el contraste y la forma de las sombras no coinciden. La solución es reajustarlo con los propios renderizados, y eso exige etiquetas, que se obtienen proyectando el catálogo como se explica en la Sección 3.6.

3.5.1. La partición del conjunto, que no es obvia

Los 700 fotogramas son *una sola trayectoria continua*. La cámara avanza 1,81 km entre fotogramas y ve una huella de hasta 196 km: dos fotogramas consecutivos comparten más del 98 % del terreno.

Con una partición aleatoria, el fotograma 300 acabaría en entrenamiento y el 301 en validación. Son casi la misma imagen. La validación daría una métrica altísima y completamente falsa, porque estaría midiendo memorización y no generalización. Es un caso de fuga de información entre particiones, y no se arregla barajando mejor: hay que separar los conjuntos *en el terreno*.

La separación necesaria se calcula, no se estima a ojo. Si la huella mide L_x y la cámara avanza Δs por fotograma, dos fotogramas dejan de compartir terreno cuando están separados por

$$n_{\text{hueco}} = \left\lceil \frac{L_x}{\Delta s} \right\rceil \quad (3.2)$$

fotogramas. Con $L_x = 196$ km y $\Delta s = 1,807$ km salen 109 fotogramas, que hay que tirar. La partición queda entonces como recoge la Tabla 3.3: los fotogramas descartados son el 16 % del total, y es un precio que hay que pagar.

Tabla 3.3: Partición del conjunto de renderizados. El hueco se calcula con la Ecuación (3.2) a partir de la trayectoria real, no se elige a mano. La separación en el suelo entre el último fotograma de entrenamiento y el primero de validación es de 200 km, mayor que la huella.

	Fotogramas	Número	Fracción
Entrenamiento	1–434	434	62 %
Hueco (descartado)	435–543	109	16 %
Validación	544–700	157	22 %

3.5.2. Hiperparámetros

La Tabla 3.4 recoge los valores empleados y el motivo de cada uno. Los dos que más condicionan el resultado son el tamaño de entrada y la ausencia de volteos.

Tabla 3.4: Hiperparámetros del ajuste fino sobre los renders. Los de aumento de datos que no aparecen se dejaron a cero.

Parámetro	Valor	Motivo
Modelo de partida	etapa 1	Ya sabe reconocer cráteres; solo hay que adaptarlo al dominio
Épocas	60	Con parada temprana
Paciencia	20	Épocas sin mejora antes de parar
Tamaño de entrada	1024	A 640 los cráteres pequeños quedan en 4 píxeles y son indetectables; a 1024 pasan a seis o siete, y los del mapa rondan los 145
Lote	4	Con entrada de 1024 no cabe más en una GPU de portátil
Clase única	sí	El problema tiene una sola categoría
Traslación	0,10	Desplazamientos pequeños de la escena
Escala	0,40	Simula la variación de altura de vuelo
Brillo (<i>hsv.v</i>)	0,25	Variación de exposición
Mosaico	1,0	Compone cuatro imágenes en una; se desactiva en las 10 últimas épocas
Volteos y giros	0	Véase el texto
Tono y saturación	0	Las imágenes están en escala de grises

La decisión de **no usar volteos ni giros** en esta etapa merece explicación, porque va en contra de lo que se hace habitualmente. En un cráter visto con sol rasante, la sombra siempre cae al mismo lado, y esa asimetría es justamente la señal que usa el detector para distinguir un cráter de un montículo: un hoyo y una loma producen la misma forma circular y

se diferencian solo por de qué lado está la sombra. Voltar la imagen genera una iluminación que no existe en ningún fotograma del conjunto, y enseña a la red una configuración imposible. En la secuencia de crucero, donde la cámara no gira, ese razonamiento es correcto. En la de descenso deja de serlo, y de ahí sale la etapa 3.

3.5.3. ¿Cuánto aporta esta etapa?

Que el ajuste fino sea necesario, y no un refinamiento cosmético, se mide. La Tabla 3.5 compara el modelo de la etapa 1 aplicado directamente sobre los renders con el de la etapa 2, sobre el mismo conjunto de validación.

Tabla 3.5: Modelo de la etapa 1 frente al de la etapa 2, evaluados sobre el mismo conjunto de validación de renderizados. La última columna es el número medio de detecciones por fotograma, frente a una media de 423 cráteres etiquetados.

Modelo	mAP@50	mAP@50-95	Precisión	Cobertura	Det./fotograma
Etapa 1 (imágenes reales)	0,380	0,125	0,614	0,435	69,6
Etapa 2 (ajuste fino)	0,669	0,514	0,952	0,675	300,0

El mAP@50 pasa de 0,380 a 0,669: el modelo sin ajustar rinde poco más de la mitad. La diferencia es aún mayor en mAP@50-95, que exige que las cajas encajen bien y no solo que estén, y que sube de 0,125 a 0,514. En número de detecciones por fotograma la diferencia es de 69,6 a 300.

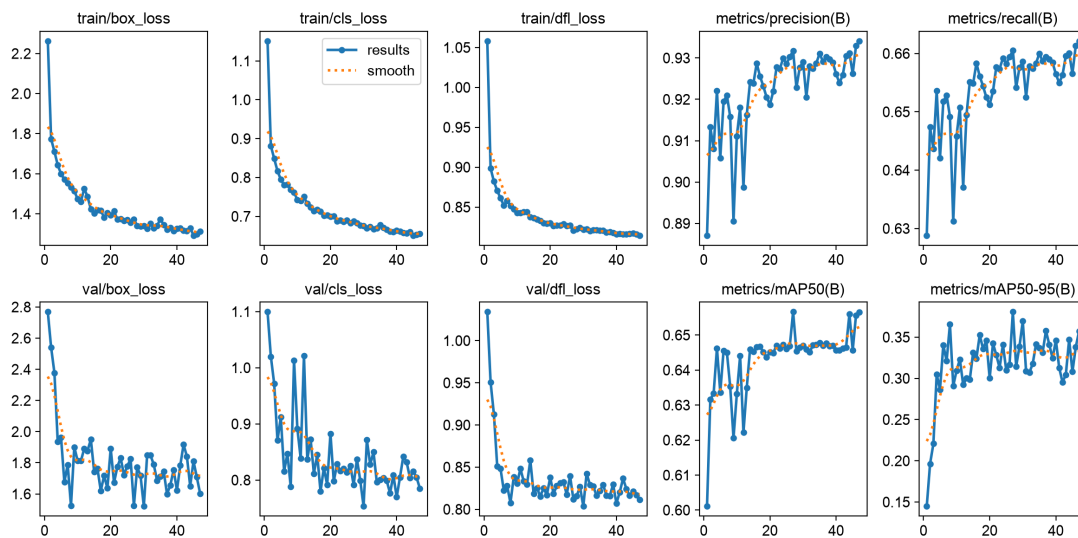


Figura 3.2: Evolución de las funciones de pérdida y de las métricas de validación durante el ajuste fino. El entrenamiento se detuvo por parada temprana al dejar de mejorar la métrica de validación.

3.6. Etiquetado automático de los renders

Para reentrenar el detector con renders hacían falta sus etiquetas: la caja de cada cráter visible en cada fotograma. Hacerlo a mano no era viable, pero tampoco hacía falta. Blender sabe en cada fotograma dónde está la cámara, y el catálogo del Capítulo 2 dice dónde está cada cráter, así que las etiquetas se calculan proyectando el catálogo sobre la imagen. Lo hace el script `blender_exportar_etiquetas.py`, y las etiquetas que salen son exactas, sin ningún criterio subjetivo de por medio.

3.6.1. Proyección del catálogo

En cada fotograma se toma la matriz $\mathbf{M}$ que lleva del sistema de la cámara al del mundo, y con su inversa se pasa cada punto al sistema de la cámara:

$$\mathbf{P}_c = \mathbf{M}^{-1} \mathbf{P}. \quad (3.3)$$

Después se pasa a coordenadas normalizadas de imagen (Sección 2.3), con v medido desde arriba como exige YOLO:

$$u = \frac{1}{2} + \frac{1}{2} \frac{P_{c,x}}{-P_{c,z} \tan(\alpha/2)}, \quad v = \frac{1}{2} - \frac{1}{2} \frac{P_{c,y}}{-P_{c,z} \tan(\alpha/2) \frac{H}{W}}. \quad (3.4)$$

Un punto solo es visible si $P_{c,z} < 0$, es decir, si está delante de la cámara.

Un cráter no es un punto sino un círculo, y en perspectiva un círculo no se proyecta como un círculo centrado en la proyección de su centro. Por eso se proyectan $n = 12$ puntos de su borde,

$$\mathbf{P}_j = (x_c + R \cos \theta_j, y_c + R \sin \theta_j, z_c), \quad \theta_j = \frac{2\pi j}{n}, \quad (3.5)$$

y la caja es el rectángulo que los contiene:

$$u_0 = \min_j u_j, \quad u_1 = \max_j u_j, \quad v_0 = \min_j v_j, \quad v_1 = \max_j v_j. \quad (3.6)$$

La altura z_c de los puntos del borde es la del terreno en el centro del cráter y no cero, porque el relieve tiene varios kilómetros de diferencia y no tenerlo en cuenta desplazaría las cajas.

Blender tiene una función para esto, `world_to_camera_view`, pero trabaja punto a punto: con 41 702 cráteres, 12 puntos por cráter y 700 fotogramas serían unos 350 millones de llamadas. Escrita con operaciones de matrices de NUMPY se reduce a unas pocas operaciones por fotograma. Además, solo se proyectan los cráteres cercanos a la vertical de la cámara, a menos de $1,8 h \tan(\alpha/2)$ en horizontal, que cubre de sobra las esquinas de la imagen.

3.6.2. ¿Qué cráteres se etiquetan?

Un cráter proyectado solo se convierte en etiqueta si cumple cinco condiciones:

1. **Los doce puntos del borde están delante de la cámara.** Si alguno queda detrás, su proyección no tiene sentido y la caja saldría mal.
2. **El centro de la caja cae dentro de la imagen**, con un margen del 2%.
3. **El lado menor de la caja mide al menos 12 píxeles.** Por debajo, el cráter no tiene detalle suficiente para que el detector lo aprenda, y etiquetarlo le enseñaría a responder a ruido.
4. **La frescura es $\phi \geq 0,30$.** Un cráter muy degradado casi no se ve, y etiquetarlo enseñaría al detector a buscar algo que no está en la imagen. Este filtro quita el 16,6 % del catálogo.
5. **Al recortar la caja al borde de la imagen queda al menos un tercio de su área.** Si el cráter está muy cortado, la caja ya no corresponde al cráter y se descarta.

Con estos criterios, la secuencia de crucero tiene 310 786 cajas en 700 fotogramas: una media de 444 por fotograma, con un mínimo de 342 y un máximo de 589. Cada etiqueta es una línea con la clase, el centro y el tamaño de la caja normalizados, en el formato de YOLO. Solo hay una clase: cráter.

3.6.3. Comprobación de las etiquetas

Al escribir la proyección a mano se corría el riesgo de meter un error que desplazara todas las etiquetas por igual; el entrenamiento aprendería ese desplazamiento sin que nada fallara. Para evitarlo, en el primer fotograma el script compara su proyección con la de `world_to_camera_view` en veinte cráteres y avisa si la diferencia supera 10^{-4} en coordenadas normalizadas.

Junto con las etiquetas se guarda `trayectoria_real.csv`, con la posición, la altura, la orientación y el número de cráteres etiquetados de cada fotograma. Es el fichero con el que se calculan todos los errores del Capítulo 5.

3.7. Un fallo que no era del modelo

Durante la caracterización del detector apareció un comportamiento que costó entender y que merece contarse, porque no está en ningún manual.

Al procesar las secuencias completas, una fracción grande de fotogramas — alrededor del 40 % — salía *sin ninguna detección*. No con pocas: con cero. Los mismos fotogramas, procesados de uno en uno, daban resultados normales. La primera hipótesis fue que se estaba alcanzando el número máximo de detecciones por imagen que la biblioteca permite; se comprobó elevándolo y quedó descartada, porque el porcentaje de fotogramas vacíos no cambió.

La causa estaba en la supresión de no máximos, el paso descrito al final de la Sección 3.2. Esa rutina tiene un límite de tiempo interno que depende del tamaño del lote de imágenes. Cuando la escena contiene muchos objetos — y un terreno cubierto de cráteres los contiene a millares — el filtrado tarda más de lo previsto, se agota el límite, y la biblioteca interrumpe el bucle y devuelve tensores vacíos para el resto del lote. No lanza una excepción: emite un aviso y sigue.

Tabla 3.6: Efecto del tamaño de lote en la inferencia. El modelo, las imágenes y el umbral de confianza son idénticos en ambos casos; lo único que cambia es cuántas imágenes se procesan a la vez.

	Lote de 8	Lote de 1
Fotogramas sin ninguna detección	~40 %	0 %
Detecciones por fotograma (mediana)	—	14,8

La conclusión práctica es sencilla y se mantuvo en todo el trabajo: la inferencia se ejecuta siempre con lotes de una imagen. El coste en velocidad es despreciable frente al de perder cuatro de cada diez fotogramas.

La conclusión metodológica es más interesante. Durante un tiempo se atribuyó el problema al modelo, y se llegó a considerar que el detector afinado era peor que el de partida. No lo era: la comparación estaba contaminada por una pérdida de datos que ocurría por debajo, en la biblioteca, y que se manifestaba como si fuera un problema de calidad de la red. Es la entrada número 2 de la Tabla 1.3, y es la razón de que esa tabla exista.

3.8. Punto de operación

Fijado el modo de inferencia, queda elegir el umbral de confianza por encima del cual una detección se acepta. La elección no es neutra y depende de para qué se va a usar el detector.

En una aplicación de conteo de cráteres interesa el equilibrio entre precisión y cobertura, porque un falso positivo contamina la estadística. En navegación no: lo que importa es la **cobertura**. El motor de emparejamiento incorpora su propia verificación geométrica, descrita en la Sección 4.3.4, que rechaza los cráteres que no encajan; un falso positivo se descarta ahí

sin consecuencias. En cambio, un cráter que el detector no ve está perdido para siempre, y si por su culpa quedan menos de tres cráteres válidos en la imagen, el fotograma entero se queda sin solución.

Con ese criterio se fijó el punto de operación en un umbral de confianza de 0,70, con las imágenes reescaladas a 1024 píxeles en el lado mayor. La cobertura resultante sobre los cráteres útiles del terreno es del 99,4 %.

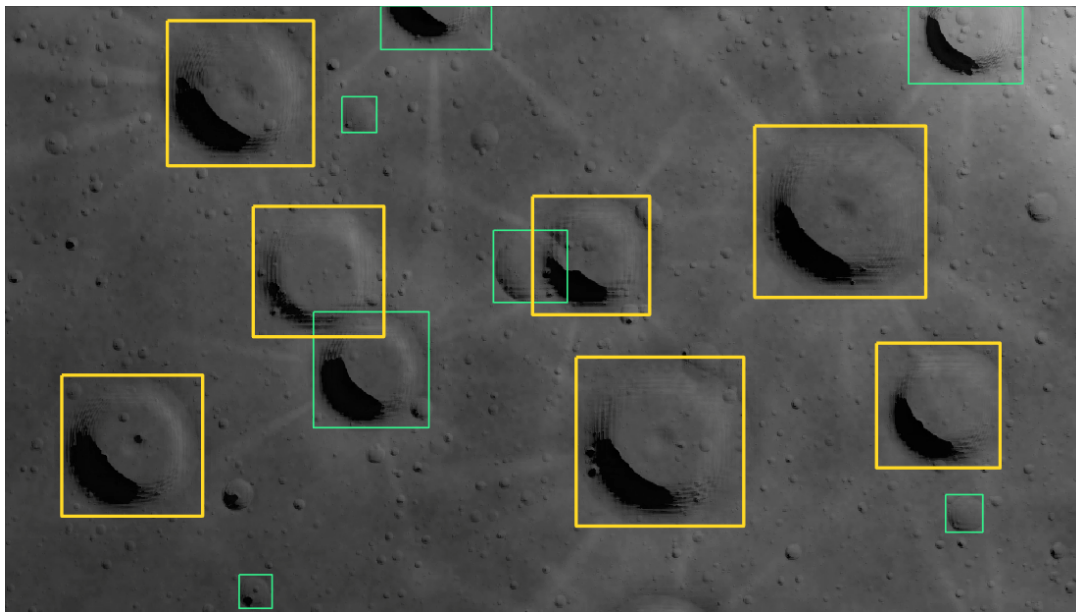


Figura 3.3: Fotograma de la secuencia de crucero con las detecciones superpuestas. En amarillo, las K detecciones de mayor tamaño aparente, que son las que entran en la formación de tripletas; en verde, el resto, que el detector encuentra pero que no se utilizan porque corresponden a cráteres demasiado pequeños para estar en el mapa embarcado. Obsérvese que toda caja está alineada con los ejes de la imagen, con la limitación que se discutió en la Sección 1.3.2.

3.9. Etapa 3: reentrenamiento para la rotación

El detector descrito hasta aquí se entrenó con imágenes de la cámara de crucero, que vuela sin giro en guiñada. Al aplicarlo a la secuencia de descenso, en la que la cámara gira, su rendimiento se hundía.

3.9.1. ¿Qué se midió?

La Tabla 3.7 recoge la degradación. No es una pérdida suave: a partir de 45° el detector deja prácticamente de ver cráteres, y a 180° no encuentra ninguno en imágenes donde las etiquetas dicen que hay más de cuatrocientos.

Antes de aceptar esta medida hubo que corregir un error en la propia medición, que conviene mencionar. La prueba consiste en rotar las imágenes y sus etiquetas y volver a medir la cobertura, y la rotación de las etiquetas estaba implementada con el convenio de ejes habitual en matemáticas, con el eje vertical hacia arriba. En coordenadas de imagen el eje vertical apunta hacia abajo (Sección 2.3), de modo que la rotación aplicada a las etiquetas era la inversa de la que sufrían los píxeles. El resultado era coherente solo en 0° y 180° , que son precisamente los ángulos en los que ambos convenios coinciden. Corregido el signo y verificado contra la matriz de rotación que emplea la propia biblioteca de tratamiento de imagen, la medida pasó a tener sentido.

Tabla 3.7: Cobertura del detector frente al giro de la imagen, antes y después del reentrenamiento con aumento por rotación. La última columna es el ancho medio de las cajas predichas, normalizado por el que tienen a 0° .

Giro	Antes (sin aumento)		Después (aumento $\pm 180^\circ$)	
	Cobertura	Ancho rel.	Cobertura	Ancho rel.
0°	1,00	1,01	1,00	1,16
15°	0,95	1,01	0,95	1,28
30°	0,75	1,01	0,85	1,38
45°	0,28	1,02	0,83	1,43
60°	0,08	1,00	0,85	1,38
90°	0,19	1,01	0,97	1,10
135°	0,02	1,07	0,78	1,45
180°	0,00	—	0,98	1,15

3.9.2. ¿Por qué ocurre?

La causa es la iluminación. El sol está fijo en el mundo, no en la cámara. Al girar la cámara, la sombra del cráter cambia de lado dentro de la imagen. La red había aprendido «cráter = borde iluminado a un lado y sombra al otro, en esta orientación», y a 180° ve exactamente lo contrario: un cráter con la sombra en el lado equivocado parece un montículo. Es la misma ambigüedad entre cráter y domo que aparece al interpretar imágenes lunares reales cuando se confunde la dirección del sol.

Esto convierte la decisión de la etapa 2 en un ejemplo útil. Desactivar los volteos era correcto para un vuelo sin guiñada, porque preservaba la coherencia de la iluminación, y era equivocado en cuanto la cámara gira. No es que la decisión anterior fuera un error: es que su justificación dependía de una hipótesis, el vuelo nadir sin giro, que la fase de descenso rompe.

3.9.3. La corrección

Se reentrena partiendo del modelo ya afinado, con el mismo conjunto de datos y cambiando únicamente el aumento: giros aleatorios en todo el rango de $\pm 180^\circ$ y volteos en ambos ejes con probabilidad 0,5. Al girar cada imagen de entrenamiento un ángulo al azar, la red ve el mismo cráter con la sombra en todas las direcciones y se ve obligada a apoyarse en la forma en lugar de en una dirección de luz fija.

Que los datos no cambien es deliberado: así la secuencia de descenso sigue siendo un conjunto de prueba completamente ajeno, y la mejora que se mida es atribuible solo al aumento y no a haber visto más imágenes. El resultado está en la mitad derecha de la Tabla 3.7: la cobertura pasa de hundirse a mantenerse entre 0,78 y 1,00 en todo el rango angular.

3.9.4. Un efecto secundario que resultó revelador

La última columna de la Tabla 3.7 contiene algo que no se buscaba. Al rotar un cráter circular, la caja alineada con los ejes que lo envuelve no cambia de tamaño; pero al rotar la *imagen*, la caja que envuelve al mismo cráter sí cambia, porque la envolvente de una forma girada se ensancha según

$$k(\theta) = |\cos \theta| + |\sin \theta|, \quad (3.7)$$

que vale 1 a 0° y $\sqrt{2}$ a 45° . El modelo sin reentrenar predice cajas de ancho constante, como muestra su columna: nunca pasa de 1,07. El reentrenado reproduce el ensanchamiento, con un máximo de 1,43 a 45° , muy cerca del $\sqrt{2} = 1,414$ que predice la Ecuación (3.7).

Esto significa que la red ha aprendido a estimar la orientación de la escena, y solo puede hacerlo a partir de la dirección aparente de la iluminación, que es la única pista disponible

en la imagen. Es un comportamiento que nadie programó, y tiene consecuencias directas sobre la estimación de la altitud, porque el tamaño de las cajas es lo que fija la escala del emparejamiento. Se retoma en el Capítulo 5.

3.10. Validación del detector

Esta sección fija *cómo* se mide el detector; las cifras obtenidas se presentan y analizan en la Sección 5.3.

3.10.1. Métricas

Una detección se considera acierto si su caja solapa con una caja de referencia por encima de un umbral de intersección sobre unión,

$$\text{IoU}(B_d, B_r) = \frac{\text{area}(B_d \cap B_r)}{\text{area}(B_d \cup B_r)}, \quad (3.8)$$

con umbral 0,50, valor habitual en detección de objetos. Cada caja de referencia puede emparejarse con una sola detección; las detecciones sobrantes son falsos positivos (FP) y las cajas de referencia sin pareja, falsos negativos (FN). Con VP el número de aciertos, las tres magnitudes que se emplean son

$$P = \frac{VP}{VP + FP}, \quad R = \frac{VP}{VP + FN}, \quad F_1 = \frac{2PR}{P + R}. \quad (3.9)$$

La *precisión* P responde a «de lo que el detector dice que hay, cuánto hay de verdad»; la *cobertura* R , a «de lo que hay, cuánto encuentra»; y F_1 es su media armónica, que penaliza el desequilibrio entre ambas.

Se emplean además las dos métricas habituales en detección de objetos. El **mAP@50** es el área bajo la curva de precisión frente a cobertura, con el umbral de IoU fijado en 0,50. El **mAP@50–95** promedia ese valor para umbrales de IoU de 0,50 a 0,95 en pasos de 0,05, y por tanto exige que las cajas no solo estén sino que encajen bien. La diferencia entre ambos es informativa: un modelo con mAP@50 alto y mAP@50–95 bajo encuentra los cráteres pero no acierta su tamaño, y el tamaño es lo que fija la escala del emparejamiento.

Conviene subrayar que para este trabajo estas métricas no deciden. El resolutor del Capítulo 4 necesita al menos tres detecciones correctas por fotograma y tolera bien las espurias, porque la verificación geométrica las descarta; en cambio, un fotograma con menos de tres detecciones válidas es irre recuperable. Por eso el punto de operación se elige privilegiando la cobertura sobre la precisión, y por eso la métrica que gobierna la decisión final no es F_1 ni el mAP sino la *cobertura de fotogramas resueltos*: la fracción de fotogramas de la secuencia para los que el sistema completo entrega una solución verificada. Es una métrica de sistema, no de detector, y es la única que mide lo que de verdad interesa.

3.10.2. Conjuntos de datos y protocolo

La partición del conjunto de renderizados es la descrita en la Sección 3.5.1: por tramos contiguos de la trayectoria y con un hueco calculado para que entrenamiento y validación no compartan terreno. La secuencia de descenso no interviene en ningún entrenamiento y se reserva íntegra como conjunto de prueba.

El protocolo de evaluación tiene tres pasos. Primero se caracteriza el detector en aislamiento, comparando los modelos de las distintas etapas sobre el conjunto de validación con las métricas anteriores. Segundo se barre el umbral de confianza y se mide, para cada valor, no solo P y R sino la cobertura de fotogramas resueltos del sistema completo, que es lo que determina el punto de operación. Tercero se somete el detector a rotaciones controladas

de la imagen, de 0° a 180° , y se mide la degradación de la cobertura y el ensanchamiento de las cajas. Todas las evaluaciones se ejecutan sobre la misma partición y con la misma semilla.

Capítulo 4

Posicionamiento relativo al terreno

Este capítulo desarrolla el núcleo del trabajo: el algoritmo que, a partir de una lista de cráteres en píxeles y de un mapa almacenado a bordo, determina dónde está el vehículo, hacia dónde mira y a qué altura vuela.

4.1. El problema: estimar el vector de estados

El sistema estima en cada fotograma un **vector de estados** de cuatro componentes,

$$\mathbf{x} = (x, y, h, \psi)^\top, \quad (4.1)$$

donde (x, y) es la posición horizontal sobre el terreno en kilómetros, h la altitud y ψ el ángulo de guiñada. No se estiman ni el cabeceo ni el alabeo: la cámara apunta al nadir, y esa hipótesis es la que permite reducir el problema a una transformación plana. Tampoco se estima la velocidad, porque el sistema no encadena fotogramas.

La observación de la que se parte es el conjunto de detecciones del Capítulo 3,

$$\mathcal{Z} = \{(u_i, v_i, w_i)\}_{i=1}^N, \quad (4.2)$$

con (u_i, v_i) el centro de la caja en píxeles y w_i su lado. El conocimiento previo es el mapa embarcado,

$$\mathcal{M} = \{(x_j, y_j, D_j)\}_{j=1}^M, \quad (4.3)$$

con $M = 258$ cráteres cuya posición y diámetro se conocen en kilómetros.

El problema tiene dos partes de naturaleza muy distinta, y conviene separarlas desde el principio porque el resto del capítulo sigue esa división:

1. **La correspondencia.** Decidir qué detección de $\mathcal{Z}$ corresponde a qué cráter de $\mathcal{M}$. Es un problema combinatorio, discreto, y es el difícil: hay M^N asignaciones posibles y ninguna pista local que permita descartarlas una a una.
2. **La estimación.** Una vez conocida la correspondencia, despejar $\mathbf{x}$. Es un problema continuo, lineal en las incógnitas adecuadas, y tiene solución cerrada.

La primera parte se resuelve con descriptores invariantes de tripletas (Sección 4.2 y Sección 4.2.3); la segunda, con un ajuste de semejanza en el plano complejo (Sección 4.3.2). Y entre ambas hace falta una tercera pieza que no es obvia y que resultó ser la más delicada: un procedimiento de *verificación* que distinga una correspondencia correcta de una correspondencia falsa que encaja igual de bien (Sección 4.3.4).

4.2. El mapa embarcado y la tabla de tripletas

4.2.1. ¿Qué cráteres van al mapa?

El terreno contiene decenas de miles de cráteres, pero el mapa que viaja a bordo no puede ni debe contenerlos todos. Hay dos razones, y la segunda es la importante.

La primera es de tamaño. Un catálogo es barato comparado con un mosaico de imagen, pero el coste del emparejamiento no crece con el número de cráteres, sino con el número de *grupos* de cráteres, que es del orden del cubo. Triplicar el catálogo multiplica por veintisiete el trabajo.

La segunda es de discriminación, y va en contra de la intuición. Cuantos más cráteres tiene el mapa, más tripletas hay, y más probable es que dos tripletas distintas tengan descriptores casi idénticos. Un catálogo denso no solo es más caro: produce *más soluciones falsas*. Esta afirmación se comprueba experimentalmente en el Capítulo 5, donde se ensayan tres mapas de densidad creciente y la fiabilidad cae del 97 % al 84 % al pasar del más disperso al más denso.

El criterio adoptado es quedarse con los cráteres grandes, que son los que un orbitador cartografía con fiabilidad y los que siguen siendo visibles desde altitudes elevadas. Aplicando un umbral de 10 km de diámetro, el catálogo aporta 281 candidatos.

Sobre ellos actúa un segundo filtro, esta vez de aspecto y no de tamaño. El generador de terreno asigna a cada cráter un parámetro de *frescura* que gobierna cuánto conserva su relieve original: un cráter fresco tiene el borde nítido y proyecta una sombra marcada, mientras que uno antiguo aparece prácticamente enrasado con la llanura. Los cráteres muy erosionados son justamente los que el detector no encuentra de forma fiable, de modo que incluirlos en el mapa no aporta posiciones utilizables y sí añade tripletas que nunca llegan a emparejarse. Se descartan por tanto los que tienen una frescura inferior a 0,30, que son 23 de los 281.

El mapa se construye por tamaño y no a partir de los 180 cráteres de referencia del generador. En una primera versión se usaron solo esos 180, pero en el terreno hay además cráteres de fondo de más de 10 km que el detector ve igual; al no estar en el mapa, sus detecciones no tenían pareja y generaban tripletas falsas. El criterio que se adoptó es que todo cráter que el detector pueda ver tiene que estar en el mapa.

El mapa embarcado queda así en **258 cráteres**, con diámetros comprendidos entre 10,1 km y 34,5 km, y ocupa unos pocos kilobytes de texto. Frente a los 41 702 cráteres del catálogo completo, el mapa retiene el 0,6 %.

4.2.2. ¿Por qué tripletas?

Un cráter aislado no es identificable, como ya se argumentó. Un par tampoco basta: dos cráteres definen una distancia y dos diámetros, y bajo un cambio de escala desconocido la distancia deja de ser informativa; quedan muy pocos números independientes y la ambigüedad es enorme.

Para ver por qué hace falta algún tipo de descriptor, basta contar. Con $N = 7$ detecciones y $M = 258$ cráteres en el mapa, el número de asignaciones posibles entre unas y otros es

$$\frac{M!}{(M - N)!} = 258 \cdot 257 \cdots 252 \approx 2,9 \times 10^{16}, \quad (4.4)$$

que es inabordable por enumeración. Un descriptor invariante convierte ese problema en una búsqueda indexada: en lugar de probar asignaciones, se calcula una firma numérica de cada grupo de tres detecciones y se busca directamente en una tabla ordenada.

Con tres cráteres el problema cambia de naturaleza. Un triángulo tiene forma, y la forma es invariante frente a traslación, giro y escala. A eso se suman los tres diámetros, cuyas razones también son invariantes. Grupos de cuatro o más darían todavía más poder discriminante,

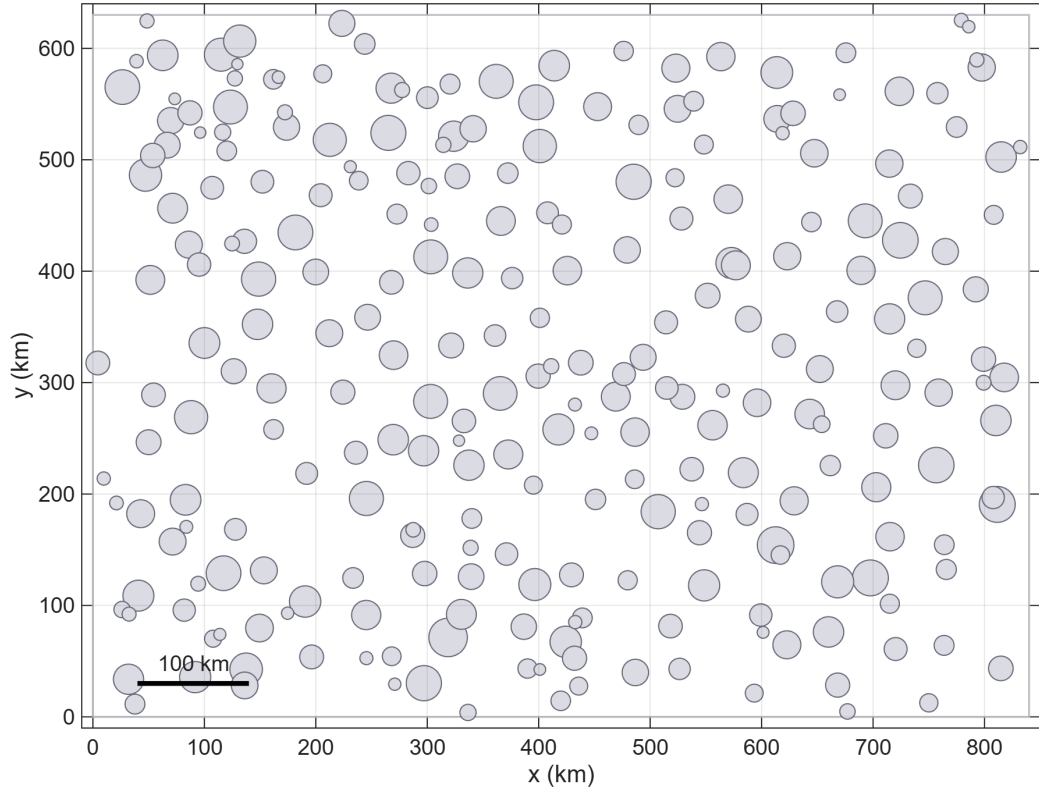


Figura 4.1: Mapa embarcado. Cada círculo es un cráter del catálogo con diámetro superior a 10 km, representado a escala. Este es todo el conocimiento previo del terreno que lleva el sistema: 258 posiciones y 258 diámetros.

pero el número de combinaciones crece demasiado deprisa y, sobre todo, exigirían que el detector encontrara al menos cuatro cráteres válidos en cada imagen, lo que a baja altitud deja de cumplirse antes. Tres es el compromiso.

4.2.3. El descriptor invariante

Sea una tripleta de cráteres con centros $\mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_3$ y diámetros D_1, D_2, D_3 . Las longitudes de los tres lados del triángulo son

$$\ell_1 = \|\mathbf{p}_2 - \mathbf{p}_3\|, \quad \ell_2 = \|\mathbf{p}_1 - \mathbf{p}_3\|, \quad \ell_3 = \|\mathbf{p}_1 - \mathbf{p}_2\|, \quad (4.5)$$

donde cada lado se ha numerado con el índice del vértice *opuesto*. Esa convención es lo que permite lo que viene ahora.

Se ordenan los tres lados de menor a mayor, $s_0 \leq s_1 \leq s_2$, y se **reetiquetan los vértices** de acuerdo con esa ordenación: el vértice *A* es el opuesto al lado más corto, el *C* el opuesto al más largo, y el *B* el restante. Con los vértices así ordenados, el descriptor de la tripleta es el vector de cuatro componentes

$$\mathbf{q} = \left(\frac{s_0}{s_2}, \quad \frac{s_1}{s_2}, \quad \frac{D_A}{D_C}, \quad \frac{D_B}{D_C} \right). \quad (4.6)$$

Las cuatro componentes son cocientes de magnitudes homogéneas, y por tanto adimensionales. De ahí se siguen sus dos propiedades esenciales. Son invariantes frente a un cambio de escala, porque multiplicar todas las longitudes por un factor deja los cocientes intactos; y lo son frente a un giro, porque las longitudes no dependen de la orientación. Las dos primeras componentes describen la *forma* del triángulo y las dos últimas la *proporción*

de tamaños entre sus cráteres. Un mismo triángulo visto desde el doble de altura y girado noventa grados produce exactamente el mismo descriptor.

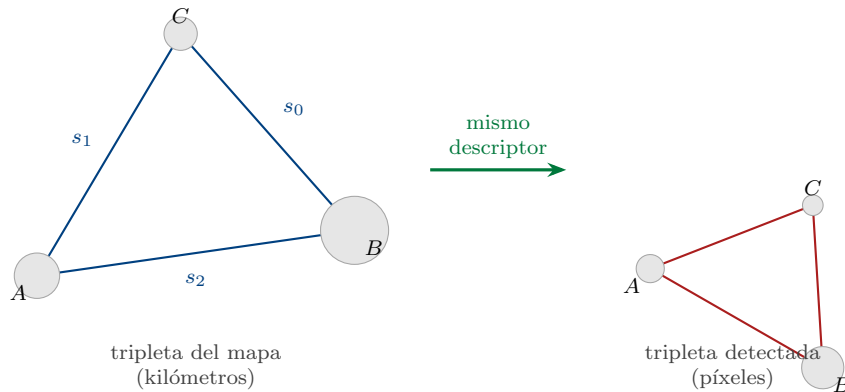


Figura 4.2: El descriptor de la Ecuación (4.6). Los vértices se etiquetan por el orden de sus lados opuestos, de modo que *A* es siempre el opuesto al lado más corto. La misma tripleta observada a otra escala y con otro giro produce el mismo vector de cuatro números, y además la correspondencia entre vértices queda resuelta sin necesidad de probar las seis permutaciones posibles.

El etiquetado canónico tiene una ventaja que no es evidente y que ahorra buena parte del trabajo. Sin él, al encontrar una tripleta del mapa cuyo descriptor coincide con el observado, habría que averiguar además qué cráter detectado corresponde a cada cráter del mapa, lo que obliga a ensayar las seis permutaciones y a quedarse con la mejor. Con los vértices ordenados por sus lados opuestos en ambos lados de la comparación, la correspondencia *ya está establecida*: el *A* observado se corresponde con el *A* del mapa. La ambigüedad que había que resolver por fuerza bruta desaparece.

Queda un caso degenerado que conviene mencionar. Si el triángulo es isósceles, dos de sus lados son iguales y el orden entre ellos es arbitrario, de modo que la etiqueta de los vértices correspondientes queda indeterminada. En la práctica el efecto es despreciable, porque las posiciones son continuas y la coincidencia exacta tiene probabilidad nula; los casos casi isósceles se resuelven porque la verificación posterior descarta la asignación incorrecta.

4.2.4. Construcción y poda de la tabla

Con 258 cráteres, el número de tripletas posibles es

$$\binom{258}{3} = 2\,829\,056, \quad (4.7)$$

que es una cantidad perfectamente manejable en tiempo de cálculo pero incómoda en memoria y, sobre todo, innecesaria. La inmensa mayoría de esas tripletas son inútiles: unen cráteres separados por cientos de kilómetros, que nunca podrán aparecer juntos en la misma imagen.

De ahí sale el criterio de poda, que es puramente geométrico: se conserva una tripleta solo si **cabe dentro de un fotograma**. Como la huella de la cámara es proporcional a la altitud, y la altitud de operación está acotada, hay un lado máximo por encima del cual la tripleta es imposible de observar. Con el umbral empleado, la tabla se reduce a **65 773 tripletas**, un factor de 43 menos.

Para los mapas más densos que se ensayan en el Capítulo 5 hizo falta una poda adicional, porque el criterio anterior por sí solo dejaba pasar demasiadas. La segunda condición acopla el tamaño del triángulo al de sus cráteres: se exige que el lado mayor no supere treinta veces el diámetro del cráter más pequeño de la tripleta. La idea de fondo es que un triángulo enorme

formado por cráteres diminutos es geoméricamente válido pero inútil, porque a la altitud a la que ese triángulo cabe en la imagen, sus cráteres son ya demasiado pequeños para que el detector los vea.

La construcción de la tabla merece una nota de implementación. La forma directa de generar las combinaciones consume memoria proporcional al número total de tripletas *antes* de podar, lo que para los mapas más densos supera varios gigabytes y resulta impracticable. La implementación final invierte el orden: recorre cada cráter tomándolo como el *menor* de la tripleta, busca solo los vecinos que caen dentro del radio admisible, y genera las combinaciones dentro de ese subconjunto reducido. El resultado es idéntico al de la enumeración exhaustiva — se comprobó que ambas producen exactamente las mismas 65 773 tripletas — pero la memoria pasa a ser proporcional al número de tripletas que sobreviven. Es la entrada número 6 de la Tabla 1.3.

Finalmente, la tabla se ordena por la primera componente del descriptor, lo que convierte la búsqueda en vuelo en una búsqueda binaria sobre un vector ordenado, de coste logarítmico.

4.3. Resolución de un fotograma

Llegado este punto, el sistema dispone de una lista de detecciones en píxeles y de una tabla de tripletas del mapa indexada por descriptor. Falta el procedimiento que convierte lo uno en lo otro y despeja la pose del vehículo.

4.3.1. Generación de hipótesis

De las detecciones del fotograma se seleccionan las K mayores, se forman todas sus tripletas y se calcula el descriptor de cada una según la Ecuación (4.6). Cada descriptor observado se busca en la tabla del mapa mediante búsqueda binaria sobre la componente de ordenación, y se recuperan todas las tripletas del catálogo cuyo descriptor cae dentro de una tolerancia. Cada pareja (*tripleta observada*, *tripleta del mapa*) constituye una **hipótesis**: una conjetura sobre dónde está el vehículo.

Quedarse con las K mayores no es una limitación técnica sino una decisión de diseño. Los cráteres grandes son los que están en el mapa, los que el detector localiza con más precisión relativa y los que menos se confunden entre sí. Añadir detecciones pequeñas multiplicaría el número de tripletas sin aportar candidatos útiles.

4.3.2. Ajuste de la transformación de semejanza

Una hipótesis afirma que tres puntos conocidos del mapa se corresponden con tres puntos observados en la imagen. Si la hipótesis es correcta y la cámara mira al nadir, la relación entre ambos conjuntos es una **semejanza directa**: un giro, un cambio de escala y una traslación.

Conviene justificar por qué basta una semejanza y no hace falta una homografía completa. La proyección de un plano por una cámara estenopeica es, en general, una homografía de ocho grados de libertad. Pero aquí concurren dos circunstancias que la degeneran. La primera es que la cámara apunta al nadir, de modo que el plano del terreno es paralelo al plano imagen y desaparecen los dos términos de fuga de la homografía. La segunda es que el terreno, a las distancias de trabajo, es casi plano: su relieve tiene un recorrido de unos 3 km frente a una altitud de 120 a 170 km, es decir, menos del 3 %. Bajo esas dos condiciones la homografía se reduce a

$$\begin{pmatrix} u \\ v \end{pmatrix} = \sigma \begin{pmatrix} \cos \psi & \sin \psi \\ -\sin \psi & \cos \psi \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} + \begin{pmatrix} t_u \\ t_v \end{pmatrix}, \quad (4.8)$$

con cuatro parámetros: la escala σ , el giro ψ y la traslación (t_u, t_v) . El relieve residual no modelado es la principal fuente de error sistemático del método, y se manifiesta sobre todo

en la altitud, porque un cráter situado en una zona elevada aparece más grande de lo que le correspondería por su distancia al plano medio.

Resulta muy cómodo escribir esa transformación en el plano complejo. Se identifica cada punto del mapa (x_i, y_i) con el número complejo $s_i = x_i + j y_i$, y cada detección (u_i, v_i) con $d_i = u_i + j v_i$. La semejanza se reduce entonces a una única expresión afín:

$$d_i = a s_i + b, \quad a, b \in \mathbb{C}, \quad (4.9)$$

donde el módulo $|a|$ es el factor de escala, el argumento $\arg(a)$ es el ángulo de giro y b es la traslación. Dos números complejos, cuatro grados de libertad, que es exactamente lo que tiene una semejanza directa en el plano.

Con más de dos correspondencias el sistema está sobredeterminado y se resuelve por mínimos cuadrados. Tomando los centroides $m_s = \frac{1}{n} \sum_i s_i$ y $m_d = \frac{1}{n} \sum_i d_i$, la solución es

$$a = \frac{\sum_i \overline{(s_i - m_s)} (d_i - m_d)}{\sum_i |s_i - m_s|^2}, \quad b = m_d - a m_s, \quad (4.10)$$

donde la barra denota conjugación compleja. La deducción completa está en el Anexo E. El resultado es que el ajuste de una semejanza por mínimos cuadrados, que en notación matricial exige montar y resolver un sistema, se reduce a un cociente de dos sumatorios. El denominador es real y positivo salvo que los puntos sean coincidentes, de modo que no hay que invertir ninguna matriz ni preocuparse por el condicionamiento.

Hay un detalle de esta formulación que condiciona todo lo que viene después. Con exactamente tres correspondencias, los cuatro grados de libertad de la semejanza ajustan seis ecuaciones, y el sistema tiene solución con **residuo prácticamente nulo** sea cual sea la tripleta elegida. Dicho de otro modo: el residuo del ajuste *no sirve* para juzgar si la hipótesis es buena. Una tripleta completamente equivocada produce un ajuste igual de limpio que la correcta. De ahí que la verificación de la Sección 4.3.4 tenga que apoyarse en cráteres que no participaron en el ajuste.

4.3.3. De la semejanza a la pose

De los dos números complejos a y b se extraen directamente las tres magnitudes que el trabajo persigue.

La **escala** $|a|$ está en píxeles por kilómetro. Para una cámara estenopeica que mira al nadir desde una altitud h , con distancia focal f expresada en píxeles, la relación es inmediata:

$$|a| = \frac{f}{h} \quad \implies \quad h = \frac{f}{|a|}. \quad (4.11)$$

Ahí está la altitud, deducida y no medida. No hace falta altímetro: el tamaño aparente del terreno la contiene.

La **guiñada** es el argumento de a , con un cambio de signo respecto al convenio matemático habitual porque el eje vertical de la imagen apunta hacia abajo:

$$\psi = -\arg(a). \quad (4.12)$$

La **posición** es el punto del mapa que se proyecta sobre el centro de la imagen. Llamando c al centro del fotograma expresado como número complejo, e invirtiendo la Ecuación (4.9):

$$s_{\text{nave}} = \frac{c - b}{a}. \quad (4.13)$$

Las tres salen del mismo ajuste, sin información adicional. Es la razón de que el sistema pueda anunciar posición, orientación y altitud a partir de una sola imagen y un catálogo de coordenadas.

4.3.4. Verificación de la hipótesis

Todo lo anterior funciona igual de bien con una hipótesis correcta que con una equivocada. Tres puntos cualesquiera del mapa y tres detecciones cualesquiera definen una semejanza perfectamente calculable, que produce una posición, una guiñada y una altitud de aspecto impecable. Sin un filtro posterior, el sistema no falla: acierta el 100 % de las veces en producir una respuesta, y una fracción sustancial de esas respuestas es falsa.

Esta es la idea central del capítulo, y la que costó más iteraciones: **un emparejador geométrico no falla con un error, falla con una respuesta plausible**. La verificación no es un refinamiento opcional; es lo que separa el sistema de un generador de números creíbles.

El procedimiento adoptado es el siguiente. Aceptada provisionalmente una hipótesis, se aplica la semejanza estimada a *todos* los cráteres del mapa para predecir dónde deberían aparecer en la imagen y con qué tamaño. Después se comprueba, para cada detección, tres condiciones que deben cumplirse simultáneamente:

1. **Proximidad.** La detección debe caer cerca de la posición predicha de algún cráter del mapa, dentro de una tolerancia en píxeles.
2. **Ancla absoluta de tamaño.** El cociente entre el diámetro observado, llevado a kilómetros mediante la escala de la hipótesis, y el diámetro que ese cráter tiene en el catálogo debe caer dentro de un intervalo acotado.
3. **Coherencia relativa.** Ese mismo cociente, dividido por su mediana sobre todas las detecciones asociadas, debe ser próximo a la unidad; es decir, todos los cráteres deben estar equivocados en el mismo factor, o en ninguno.

Una hipótesis se acepta si el número de cráteres que superan las tres condiciones alcanza un mínimo, y se elige la que más corrobora. A eso se añade una restricción global: la altitud despejada por la Ecuación (4.11) debe caer dentro del rango físicamente posible del vuelo.

Que hagan falta tres condiciones y no una es el resultado de tres intentos fallidos, los números 8, 9 y 10 de la Tabla 1.3, y el razonamiento de por qué ninguna basta por separado es instructivo.

Verificar *solo* por proximidad es insuficiente porque en un terreno densamente craterizado casi cualquier posición predicha cae cerca de algún cráter. Verificar *solo* por coherencia relativa entre diámetros es peor, porque es trivialmente satisfacible: una hipótesis que se equivoca de escala por un factor uniforme produce cráteres que son todos consistentes entre sí, y la condición se cumple a la perfección mientras la solución es completamente falsa. Y acotar la mediana de ese factor tampoco resuelve nada, porque el descriptor es invariante a escala por construcción: una copia del mapa al doble de tamaño da descriptores idénticos, de modo que la escala no está determinada por el emparejamiento y hay que anclarla desde fuera.

Ese anclaje externo es el que aportan, juntas, la condición absoluta de tamaño y la restricción de altitud. Sin ellas el sistema es libre de elegir cualquier escala, y elige. Cada vez que se relajó una restricción confiando en que otra la cubriría, el resultado fue el mismo: no aparecieron errores, aparecieron soluciones bonitas y falsas.

4.3.5. El algoritmo completo

Reuniendo todo lo anterior, el procedimiento que resuelve un fotograma es:

1. Leer las detecciones del fotograma y quedarse con las K de mayor tamaño aparente.
2. Formar todas sus tripletas y calcular el descriptor de cada una según la Ecuación (4.6).
3. Para cada descriptor, buscar por búsqueda binaria las tripletas del mapa compatibles dentro de la tolerancia. Cada coincidencia es una hipótesis.
4. Para cada hipótesis, ajustar la semejanza con la Ecuación (4.10) usando la correspondencia de vértices que da el etiquetado canónico.
5. Descartar de entrada las hipótesis cuya altitud despejada caiga fuera del rango admisible.
6. Reproyectar el mapa completo y contar cuántas detecciones superan las tres condiciones de la Sección 4.3.4.
7. Quedarse con la hipótesis de mayor número de cráteres corroborados, siempre que alcance el mínimo exigido. Si ninguna lo alcanza, declarar el fotograma *sin solución*.
8. Reajustar la semejanza final empleando todos los cráteres corroborados, no solo los tres de la tripla, y extraer la pose con las ecuaciones (4.11) a (4.13).

La Figura 4.3 recoge el procedimiento completo en forma de diagrama.

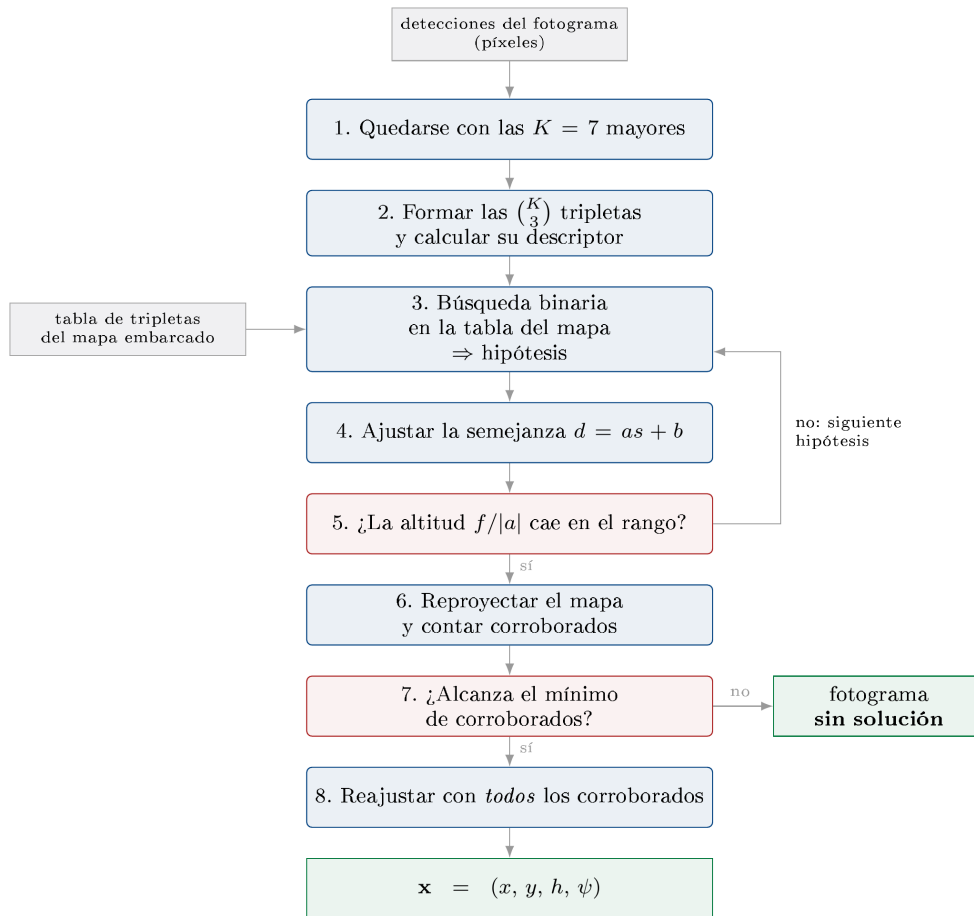


Figura 4.3: Algoritmo que resuelve un fotograma. Los pasos 1 a 3 generan hipótesis, los pasos 4 y 5 las evalúan, y los pasos 6 y 7 deciden cuál sobrevive. El paso 8 no busca: mide. Declarar el fotograma sin solución es una salida legítima del algoritmo, no un fallo.

El último paso importa más de lo que parece. La tripleta sirve para *encontrar* la solución, pero es una base pobre para *medirla*: tres puntos son el mínimo y cualquier error de detección se transmite íntegro. Con seis o siete cráteres corroborados, el ajuste por mínimos cuadrados promedia esos errores y la precisión mejora de forma apreciable.

Declarar un fotograma sin solución es, por último, una salida legítima y no un fallo. Un sistema de navegación que admite no saber dónde está es infinitamente preferible a uno que siempre responde algo. La proporción de fotogramas sin solución es una de las métricas que se reportan en el Capítulo 5, y su crecimiento con la disminución de la altitud es precisamente el fenómeno que allí se caracteriza.

4.4. Validación del posicionamiento

Esta sección fija el protocolo con el que se mide el motor de navegación y las comprobaciones de consistencia que se le han aplicado; las cifras se presentan en el Capítulo 5.

4.4.1. Magnitudes de error y criterio de acierto

Para cada fotograma resuelto se compara la solución $(\hat{x}, \hat{y}, \hat{h}, \hat{\psi})$ con la verdad de referencia exportada de Blender. Se emplean el error horizontal de posición, el error de altitud y el error de guiñada:

$$e_p = \sqrt{(\hat{x} - x)^2 + (\hat{y} - y)^2}, \quad e_h = |\hat{h} - h|, \quad e_\psi = |\text{wrap}(\hat{\psi} - \psi)|, \quad (4.14)$$

donde wrap lleva la diferencia angular al intervalo $(-180^\circ, 180^\circ]$. Se reporta la mediana y no la media, porque la distribución de e_p tiene cola pesada: un puñado de emparejamientos erróneos produce errores de centenares de kilómetros que desplazan la media sin representar el comportamiento típico. Se reportan además el máximo y los cuantiles.

Un fotograma se considera **mal emparejado** cuando e_p supera los 10 km. El umbral se fija en ese valor, y no menor, porque corresponde aproximadamente al diámetro del cráter más pequeño del mapa: por debajo de él el error es atribuible a la imprecisión del centro detectado, no a una identificación equivocada.

Se define la **cobertura** de un ensayo como la fracción de fotogramas para los que el sistema entrega una solución que supera la verificación de la Sección 4.3.4. Los fotogramas no resueltos no se promedian con los resueltos: se cuentan aparte, porque un sistema que resuelve pocos fotogramas con mucha precisión y otro que resuelve todos con menos no son comparables por el error mediano.

4.4.2. Protocolo de los ensayos

Se realizan tres ensayos, cada uno dirigido a una pregunta distinta. El primero recorre la secuencia de crucero completa en diferido, con las detecciones previamente calculadas y volcadas a fichero, y responde a la precisión alcanzable en régimen nominal. El segundo repite el mismo recorrido con el detector dentro del bucle, y responde a si el sistema sostiene la cadencia de fotogramas. El tercero recorre la secuencia de descenso con guiñada, y responde a hasta qué altitud inferior y con qué rotación el sistema sigue resolviendo.

En los tres, el sistema opera sin memoria entre fotogramas y sin recibir la altitud, según se estableció en la Sección 2.1. El mapa embarcado, el umbral de confianza del detector y todos los parámetros geométricos se fijan antes de empezar y no se ajustan a la vista de los resultados.

Para caracterizar el límite superior de altitud se emplea además una **cascada de tres mapas** de densidad creciente sobre la misma secuencia, lo que permite separar el efecto del número de cráteres disponibles del efecto de la altura.

4.4.3. Comprobaciones de consistencia

Más allá de medir el error, se han aplicado dos comprobaciones que no dependen de la verdad de referencia y que acotan la posibilidad de un fallo de implementación.

La primera es una **doble implementación independiente**. El motor de navegación se escribió dos veces, primero en Python durante el diseño y después en MATLAB para la versión definitiva. Esa duplicación, que empezó siendo un accidente del orden de trabajo, acabó siendo la validación más útil del desarrollo: ambas implementaciones, alimentadas con las mismas detecciones y el mismo mapa, producen un error mediano de posición que coincide hasta la tercera cifra decimal. Dos programas escritos en lenguajes distintos y con estructuras de datos distintas que llegan al mismo número no comparten errores de implementación; si hubiera un fallo, sería de concepto y estaría en la formulación.

La segunda es una **comprobación geométrica directa**, sin intervención del detector: proyectando hacia delante las posiciones conocidas de los cráteres del catálogo con los parámetros de cámara declarados se reproducen las etiquetas exportadas por Blender con error nulo a nivel de píxel, y la razón entre el ancho proyectado y el real vale la unidad. Esto cierra el círculo sobre la cadena de coordenadas descrita en la Sección 2.3: cualquier discrepancia que se observe procede de la percepción o del emparejamiento, y no de una inconsistencia entre el mundo generado y el modelo de cámara empleado para interpretarlo.

4.4.4. Implementación

El sistema está repartido entre dos entornos. La percepción vive en Python, que es donde están las bibliotecas de aprendizaje profundo, y el motor de navegación en MATLAB, que es donde resulta cómodo el trabajo geométrico y la representación gráfica. Para la demostración en tiempo real ambos tienen que convivir dentro del mismo bucle: MATLAB abre un intérprete de Python como proceso independiente, le pasa cada fotograma y recibe las detecciones, todo ello mientras va actualizando las gráficas. El detalle de esa comunicación, incluida la comprobación de que el fotograma que devuelve el detector es el mismo que se le pidió, se describe en el Anexo A, y el código está en el repositorio.

Capítulo 5

Simulación y validación del sistema de navegación

Este capítulo somete a prueba el sistema completo descrito en los Capítulos 2 a 4. Empieza presentando los tres escenarios simulados con el detalle suficiente para que puedan repetirse, sigue con el criterio de medida, y después recorre el detector, el ajuste del emparejador y los tres ensayos de navegación, cada uno más exigente que el anterior. La última parte se dedica a lo que el sistema *no* consigue, que en un trabajo de ingeniería suele ser la parte más informativa.

5.1. Escenarios de validación

Los tres escenarios comparten el mismo mundo, el mismo modelo de cámara y el mismo mapa embarcado, y se diferencian en la trayectoria, en la actitud y en cómo se alimenta el detector. La Tabla 5.1 los recoge de forma autocontenida.

Tabla 5.1: Los tres escenarios de validación. Los tres se ejecutan sobre el mismo terreno de 840×630 km, con la misma cámara (60° de campo horizontal, 1920×1080) y el mismo mapa embarcado de 258 cráteres, salvo donde se indica lo contrario.

	Ensayo 1 en diferido	Ensayo 2 en tiempo real	Ensayo 3 descenso
Trayectoria	crucero	crucero (la misma)	descenso
Posición inicial	(100, 350) km	(100, 350) km	(130, 350) km
Altitud	170 $\rightarrow$ 120 km	170 $\rightarrow$ 120 km	170 $\rightarrow$ 55 km
Actitud	nadir, sin guiñada	nadir, sin guiñada	nadir, guiñada de $\pm 180^\circ$
Fotogramas	700	700	700
Avance por fotograma	1,81 km	1,81 km	variable
Entrada al sistema	detecciones en fichero	vídeo, detector en el bucle	vídeo, detector en el bucle
Detector	de crucero	de crucero	de descenso
Pregunta que responde	¿qué precisión alcanza el motor geométrico?	¿aguanta con el detector dentro del bucle?	¿hasta qué altitud y con qué giro sigue resolviendo?

5.1.1. Condiciones de simulación

La trayectoria de crucero es la definida por las Ecuaciones (2.28) a (2.30): avance uniforme en x de 100 a 740 km, serpenteo sinusoidal de dos ciclos en y con amplitud de 130 km, y descenso suavizado de 170 a 120 km. La cámara apunta al nadir en todo momento. El recorrido sobre el suelo es de 1263 km.

La secuencia no tiene una base de tiempos propia, porque el sistema resuelve cada fotograma por separado y no usa la velocidad para nada. Si se quiere interpretarla como un

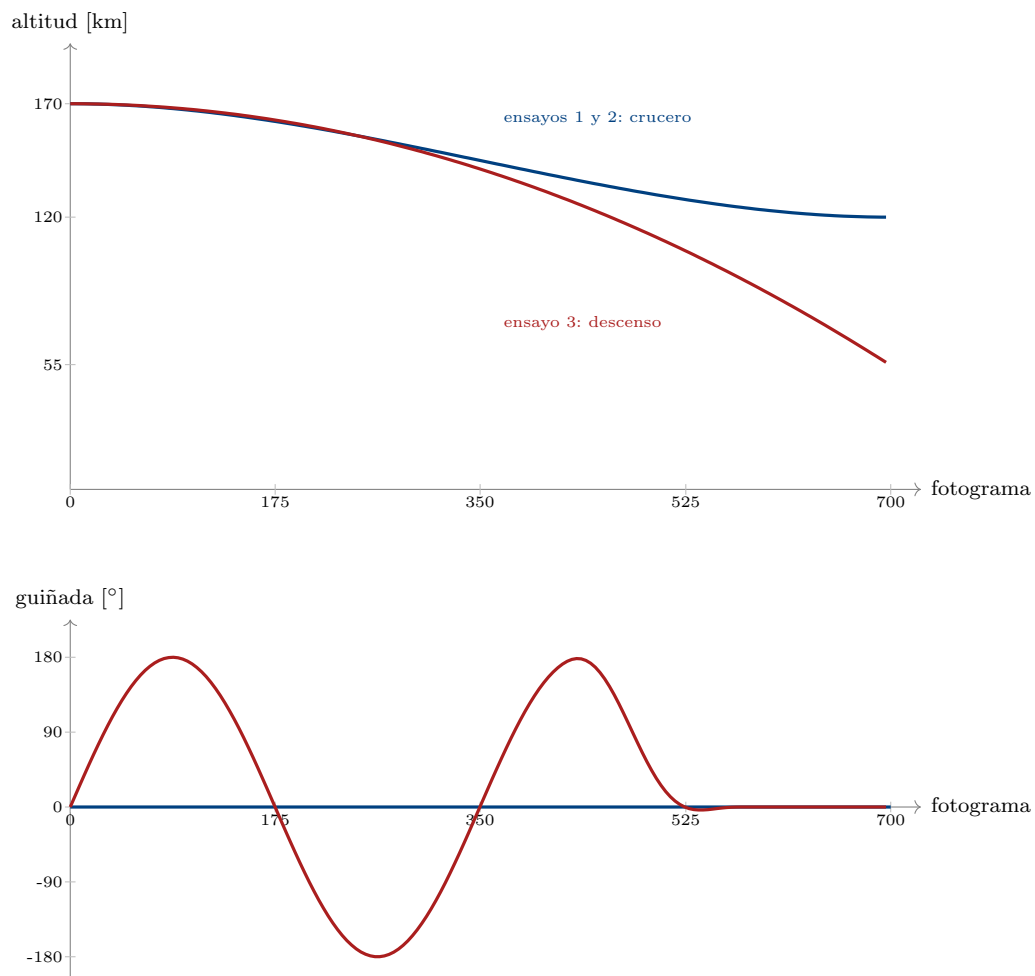


Figura 5.1: Perfiles de altitud y de guiñada de los tres escenarios. Los ensayos 1 y 2 recorren el mismo vuelo de crucero, de 170 a 120 km y sin giro. El ensayo 3 desciende hasta 55 km mientras la cámara gira una vuelta completa en guiñada.

vuelo real, conviene el siguiente cálculo: a 145 km sobre la superficie lunar la velocidad circular es de $1,61 \text{ km s}^{-1}$, de modo que avanzar 1,81 km entre fotogramas equivale a una cadencia de captura de aproximadamente 1,1 s por imagen, es decir, algo menos de un fotograma por segundo. Es una cadencia modesta y deliberadamente conservadora: el sistema no necesita más, porque no integra información entre fotogramas.

El escenario de descenso recorre un intervalo de altitud mucho mayor, de 170 a 55 km, y añade un giro en guiñada que recorre $\pm 180^\circ$. Es el único de los tres en el que el sistema tiene que resolver simultáneamente las cuatro incógnitas.

5.1.2. ¿Qué recibe el sistema y qué no?

Conviene repetir aquí, aunque ya se dijera en la Sección 2.1, las dos restricciones que hacen que estos ensayos sean exigentes:

- **La altitud no se proporciona.** El motor de navegación recibe únicamente coordenadas en píxeles. La altitud se despeja de la escala del emparejamiento mediante la Ecuación (4.11).
- **No hay memoria entre fotogramas.** Cada imagen se resuelve desde cero contra el catálogo completo, sin filtro, sin seguimiento y sin usar la solución del fotograma anterior para acotar la búsqueda.

La verdad de referencia —posición, altitud y guiñada en cada fotograma— procede de Blender y no interviene en ningún punto del cálculo: se usa exclusivamente para medir el error al final.

5.1.3. Lo que ve la cámara

La Figura 5.2 muestra cuatro fotogramas repartidos a lo largo del vuelo de crucero, con las detecciones superpuestas. Sirve para hacerse una idea de la escena real sobre la que trabaja el sistema, y anticipa el resultado más importante del capítulo: el número de detecciones cae de 23 a 10 conforme la nave desciende de 169 a 120 km, simplemente porque la cámara ve menos terreno.

5.2. ¿Cómo se mide?

Todos los ensayos se evalúan contra la verdad de referencia que exporta Blender, descrita en la Sección 3.6. Para cada fotograma se conocen exactamente la posición de la cámara, su altitud y su ángulo de guiñada, de modo que el error no se estima: se calcula.

Se usan cuatro métricas. La **cobertura** es el porcentaje de fotogramas para los que el sistema entrega una solución que supera la verificación descrita en la Sección 4.3.4. Es la métrica de fiabilidad: un navegador que acierta mucho pero solo responde en la mitad de los fotogramas no sirve para cerrar un lazo de control. El **error de posición** es la distancia entre la posición estimada y la real, medida en el plano del terreno. El **error de altitud** y el **error de guiñada** completan la pose.

Una decisión merece justificarse. Los errores se resumen con la *mediana* y no con la media. La razón es que la distribución de errores de un sistema de emparejamiento no es una campana: la inmensa mayoría de los fotogramas caen en unos pocos centenares de metros, y unos pocos, cuando el emparejamiento es erróneo, se van a más de cien kilómetros. Un solo fotograma de éstos arrastra la media de todo un ensayo y la convierte en un número que no describe a ninguno de los fotogramas medidos. La mediana, en cambio, dice qué le pasa al fotograma típico. Los fotogramas atípicos no se esconden: se cuentan aparte, porque son justamente el fallo que importa. La Figura 5.4 hace visible el argumento: en los dos ensayos

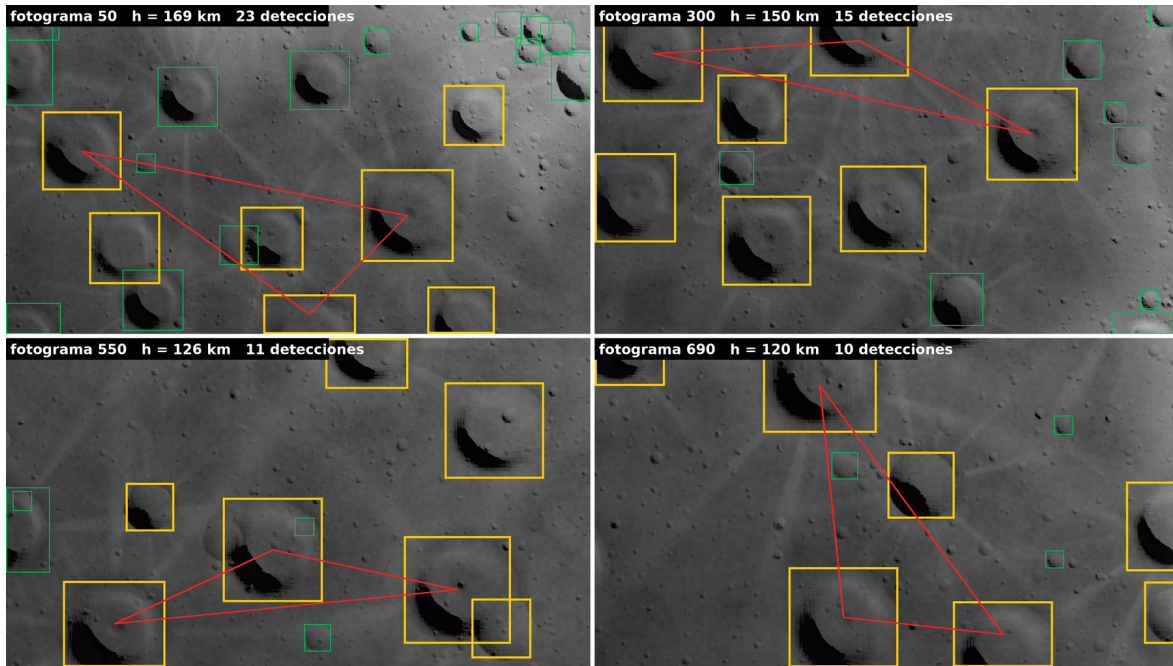


Figura 5.2: Cuatro fotogramas del vuelo de crucero, a 169, 150, 126 y 120 km de altitud. En verde, todas las detecciones del detector; en amarillo, las $K = 7$ de mayor tamaño aparente, que son las únicas que entran en la formación de tripletas; en rojo, una de las tripletas candidatas. Obsérvese cómo disminuye el número de cráteres visibles a medida que la nave desciende, que es la causa del límite de altitud de la Sección 5.8.

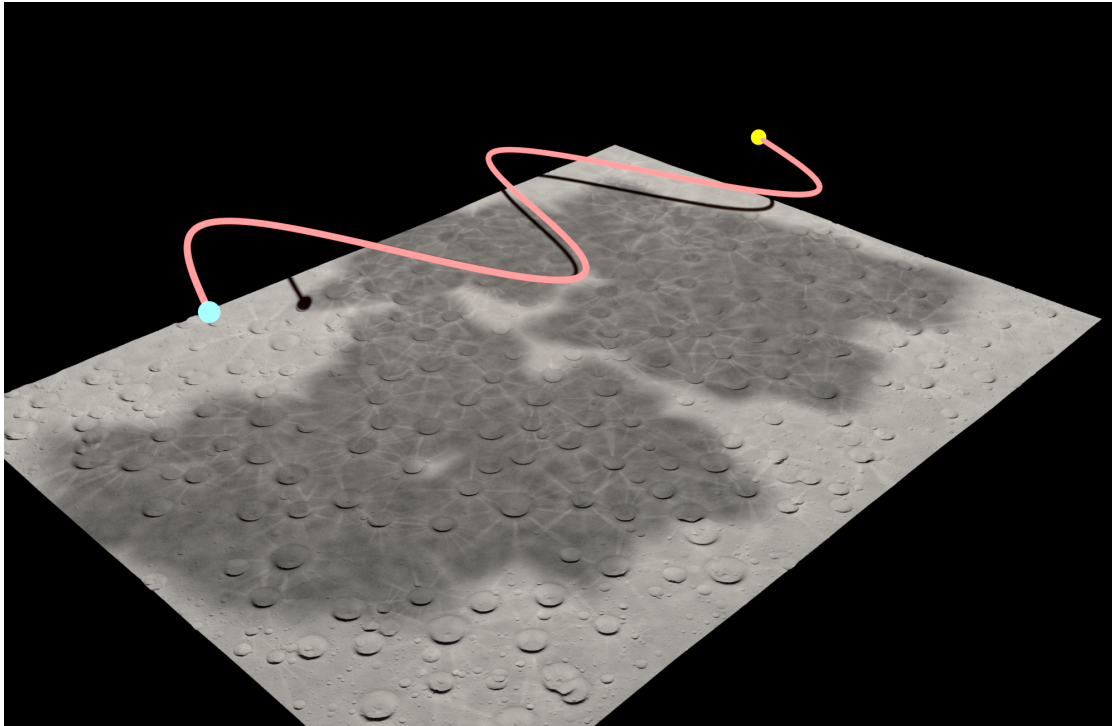


Figura 5.3: Vista tridimensional del terreno sintético con la trayectoria de crucero dibujada sobre él. La cámara recorre 1263 km de oeste a este mientras desciende de 170 a 120 km.

la media queda a la derecha de la mediana, cerca del doble, arrastrada por una cola de pocos fotogramas.

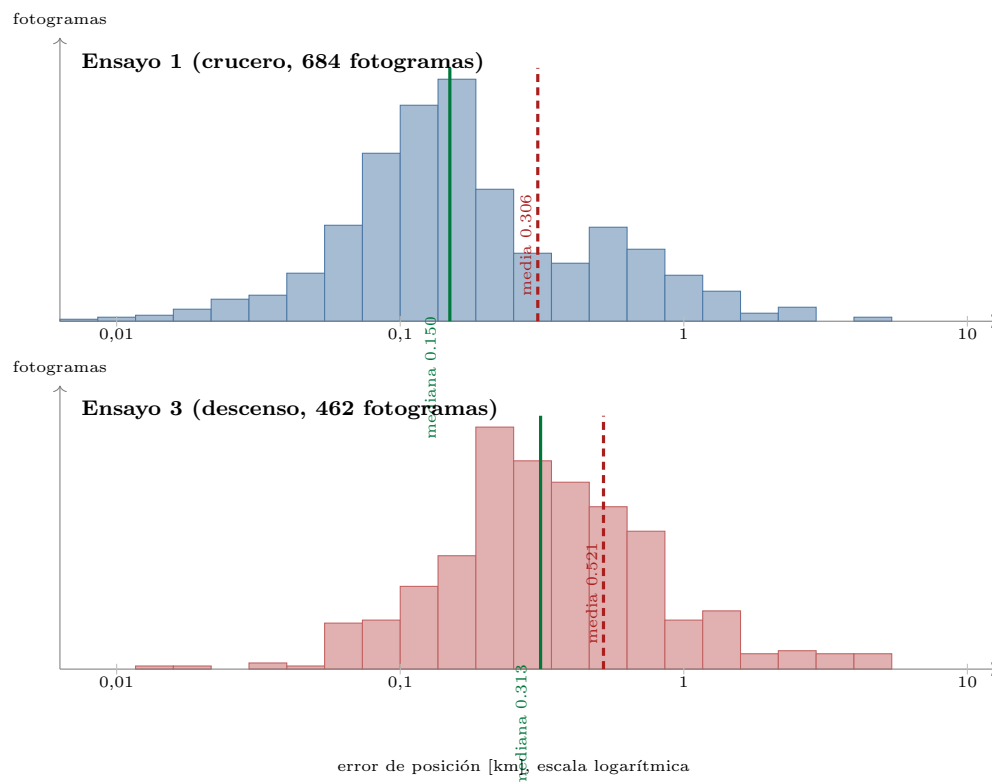


Figura 5.4: Distribución del error de posición en los ensayos 1 y 3, en escala logarítmica. La línea continua marca la mediana y la discontinua la media. La separación entre ambas es la razón por la que en todo el capítulo se informa la mediana: la cola derecha, de unos pocos fotogramas, desplaza la media hasta el doble del valor típico.

Eso obliga a distinguir dos cosas que es fácil confundir. Un fotograma **resuelto** es aquel para el que el sistema entrega una respuesta. Un fotograma **correcto** es aquel cuya respuesta está cerca de la verdad. La diferencia entre ambos son los *falsos emparejamientos*: soluciones que pasan la verificación geométrica pero corresponden a una zona equivocada del mapa. A lo largo del capítulo se considera correcta una solución con error de posición inferior a 10 km. Dónde se ponga exactamente esa línea resulta indiferente, porque los datos no dejan ninguna zona gris: de todas las soluciones entregadas en los tres ensayos, la peor de las correctas se queda en 5,38 km y la mejor de las erróneas está en 142 km. No hay nada en medio.

5.3. Rendimiento del detector

5.3.1. El ajuste fino sobre el terreno sintético

El detector se evaluó sobre el conjunto de validación del terreno sintético, separado del de entrenamiento por el hueco geométrico de la Sección 3.5.1. La Tabla 5.2 compara tres modelos: el que se venía usando al principio del trabajo, un segundo modelo afinado sobre fotografías reales de cráteres, y un tercero afinado sobre los renders de Blender, que es el que finalmente se emplea. Las dos últimas filas repiten, para comodidad del lector, las cifras que la Tabla 3.5 justifica desde el punto de vista del entrenamiento.

El salto es grande y tiene una explicación sencilla: el modelo afinado sobre fotografías reales está resolviendo un problema parecido pero no el mismo. La iluminación, el contraste y el grano del terreno sintético son distintos de los de una fotografía orbital, y esa diferencia —

Tabla 5.2: Comparación de los tres detectores sobre el mismo conjunto de validación. La precisión y la exhaustividad se miden al umbral por defecto de la biblioteca.

Modelo	mAP@50	mAP@50–95	Precisión	Exhaustividad
Punto de partida	0,123	0,038	0,296	0,198
Afinado con imágenes reales	0,380	0,125	0,614	0,435
Afinado con Blender	0,669	0,514	0,952	0,675

lo que en la literatura se denomina salto de dominio — basta para que un detector competente en un mundo sea mediocre en el otro. Es un resultado esperable, pero conviene tenerlo medido: justifica por qué el trabajo dedica una etapa de entrenamiento propia en lugar de reutilizar un modelo publicado.

Merece un comentario la exhaustividad de 0,675, que a primera vista parece baja para un modelo cuya precisión es de 0,952. No es un límite del modelo sino del protocolo de evaluación. Las imágenes de validación contienen del orden de 423 cráteres etiquetados de media, mientras que la biblioteca limita la salida a 300 detecciones por imagen; el detector afinado entrega exactamente esas 300 en todas las imágenes, es decir, está saturando el tope. La exhaustividad medida así no puede superar el cociente entre ambas cifras. Lo relevante para este trabajo, en todo caso, no es cuántos cráteres diminutos encuentra, sino cuántos de los que están en el mapa embarcado encuentra, y eso se mide en la Sección 5.3.3.

Las curvas de entrenamiento aparecen en la Figura 3.2. El ajuste fino se detuvo por parada temprana en la época 43 de las 60 previstas, después de 20 épocas sin mejorar la métrica de validación y de hora y media de cálculo sobre la tarjeta gráfica del equipo de desarrollo.

5.3.2. El coste real del fallo de filtrado

La Sección 3.7 describe el fallo de filtrado de detecciones solapadas que vaciaba fotogramas enteros. Conviene cuantificar lo que costaba, porque el detector es exactamente el mismo antes y después: lo único que cambió fue procesar las imágenes de una en una.

Tabla 5.3: El mismo modelo, las mismas imágenes y el mismo umbral, evaluados antes y después de corregir el tamaño de lote, ambos al umbral de confianza 0,25 que se empleaba entonces. Se mide sobre los cráteres del mapa embarcado, que son los únicos que el navegador puede usar.

	Antes	Después
Cráteres del mapa recuperados	59,9 %	99,4 %
Fotogramas con al menos tres cráteres	57,3 %	98,1 %
Fotogramas sin ninguna detección	41,4 %	0 %

La fila que decide el resultado es la segunda. El motor de emparejamiento necesita tres cráteres para formar una tripleta; por debajo de tres, el fotograma está perdido antes de empezar. Con el fallo activo, cuatro de cada diez fotogramas eran inutilizables, y el sistema completo habría alcanzado como mucho una cobertura del 57 % por mucho que se afinara el emparejador. El mismo detector, alimentado correctamente, deja el 98 % de los fotogramas en condiciones de resolverse. Ninguna mejora del algoritmo de navegación habría compensado eso.

5.3.3. El punto de operación

El umbral de confianza decide qué detecciones se aceptan. Un umbral bajo entrega muchas cajas, algunas falsas; uno alto entrega pocas y seguras. La Tabla 5.4 recoge el barrido.

El resultado es más limpio de lo que cabría esperar: la capacidad de recuperar los cráteres del mapa no se degrada en absoluto al subir el umbral de 0,05 a 0,70. La explicación está en el tamaño. Los cráteres del mapa son los grandes, los de diámetro superior a 10 km, y

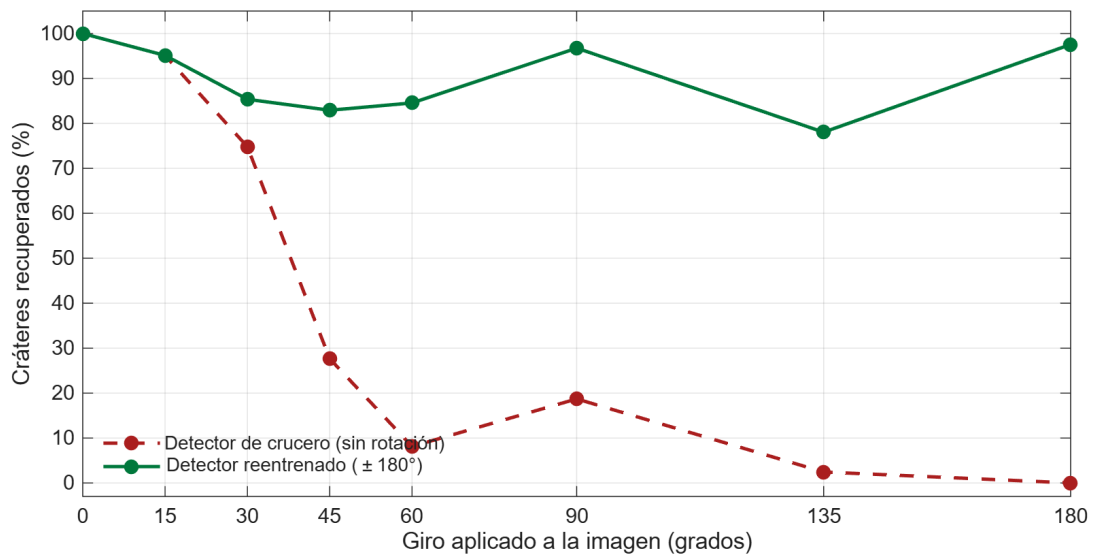
Tabla 5.4: Comportamiento del detector afinado frente al umbral de confianza, medido sobre los cráteres del mapa embarcado. La última columna es el tiempo de inferencia por fotograma.

Umbral	0,05	0,25	0,50	0,70
Cajas por fotograma	583	394	316	251
Cráteres del mapa recuperados	99,4 %	99,4 %	99,4 %	99,4 %
Fotogramas con tres o más	98,7 %	98,1 %	97,5 %	96,8 %
Tiempo por fotograma (s)	0,44	0,31	0,26	0,27

son precisamente los que el detector reconoce con más seguridad; lo que se pierde al subir el umbral son cráteres pequeños que el mapa no contiene y que el navegador no puede usar. Subir el umbral a 0,70 descarta más de la mitad de las cajas — de 583 a 251 — sin perder nada de lo que importa, y reduce a la vez el trabajo combinatorio del emparejador, que crece con el cubo del número de detecciones. Ése es el punto de operación adoptado en todos los ensayos.

5.3.4. Robustez frente a la rotación

El detector de crucero se entrenó sin rotación, porque durante el crucero la cámara mantiene una orientación fija. La fase de descenso rompe esa hipótesis: la nave gira una vuelta completa en guiñada. La Figura 5.5 mide qué pasa cuando se le presentan al detector las mismas imágenes giradas; los valores numéricos están en la Tabla 3.7.

**Figura 5.5:** Porcentaje de cráteres del mapa recuperados en función del ángulo de giro aplicado a la imagen. El detector de crucero se derrumba a partir de los 30° y no encuentra absolutamente nada a 180°; el detector reentrenado con aumento por rotación se mantiene por encima del 78 % en todo el recorrido.

El comportamiento del detector de crucero es llamativo. A 0° recupera el 100 % de los cráteres del mapa; a 15° todavía el 95 %; a 45° ya solo el 28 %; y a 180° exactamente ninguno. Una red convolucional no es invariante al giro por construcción, y si el entrenamiento nunca le muestra un cráter iluminado desde abajo, no aprende que eso también es un cráter. El caso de 180° es el peor posible porque invierte la dirección de la sombra, que es la señal principal por la que se reconoce un cráter en una imagen monocroma.

El reentrenamiento con aumento por rotación de $\pm 180^\circ$ resuelve el problema, pero no gratis. El modelo reentrenado devuelve cajas sistemáticamente mayores que las verdaderas: un 16 % más anchas incluso a 0°, y hasta un 45 % más anchas en los ángulos intermedios. La causa es geométrica y está explicada en la Sección 3.9: al girar una imagen, la caja

rectangular alineada con los ejes que envuelve un objeto crece, y el aumento por rotación enseña al modelo esas cajas infladas como si fueran correctas. La consecuencia práctica es que la fase de descenso necesita una ventana de verificación de diámetro más ancha, y que el diámetro estimado por el detector deja de ser una medida fiable del tamaño del cráter. Es un compromiso deliberado: se sacrifica precisión en el tamaño a cambio de detectar algo en absoluto.

Esto justifica la decisión de mantener *dos* detectores, uno por fase, en lugar de usar el modelo reentrenado para todo. Durante el cruce no hay giro, y usar allí un modelo con cajas infladas empeoraría la verificación sin ninguna contrapartida.

5.4. Ajuste del emparejador

Dos parámetros gobiernan el motor de emparejamiento: cuántos cráteres del fotograma se usan para formar tripletas, y qué cráteres entran en el mapa embarcado. Ambos se barrieron conjuntamente porque interactúan.

El primero es el número K de detecciones más grandes que se retienen por fotograma. Como el número de tripletas crece con K^3 , cada cráter añadido cuesta caro; pero con pocos cráteres el sistema se queda sin candidatos en cuanto uno se pierde. La Tabla 5.5 recoge el barrido para el mapa finalmente adoptado.

Tabla 5.5: Efecto del número K de detecciones retenidas por fotograma, con el mapa de 258 cráteres. La precisión es la fracción de detecciones retenidas que corresponden realmente a un cráter del mapa; la exhaustividad, la fracción de cráteres del mapa presentes en la imagen que quedan retenidos.

K	4	5	6	7	10
Precisión	0,898	0,869	0,855	0,812	0,624
Exhaustividad	0,605	0,732	0,864	0,957	0,994
Fotogramas con cuatro o más	66,2 %	85,4 %	98,1 %	98,7 %	98,7 %

El codo está en $K = 7$. Hasta ahí, cada detección añadida compra exhaustividad barata; a partir de ahí, la exhaustividad ya no puede subir mucho más — está en 0,957 — y lo único que crece es el número de detecciones espurias, que son las que generan tripletas sin correspondencia y hacen trabajar al emparejador en balde. El valor $K = 7$ deja además el 98,7 % de los fotogramas con al menos cuatro cráteres del mapa, que es la condición para poder verificar una triplete con un cuarto cráter independiente.

El segundo parámetro es el umbral de diámetro del mapa. Un mapa más denso contiene más cráteres pequeños y por tanto sigue siendo útil a menor altitud, pero su tabla de tripletas crece con el cubo del número de cráteres. Los tres mapas construidos se recogen en la Tabla 5.6; su efecto sobre la altitud mínima alcanzable se caracteriza en la Sección 5.8.

Tabla 5.6: Los tres mapas embarcados ensayados. La última columna es el número de tripletas antes de la poda descrita en la Sección 4.2.4.

Umbral de diámetro	$D \geq 10$ km	$D \geq 6$ km	$D \geq 4$ km
Cráteres en el catálogo	281	529	973
Cráteres en el mapa	258	440	753
Tripletas posibles	2 829 056	14 100 680	70 876 376

La diferencia entre las dos primeras filas es el filtro de frescura. Un cráter muy erosionado tiene el borde difuminado y apenas proyecta sombra, de modo que el detector no lo encuentra de forma fiable; incluirlo en el mapa no aporta posiciones nuevas y sí añade tripletas que nunca se emparejan. Se descartan por tanto los cráteres cuyo parámetro de frescura es inferior a 0,30, lo que elimina 23 de los 281 candidatos del mapa principal.

5.5. Ensayo 1: navegación en diferido

El primer ensayo recorre los 700 fotogramas del vuelo de crucero, de 170 km a 120 km de altitud y sin rotación, usando las detecciones exportadas previamente a un fichero. Al no ejecutar el detector dentro del bucle, este ensayo mide el motor de navegación aislado.

Tabla 5.7: Ensayo 1. Los errores se calculan sobre los fotogramas resueltos.

Fotogramas resueltos	684 de 700 (97,7 %)
Emparejamientos falsos	0
Error de posición, mediana	0,150 km
Error de posición, percentil 95	1,048 km
Error de posición, máximo	5,38 km
Error de altitud, mediana	0,431 km
Cráteres corroborados por fotograma, mediana	6

El error mediano de 150 m sobre un terreno de $840 \text{ km} \times 630 \text{ km}$ equivale a poco más de una diezmilésima de la dimensión del mapa. Puesto en relación con la altitud de vuelo — entre 170 y 120 km — el error es aproximadamente el uno por mil de la altura. Conviene ser prudente al comparar esta cifra con las de las misiones reales citadas en la Sección 1.2.7: el sistema de Marte 2020 alcanzó unos 40 m (Chen et al., 2021) y SLIM del orden de 10 m (Ishida et al., 2025), pero ambos midieron en las decenas de kilómetros finales del descenso, donde cada píxel cubre mucho menos terreno, y ambos combinaban la medida óptica con una unidad inercial. La comparación honesta no es entre los números absolutos sino entre los órdenes de magnitud del error relativo, y en ese terreno el resultado de este trabajo es razonable para un sistema que resuelve cada fotograma de forma independiente y sin ningún filtro que acumule información entre fotogramas.

La Figura 5.6 presenta el mismo resultado de forma gráfica. A la escala del terreno, las 684 posiciones estimadas caen sobre la trayectoria real sin que se distingan de ella; ampliando el error se ve que la nube está centrada en el origen, sin sesgo apreciable en ninguna dirección, lo que indica que no hay ningún error sistemático de calibración residual.

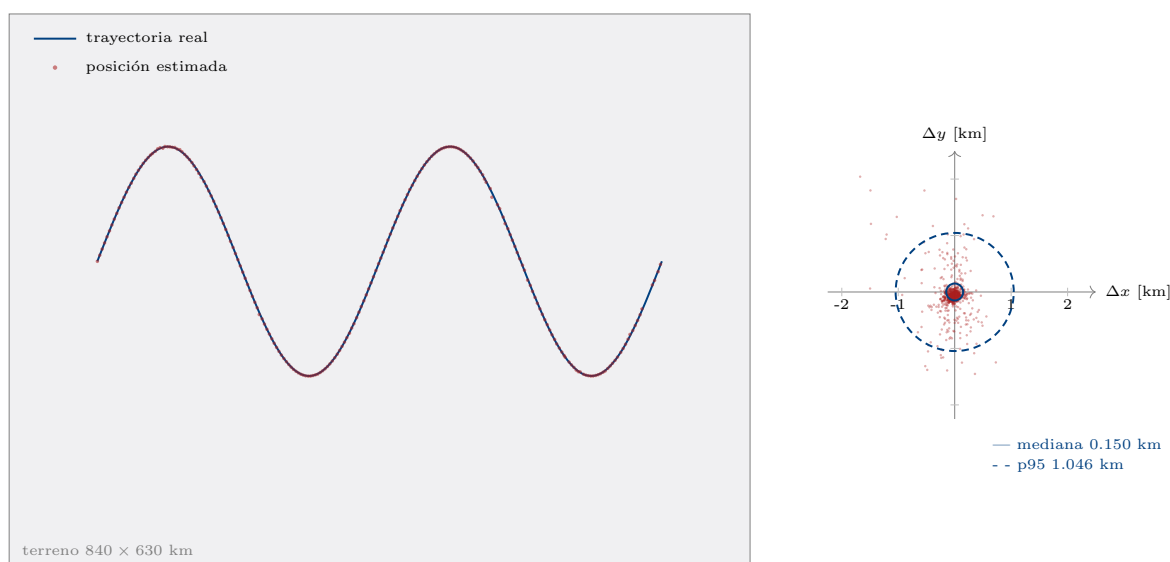


Figura 5.6: Izquierda: trayectoria real del ensayo 1 (línea azul) y posiciones estimadas por el sistema (puntos rojos), una por fotograma resuelto. A la escala del terreno las estimaciones son indistinguibles de la trayectoria. Derecha: el mismo error visto de cerca, como dispersión de $(\Delta x, \Delta y)$ en kilómetros; los círculos marcan la mediana y el percentil 95. La nube está centrada, sin sesgo apreciable en ninguna dirección.

No se produjo ningún emparejamiento falso: la peor de las 684 soluciones se queda en

5,38 km. Los 16 fotogramas no resueltos se concentran, como se ve en la Figura 5.7, en el tramo final del recorrido, que es también el de menor altitud. Es el primer indicio del comportamiento que la Sección 5.8 caracteriza en detalle.

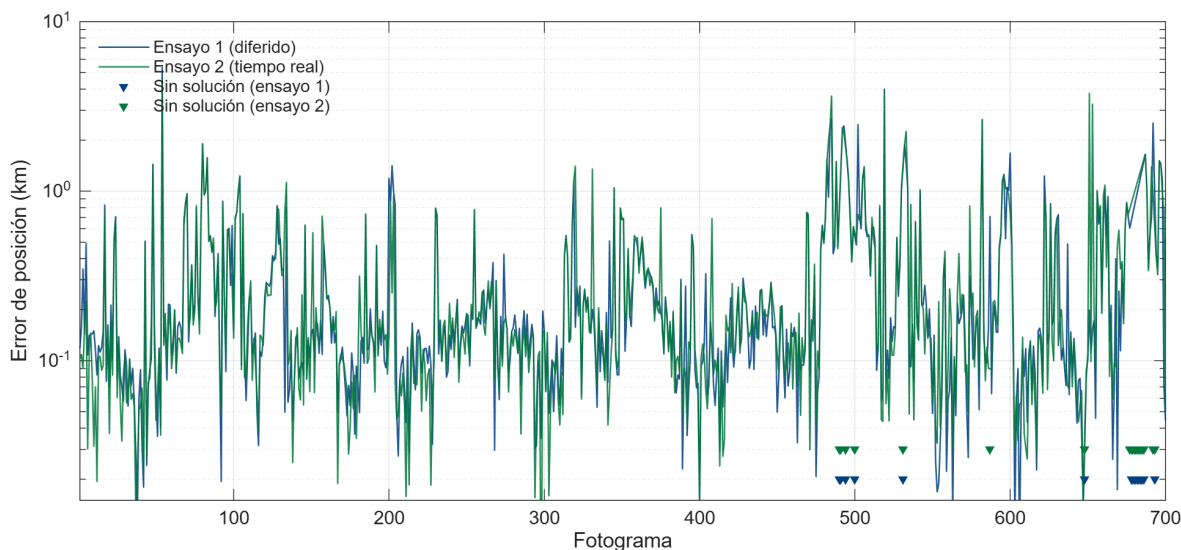


Figura 5.7: Error de posición fotograma a fotograma en los ensayos 1 y 2, en escala logarítmica. Los triángulos de la parte inferior marcan los fotogramas para los que el sistema no entregó solución. Las dos curvas se solapan casi por completo, lo que confirma que ejecutar el detector dentro del bucle no degrada la precisión.

Una observación sobre el coste de la verificación. La mediana de hipótesis evaluadas por fotograma es de 34, pero el máximo registrado supera las 1500. Esa dispersión de dos órdenes de magnitud tiene consecuencias para un sistema de tiempo real: el caso medio es barato, pero el peor caso no lo es, y el presupuesto de cómputo hay que dimensionarlo por el segundo, no por el primero.

5.6. Ensayo 2: navegación en tiempo real

El segundo ensayo repite el mismo vuelo, pero ejecutando el detector dentro del bucle sobre el vídeo comprimido, con la representación gráfica actualizándose a la vez. Es la demostración de que el sistema funciona como un navegador y no solo como un procedimiento de análisis posterior.

Sobre 700 fotogramas procesados, el sistema resolvió 681 (97,3%), con un error mediano de posición de 0,146 km y un máximo de 4,67 km. No se produjo ningún emparejamiento falso.

Dos conclusiones. La primera es que trabajar sobre vídeo comprimido en lugar de sobre las imágenes originales no cuesta precisión: el error mediano difiere del obtenido en diferido en cuatro metros, y las dos curvas de la Figura 5.7 se superponen. La compresión degrada detalles finos que el detector no necesita, porque los cráteres que usa ocupan decenas o centenares de píxeles.

La segunda es que la penalización por ejecutar el detector dentro del bucle es mínima y no afecta a la calidad de la solución sino a su disponibilidad: tres fotogramas menos de 700, y ninguna solución errónea. Que un sistema que además está dibujando la trayectoria en pantalla mientras navega pierda tres fotogramas de setecientos respecto al procesamiento en diferido es un resultado mejor de lo que cabía anticipar.

5.7. Ensayo 3: descenso con rotación en guiñada

El tercer ensayo es el exigente. La cámara desciende de 170 km a 55 km mientras la nave completa un giro de $\pm 180^\circ$ en guiñada, de modo que el sistema tiene que resolver simultáneamente posición, altitud y orientación sin ninguna información previa sobre ellas.

Tabla 5.8: Ensayo 3. Los errores se calculan sobre los 462 fotogramas correctos; los cinco emparejamientos falsos se contabilizan aparte.

Fotogramas resueltos	467 de 700 (66,7 %)
De ellos, correctos	462 (98,9 %)
Emparejamientos falsos	5
Error de posición, mediana	0,314 km
Error de posición, percentil 95	1,50 km
Error de altitud, mediana	0,95 km
Error de guiñada, mediana	0,11°
Error de guiñada, percentil 95	1,89°

El error de guiñada es el resultado más satisfactorio del ensayo. Una mediana de una décima de grado sobre un giro completo significa que la orientación sale prácticamente gratis del mismo ajuste que da la posición, sin necesidad de ninguna búsqueda angular. Es la consecuencia directa de haber formulado el problema como una semejanza en el plano complejo, según la Ecuación (4.9): el argumento del número complejo a es el ángulo de guiñada, y se obtiene resolviendo un sistema lineal.

El error de altitud, con una mediana de casi un kilómetro, es notablemente peor que el de posición. También tiene explicación. La altitud sale del módulo de la escala según la Ecuación (4.11), y el módulo de la escala es sensible al tamaño aparente de la constelación de cráteres, mientras que la posición depende solo de dónde está su centro. Un error pequeño en la posición de los vértices de la tripleta apenas mueve el centro, pero sí cambia el tamaño. A esto se suma que el detector de descenso, según la Sección 5.3.4, devuelve cajas infladas.

Lo que domina el ensayo, sin embargo, es la caída de cobertura al 66,7 %, y no está repartida: está concentrada en un umbral de altitud.

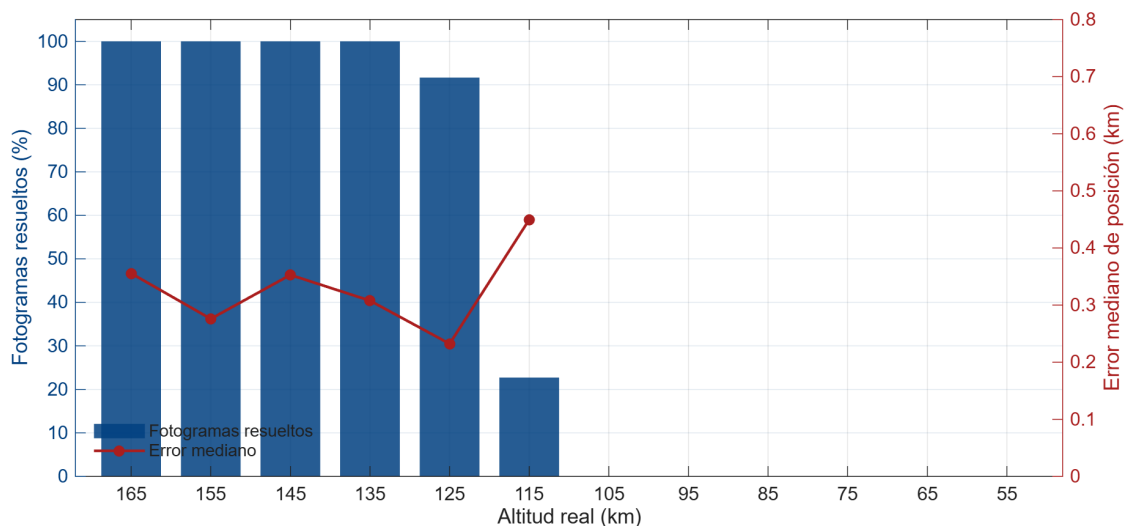


Figura 5.8: Fotogramas resueltos y error mediano por franjas de 10 km de altitud durante el descenso. La cobertura es total por encima de 130 km y se desploma entre 120 y 110 km. El error mediano de los fotogramas que sí se resuelven no empeora al bajar: el fallo no es de precisión, es de disponibilidad.

Por encima de 130 km el sistema resuelve el 100 % de los fotogramas. Entre 130 y 120 km todavía el 91,7 %. Entre 120 y 110 km cae al 22,7 %, y por debajo de 110 km no resuelve ninguno. El primer fallo se produce en el fotograma 458, a 120,8 km, y el último acierto en el

504, a 110,5 km.

La causa es puramente geométrica y estaba anticipada en la Sección 4.2. El campo de visión de la cámara cubre un cuadrado sobre el terreno cuyo lado es proporcional a la altitud. Al descender, ese cuadrado encoge con el cuadrado de la altura, y el número de cráteres del mapa que caben dentro encoge con él. Por debajo de cierta altitud quedan menos de tres cráteres del mapa a la vista, y sin tres cráteres no hay tripleta que emparejar. El sistema no falla por imprecisión: se queda literalmente sin cráteres.

Los cinco emparejamientos falsos merecen atención porque son el modo de fallo más peligroso. Cuatro de ellos se agrupan entre los fotogramas 466 y 470, justo en la franja donde el sistema empieza a quedarse sin cráteres, y el quinto en el 504. Todos ellos tienen errores de entre 142 y 149 km, es decir, apuntan a una zona completamente distinta del mapa. El patrón es coherente: cuando quedan exactamente tres cráteres a la vista, la verificación por cuarto cráter independiente descrita en la Sección 4.3.4 no puede aplicarse, y una tripleta con un descriptor casualmente parecido pasa el filtro. La lección de diseño es clara: la verificación no debería ser opcional, y un fotograma que no puede verificarse debería declararse no resuelto en lugar de resuelto.

5.8. El muro de altitud y la densidad del mapa

Si la hipótesis de la sección anterior es correcta — que el límite lo impone quedarse sin cráteres del mapa dentro del campo de visión — entonces un mapa más denso debería bajar ese límite. El experimento consiste en repetir el mismo descenso, con el mismo vídeo y las mismas detecciones, cambiando únicamente el mapa embarcado por los tres de la Tabla 5.6.

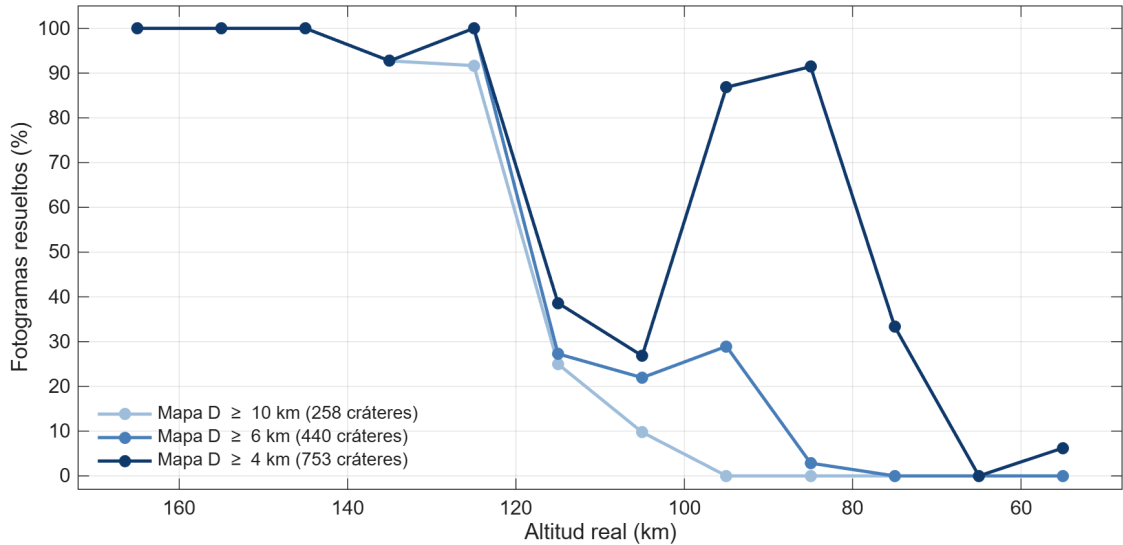


Figura 5.9: Efecto de la densidad del mapa embarcado sobre la altitud mínima de operación. Los tres mapas se comportan igual mientras hay cráteres de sobra; se separan justamente donde empiezan a escasear.

Tabla 5.9: Resultado del descenso completo con los tres mapas. La última fila es la altitud del fotograma resuelto más bajo.

	$D \geq 10$ km	$D \geq 6$ km	$D \geq 4$ km
Cráteres en el mapa	258	440	753
Fotogramas resueltos	468 (66,9 %)	490 (70,0 %)	562 (80,3 %)
Error de posición, mediana	0,314 km	0,315 km	0,356 km
Altitud mínima resuelta	108,5 km	88,1 km	57,6 km

La hipótesis queda confirmada. Triplicar la densidad del mapa baja el suelo de operación de 108 a 58 km, casi a la mitad, y el precio en precisión es despreciable: el error mediano pasa de 0,314 a 0,356 km, un 13 % peor. El precio real no está en la precisión sino en la memoria y en el cómputo: la tabla de tripletas pasa de 2,8 a 70,9 millones de entradas, veinticinco veces más, para un sistema que se supone embarcado.

Hay un matiz que conviene no ocultar. La cobertura del mapa más denso no cae de forma limpia: se mantiene por encima del 86 % entre 100 y 80 km, baja al 33 % entre 80 y 70 km, se anula entre 70 y 60 km y vuelve a dar algún acierto aislado por debajo. Ese comportamiento a tirones sugiere que por debajo de los 100 km el resultado depende de qué haya concretamente bajo la cámara en cada momento: si la trayectoria sobrevuela una zona rica en cráteres grandes, resuelve; si cruza una llanura, no. La altitud mínima de 57,6 km de la Tabla 5.9 es por tanto el mejor caso encontrado, no una garantía.

5.9. Repetibilidad

Un número solo vale lo que vale su reproducibilidad. Los tres ensayos se ejecutaron tres veces consecutivas con el mismo vídeo, el mismo modelo y los mismos parámetros, archivando por separado los ficheros de resultados de cada pasada.

Las tres pasadas son idénticas byte a byte. No parecidas: los ficheros de resultados de las tres tienen la misma suma de verificación, de modo que los 700 fotogramas de cada ensayo produjeron exactamente los mismos valores hasta el último decimal.

Que el ensayo en diferido saliera determinista era previsible, porque lee las detecciones de un fichero y el motor de emparejamiento es puramente combinatorio, sin ninguna fuente de azar. Los ensayos en directo no lo eran tanto. Ejecutan la red neuronal en la tarjeta gráfica dentro del bucle, y la aritmética en punto flotante sobre este tipo de hardware no garantiza en general el mismo orden de operaciones entre ejecuciones. Bastaría una diferencia en el último bit para que una detección cuya confianza cayera justo sobre el umbral de 0,70 entrase en una pasada y no en la siguiente; si esa detección fuera el tercer cráter de un fotograma justo, el fotograma entero pasaría de resuelto a no resuelto. No ocurre, y eso indica que la configuración de inferencia empleada — imágenes de una en una, resolución fija y precisión sencilla — es determinista en esta plataforma.

Conviene acotar el alcance de la afirmación. El determinismo está comprobado en un equipo concreto, con una versión concreta de las bibliotecas y sobre tres repeticiones; no es una propiedad que la biblioteca de inferencia garantice. Cambiar de tarjeta gráfica, activar precisión reducida o volver a procesar las imágenes por lotes podría romperlo. La tercera de esas tres condiciones es precisamente la que provocó el fallo descrito en la Sección 5.3.2, lo que sugiere que la configuración conservadora adoptada allí por necesidad tiene un beneficio secundario que no se buscaba.

5.10. Síntesis

La Tabla 5.10 reúne los tres ensayos.

Tres conclusiones se sostienen sobre estos datos.

La precisión no es el problema. En los tres ensayos, el fotograma típico se sitúa entre 150 y 320 m de su posición verdadera, sin ningún filtro que acumule información y resolviendo cada fotograma desde cero. Añadir un filtro que enlace fotogramas consecutivos, que es lo que hacen todos los sistemas reales citados en la Sección 1.2.6, reduciría estas cifras todavía más.

La disponibilidad sí lo es, y tiene una causa identificada. El sistema deja de funcionar cuando el campo de visión contiene menos de tres cráteres del mapa, y eso ocurre a una altitud que depende únicamente de la densidad del mapa embarcado. No es un defecto del

Tabla 5.10: Resumen comparativo. El ensayo 3 recorre un intervalo de altitud mucho mayor, lo que explica por sí solo la diferencia de cobertura.

	Ensayo 1 diferido	Ensayo 2 tiempo real	Ensayo 3 descenso
Altitud recorrida (km)	170–120	170–120	170–55
Giro en guiñada	no	no	$\pm 180^\circ$
Detector en el bucle	no	sí	sí
Fotogramas resueltos	97,7 %	97,3 %	66,7 %
Error de posición, mediana	0,150 km	0,146 km	0,314 km
Emparejamientos falsos	0	0	5

algoritmo sino una propiedad estructural de cualquier método basado en constelaciones de landmarks, y la única forma de bajarla es densificar el mapa, pagando memoria.

La verificación es la línea de defensa que decide si un fallo es benigno o peligroso. Los cinco emparejamientos falsos de todo el trabajo aparecen exclusivamente en fotogramas con exactamente tres cráteres visibles, donde no queda ningún cráter libre con el que corroborar la hipótesis. Un sistema real debería tratar la imposibilidad de verificar como un fallo declarado, y no entregar una posición que el resto del sistema de guiado creería.

Capítulo 6

Conclusiones y trabajos futuros

6.1. Conclusiones

El trabajo partía de una pregunta concreta: si un vehículo que se aproxima a la Luna, sin señal de navegación externa y sin ninguna idea previa de dónde está, puede averiguarlo mirando los cráteres que tiene debajo y comparándolos con un mapa que lleva en memoria. La respuesta que arrojan los ensayos del Capítulo 5 es que sí, con una precisión del orden de la centena de metros, y que el problema no está en acertar sino en dejar de tener con qué acertar. Esta sección recorre los cinco objetivos específicos de la Sección 1.4 y cierra cada uno con la medida que lo respalda.

El entorno de validación

El primer objetivo era construir un entorno lunar sintético con verdad de referencia exacta. El resultado es un terreno de $840\text{ km} \times 630\text{ km}$ poblado por 41 702 cráteres generados con perfiles morfométricos realistas, sobre el que se renderizaron dos trayectorias completas de 700 fotogramas cada una. De cada fotograma se conocen la posición, la altitud y la orientación de la cámara con error nulo, no estimados sino impuestos por construcción.

La decisión de trabajar sobre un mundo sintético fue deliberada y tiene un coste que conviene no disimular: los resultados no demuestran que el sistema funcione sobre la Luna real. Lo que sí permite es algo que ningún conjunto de imágenes reales ofrece hoy, que es separar limpiamente el error del sistema del error de la referencia. Cuando este trabajo afirma que el error mediano de posición es de 150 m, esos 150 m son enteramente del sistema. La comprobación geométrica descrita en la Sección 4.4.4 cierra esa cadena: proyectando el catálogo hacia delante con los parámetros de cámara declarados se reproducen las etiquetas exportadas por Blender sin error de píxel, de modo que cualquier discrepancia posterior procede de la percepción o del emparejamiento y no de una inconsistencia del entorno.

El detector

El segundo objetivo era obtener un detector capaz de recuperar la práctica totalidad de los cráteres útiles y fijar su punto de operación con datos. Al umbral de confianza adoptado, 0,70, el detector recupera el 99,4 % de los cráteres del mapa embarcado presentes en la imagen y deja el 96,8 % de los fotogramas con al menos tres, que es la condición mínima para formar una tripleta. El barrido de umbrales mostró además algo que no era evidente de antemano: entre 0,05 y 0,70 esa capacidad no se degrada en absoluto, porque los cráteres que el mapa contiene son precisamente los grandes, que son los que el detector reconoce con más seguridad. Subir el umbral descarta más de la mitad de las cajas sin perder nada aprovechable, y de paso alivia el trabajo combinatorio que viene después.

De este objetivo salió también el episodio más instructivo del desarrollo. Al procesar las secuencias por lotes, el 41 % de los fotogramas quedaba sin ninguna detección, no por un defecto del modelo sino por un límite de tiempo interno de la rutina de filtrado de cajas

solapadas que se agota cuando la escena contiene millares de objetos. La biblioteca no lanza ninguna excepción: emite un aviso y devuelve tensores vacíos. Procesar las imágenes de una en una lleva ese porcentaje a cero. La lección se generaliza mal a un enunciado bonito y por eso merece enunciarse en crudo: durante días se estuvo evaluando un modelo correcto a través de una tubería rota, y ninguna métrica del modelo lo delataba.

El mapa y el motor de emparejamiento

El tercer objetivo era identificar los cráteres observados dentro del mapa sin ninguna estimación previa de la posición. La solución adoptada indexa el mapa por tripletas de cráteres descritas mediante cuatro razones adimensionales que no cambian al girar, trasladar o escalar la constelación, lo que convierte el problema de localización global en una búsqueda en una tabla. El mapa embarcado son 258 cráteres y ocupa unos pocos kilobytes; la tabla completa de tripletas tendría 2 829 056 entradas, que la poda por criterio de visibilidad reduce a 65 773, un 2,3 % del total.

La parte de este objetivo que más rendimiento ha dado no es la que genera hipótesis sino la que las rechaza. De las cuatro comprobaciones encadenadas que se aplican a cada candidata, la decisiva es la corroboración con un cuarto cráter independiente, ajeno a la triplete que produjo la hipótesis. En los tres ensayos completos, los únicos cinco emparejamientos erróneos que llegaron a la salida aparecieron exclusivamente en fotogramas con exactamente tres cráteres visibles, donde no queda ningún cráter libre con el que corroborar. La conclusión transversal del trabajo está justamente ahí, y es la que un sistema real debería heredar: un emparejador geométrico no falla emitiendo un error, falla emitiendo una respuesta plausible y equivocada. Un fotograma que no puede verificarse debe declararse no resuelto, aunque tenga una solución que encaja.

La navegación sobre el crucero

El cuarto objetivo era demostrar la navegación completa, primero en diferido y después en tiempo real con la red dentro del bucle. En diferido el sistema resuelve 684 de 700 fotogramas con un error mediano de 0,150 km; en tiempo real, sobre el vídeo comprimido y dibujando simultáneamente la trayectoria en pantalla, resuelve 681 con un error mediano de 0,146 km y sin una sola solución errónea. La diferencia entre ambos modos es de tres fotogramas de setecientos y de cuatro metros de error, lo que responde de forma bastante concluyente a la duda de partida sobre si la compresión de vídeo y la latencia de la inferencia iban a pasar factura.

Dos propiedades del resultado merecen destacarse por encima de las cifras. La primera es que cada fotograma se resuelve desde cero, sin arrastrar ninguna información del anterior, de modo que 150 m es el error de una medida aislada y no el de una trayectoria filtrada. La segunda es que el sistema es reproducible: tres ejecuciones consecutivas de los tres ensayos produjeron ficheros de resultados idénticos byte a byte, incluidos los que ejecutan la red neuronal en la tarjeta gráfica dentro del bucle. Eso no estaba garantizado de antemano y es la razón por la que las cifras de este trabajo pueden repetirse.

El descenso y el límite de operación

El quinto objetivo pedía llevar el sistema hasta el punto de fallo y explicar dónde está y por qué. Durante el descenso, con la cámara girando una vuelta completa en guiñada, el sistema resuelve el 66,7 % de los fotogramas con un error mediano de 0,314 km y un error de guiñada de una décima de grado. La orientación sale prácticamente gratis del mismo ajuste que da la posición, sin ninguna búsqueda angular, porque el problema está formulado como una semejanza en el plano complejo y el ángulo es sencillamente el argumento de un número.

El fallo, cuando llega, no es progresivo. Por encima de 130 km el sistema resuelve todos los fotogramas; el primero que falla está a 120,8 km y el último que acierta a 110,5 km. Por debajo no resuelve ninguno. La causa es geométrica y no algorítmica: el campo de visión encoge con el cuadrado de la altitud, y llega un momento en que contiene menos de tres cráteres del mapa. El sistema no pierde precisión, se queda sin material.

Repetir el descenso con mapas más densos confirma esa explicación y la cuantifica. Pasando de 258 a 440 y a 753 cráteres, el suelo de operación baja de 108,5 a 88,1 y a 57,6 km, casi a la mitad, mientras el error mediano solo empeora un 13 %. Lo que se paga no es precisión sino memoria y cómputo: la tabla de tripletas se multiplica por veinticinco. Ese compromiso entre altitud alcanzable y tamaño del mapa es, probablemente, el resultado más útil del trabajo para quien tenga que dimensionar un sistema parecido, y es lo que motiva la primera de las líneas futuras que siguen.

6.2. Trabajos futuros

6.2.1. Detección de elipses orientadas

El detector actual entrega cajas rectangulares alineadas con los ejes de la imagen. Esa representación lleva escondida una hipótesis: que el cráter se proyecta como una circunferencia, lo cual solo es cierto mirando al nadir. En cuanto la cámara se inclina, un cráter circular se proyecta como una elipse orientada, y una caja alineada con los ejes deja de describirlo; el efecto ya se observó de forma indirecta en este trabajo, cuando el detector reentrenado con aumento por rotación empezó a devolver cajas hasta un 45 % más anchas de lo debido, según se documenta en la Sección 5.3.4.

La línea natural es sustituir la caja por la cónica completa, mediante detección orientada o regresión directa de los cinco parámetros de la elipse. La ganancia no sería solo de exactitud en el tamaño. Disponer de la cónica abre la puerta a los invariantes proyectivos de Christian et al. (2021), que permiten emparejar cráteres bajo una vista oblicua arbitraria en lugar de exigir la semejanza plana que este trabajo asume. Eso convertiría el sistema en utilizable durante una aproximación rasante, que es justamente la fase en la que el método actual deja de servir.

6.2.2. Incorporación de un filtro de navegación

Este trabajo resuelve cada fotograma de forma independiente y deliberadamente prescinde de cualquier filtro, porque el objetivo era medir lo que la percepción aporta por sí sola, sin que un estimador enmascare sus fallos. Un sistema destinado a volar haría lo contrario. Un filtro de error de estado, como los que emplean las misiones revisadas en la Sección 1.2.6, combinaría estas medidas con las de una unidad inercial y aportaría dos cosas que hoy faltan: suavizado del error, que reduciría la dispersión observada en el percentil 95, y sobre todo continuidad por debajo del límite de altitud, propagando la última solución válida durante los segundos en que la cámara ya no ve suficientes cráteres. El límite dejaría de ser el punto en que el sistema se apaga para convertirse en el punto en que deja de recibir correcciones.

6.2.3. Mapa jerárquico por altitud

El barrido de la Sección 5.8 demuestra que densificar el mapa baja el suelo de operación, pero también que hacerlo de forma uniforme es caro: el número de tripletas crece con el cubo del número de cráteres, y llevar el mapa de 258 a 753 entradas multiplica la tabla por veinticinco. La observación que sugiere la alternativa es que esa densidad extra solo hace falta abajo. A 170 km el mapa reducido resuelve el 100 % de los fotogramas y los cráteres adicionales no aportan nada.

Un mapa organizado por niveles de detalle, en el que la altitud estimada en el fotograma anterior seleccione el nivel a consultar en el siguiente, daría la cobertura del mapa denso al precio del mapa reducido durante la mayor parte del vuelo. La estructura tiene además una propiedad atractiva para un sistema embarcado: los niveles de detalle fino pueden mantenerse en almacenamiento lento y cargarse solo cuando la altitud lo justifica.

6.2.4. Validación sobre imágenes lunares reales

La comprobación pendiente es el salto al mundo real, usando imágenes de misiones orbitales y un catálogo de cráteres publicado en lugar del sintético. Conviene abordarlo con una expectativa realista: el propio capítulo de introducción documenta que el rendimiento de los detectores de cráteres cae de forma acusada al cambiar de conjunto de datos, y este trabajo ha medido de primera mano ese mismo efecto en pequeño, al comprobar que un modelo afinado sobre fotografías reales rinde poco más de la mitad sobre los renders sintéticos que uno afinado sobre ellos, y menos de la cuarta parte si se mide con el criterio de solape más exigente.

El experimento tiene además un problema metodológico que no debe pasarse por alto y que es la contrapartida exacta de la ventaja descrita al principio de estas conclusiones: sobre imágenes reales la verdad de referencia deja de ser exacta. La posición del vehículo se conoce con la incertidumbre de la reconstrucción de la órbita y las posiciones de los cráteres con la del catálogo, de modo que el error medido sería la suma del error del sistema y el de la referencia, sin forma limpia de separarlos. Un diseño experimental honesto tendría que empezar por acotar esa segunda contribución antes de atribuir cualquier degradación al sistema.

Bibliografía

- Agencia Espacial Europea. Argonaut: Europe's lunar lander programme. https://www.esa.int/Science_Exploration/Human_and_Robotic_Exploration/Exploration/Argonaut_Europe_s_lunar_lander_programme, 2026. Consultado en agosto de 2026.
- Michael K. Barker, Erwan Mazarico, Gregory A. Neumann, Maria T. Zuber, Junichi Haruyama, and David E. Smith. A new lunar digital elevation model from the Lunar Orbiter Laser Altimeter and SELENE Terrain Camera. *Icarus*, 273:346–355, 2016. doi: 10.1016/j.icarus.2015.07.039. URL <https://doi.org/10.1016/j.icarus.2015.07.039>.
- John M. Carson, Edward A. Robertson, Nikolas Trawny, and Farzin Amzajerdian. Flight testing ALHAT precision landing technologies integrated onboard the Morpheus rocket vehicle. Technical report, NASA, 2014. URL <https://ntrs.nasa.gov/api/citations/20170009170/downloads/20170009170.pdf>.
- Allen Chen, Jordi Casoliva, James Montgomery, Frederick Serricchio, Seth Aaron, Nikolas Trawny, Richard Otero, Yang Cheng, Paul Brugarolas, Andrew E. Johnson, and Swati Mohan. Assessment of M2020 terrain relative landing accuracy: Flight performance vs predicts. Technical report, NASA Jet Propulsion Laboratory, 2021. URL <https://ntrs.nasa.gov/citations/20230007014>.
- Yang Cheng, Andrew E. Johnson, et al. Making an onboard reference map from MRO/CTX imagery for Mars 2020 lander vision system. *Earth and Space Science*, 8, 2021. doi: 10.1029/2020EA001560. URL <https://doi.org/10.1029/2020EA001560>.
- John A. Christian, Harm Derksen, and Ryan Watkins. Lunar crater identification in digital images. *The Journal of the Astronautical Sciences*, 68:1056–1144, 2021. doi: 10.1007/s40295-021-00287-8. URL <https://doi.org/10.1007/s40295-021-00287-8>.
- Wenbo Dong, Pravakar Roy, Cheng Peng, and Volkan Isler. Ellipse R-CNN: Learning to infer elliptical object from clustering and occlusion. *IEEE Transactions on Image Processing*, 30: 2193–2206, 2021. doi: 10.1109/TIP.2021.3050673. URL <https://arxiv.org/abs/2001.11584>.
- Chad Hanak, Timothy Crain, and Robert Bishop. Crater identification algorithm for the lost in low lunar orbit scenario. In *AAS/AIAA Space Flight Mechanics Meeting*, 2009. URL <https://ntrs.nasa.gov/api/citations/20090032647/downloads/20090032647.pdf>.
- James W. Head, Caleb I. Fassett, Seth J. Kadish, David E. Smith, Maria T. Zuber, Gregory A. Neumann, and Erwan Mazarico. Global distribution of large lunar craters: Implications for resurfacing and impactor populations. *Science*, 329(5998):1504–1507, 2010. doi: 10.1126/science.1195050. URL <https://doi.org/10.1126/science.1195050>.
- International Electrotechnical Commission. Multimedia systems and equipment — colour measurement and management — part 2-1: Colour management — default RGB colour space — sRGB. Technical Report IEC 61966-2-1:1999, IEC, Ginebra, 1999.

- Tetsuo Ishida, Seisuke Fukuda, Keisuke Kariya, Hiroyuki Kamata, Keiki Takadama, Hirotaka Kojima, Shujiro Sawai, and Shin-ichiro Sakai. Vision-based navigation and obstacle detection flight results in SLIM lunar landing. *Acta Astronautica*, 226:772–781, 2025. doi: 10.1016/j.actaastro.2024.11.002. URL <https://doi.org/10.1016/j.actaastro.2024.11.002>.
- JAXA. Outcome for the Smart Lander for Investigating Moon (SLIM)’s moon landing. https://global.jaxa.jp/press/2024/01/20240125-1_e.html, 2024. Nota de prensa del 25 de enero de 2024.
- Ming Jin and Wei Shao. Crater triangle matching algorithm based on fused geometric and regional features. *Aerospace*, 11(6):417, 2024. doi: 10.3390/aerospace11060417. URL <https://doi.org/10.3390/aerospace11060417>.
- Glenn Jocher, Ayush Chaurasia, and Jing Qiu. Ultralytics YOLOv8, 2023. URL <https://github.com/ultralytics/ultralytics>. Versión 8.0.0. Programa de ordenador.
- Andrew Johnson, Reg Willson, Yang Cheng, Jay Goguen, Chris Leger, Miguel SanMartin, and Larry Matthies. Design through operation of an image-based velocity estimation system for Mars landing. *International Journal of Computer Vision*, 74(3):319–341, 2007. doi: 10.1007/s11263-006-0022-z. URL <https://doi.org/10.1007/s11263-006-0022-z>.
- Andrew E. Johnson and Tonislav I. Ivanov. Analysis and testing of a LIDAR-based approach to terrain relative navigation for precise lunar landing. In *AIAA Guidance, Navigation, and Control Conference*, 2010. URL https://www-robotics.jpl.nasa.gov/media/documents/FT3_LidarTRN_Paper_v12.pdf.
- Andrew E. Johnson, Yang Cheng, Nikolas Trawny, James F. Montgomery, Stephen Schroeder, Johnny Chang, David Clouse, Seth Aaron, and Swati Mohan. Implementation of a map relative localization system for planetary landing. *Journal of Guidance, Control, and Dynamics*, 46(4):618–637, 2023. doi: 10.2514/1.G006780. URL <https://doi.org/10.2514/1.G006780>.
- David G. Lowe. Distinctive image features from scale-invariant keypoints. *International Journal of Computer Vision*, 60(2):91–110, 2004. doi: 10.1023/B:VISI.0000029664.99615.94. URL <https://doi.org/10.1023/B:VISI.0000029664.99615.94>.
- Paul G. Lucey, Gregory A. Neumann, Miriam A. Riner, Erwan Mazarico, David E. Smith, Maria T. Zuber, David A. Paige, Donald B. Bussey, Joshua T. Cahill, Andrew McGovern, Peter Isaacson, Laura M. Corley, Mark H. Torrence, H. Jay Melosh, James W. Head, and Eugenie Song. The global albedo of the Moon at 1064 nm from LOLA. *Journal of Geophysical Research: Planets*, 119(7):1665–1679, 2014. doi: 10.1002/2013JE004592.
- Lunar Reconnaissance Orbiter Camera. First look: Apollo 12 and Surveyor 3. <https://lroc.im-ldi.com/images/141>, 2009. Arizona State University.
- H. Jay Melosh. *Impact Cratering: A Geologic Process*. Number 11 in Oxford Monographs on Geology and Geophysics. Oxford University Press, New York, 1989.
- Gerhard Neukum, Boris A. Ivanov, and William K. Hartmann. Cratering records in the inner solar system in relation to the lunar reference system. *Space Science Reviews*, 96, 2001. doi: 10.1023/A:1011989004263.
- Coralie D. Norman, Christopher J. Miller, Ryan D. Olds, Courtney E. Mario, Eric E. Palmer, Olivier S. Barnouin, Michael G. Daly, John R. Weirich, Jeffrey A. Seabrook, Carina A. Bennett, Bashar Rizk, Brent J. Bos, and Dante S. Lauretta. Autonomous navigation

- performance using natural feature tracking during the OSIRIS-REx touch-and-go sample collection event. *The Planetary Science Journal*, 3(5):101, 2022. doi: 10.3847/PSJ/ac5183. URL <https://doi.org/10.3847/PSJ/ac5183>.
- Richard J. Pike. Size-dependence in the shape of fresh impact craters on the Moon. In D. J. Roddy, R. O. Pepin, and R. B. Merrill, editors, *Impact and Explosion Cratering*, pages 489–509. Pergamon Press, New York, 1977.
- Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 779–788, 2016. doi: 10.1109/CVPR.2016.91.
- Stuart J. Robbins. A new global database of lunar impact craters >1–2 km: 1. crater locations and sizes, comparisons with published databases, and global analysis. *Journal of Geophysical Research: Planets*, 124(4):871–892, 2019. doi: 10.1029/2018JE005592. URL <https://doi.org/10.1029/2018JE005592>.
- Matthew Rodda, Sofia McLeod, Ky Cuong Pham, and Tat-Jun Chin. Camera-pose robust crater detection from Chang’e 5. arXiv:2406.04569, 2024. URL <https://arxiv.org/abs/2406.04569>.
- Michael K. Shepard, Bruce A. Campbell, Mark H. Bulmer, Tom G. Farr, Lisa R. Gaddis, and Jeffrey J. Plaut. The roughness of natural terrain: A planetary and remote sensing perspective. *Journal of Geophysical Research: Planets*, 106(E12):32777–32795, 2001. doi: 10.1029/2000JE001429.
- Ari Silburt, Mohamad Ali-Dib, Chenchong Zhu, Alan Jackson, Diana Valencia, Yevgeni Kissin, Daniel Tamayo, and Kristen Menou. Lunar crater identification via deep learning. *Icarus*, 317:27–38, 2019. doi: 10.1016/j.icarus.2018.06.022. URL <https://doi.org/10.1016/j.icarus.2018.06.022>.
- Y. Tang et al. SCCA-YOLO: Spatial channel fusion and context-aware attention for lunar crater detection. *Remote Sensing*, 2025. URL <https://arxiv.org/abs/2506.21970>.
- Jason Yosinski, Jeff Clune, Yoshua Bengio, and Hod Lipson. How transferable are features in deep neural networks? In *Advances in Neural Information Processing Systems 27 (NIPS 2014)*, 2014. arXiv:1411.1792.

Anexos

Anexo A

Repositorio de código

Todo el código desarrollado en este trabajo está disponible de forma pública en

<https://github.com/sergiogonzalezgil21/tfg-navegacion-crateres>

Se ha optado por remitir al repositorio en lugar de reproducir los listados en la memoria por dos razones. La primera es de extensión: el trabajo consta de una cuarentena de ficheros repartidos en tres entornos, y volcarlos aquí ocuparía varios centenares de páginas que nadie leería de corrido. La segunda es de fidelidad: un listado impreso queda congelado en el momento de la entrega, mientras que el repositorio conserva el historial completo y permite descargar, ejecutar y comprobar el código en lugar de solo leerlo.

El Anexo D indica en qué orden hay que ejecutar los ficheros para reconstruir los resultados de la memoria.

A.1. Organización del repositorio

El repositorio reproduce la estructura de carpetas con la que se ha trabajado. Cada carpeta corresponde a uno de los tres entornos del sistema.

<code>blender/</code>	Generación del terreno, definición de las trayectorias, renderizado de las secuencias y exportación de etiquetas y verdad de referencia
<code>ia/</code>	Preparación del conjunto de datos, entrenamiento y evaluación del detector, y exportación de las detecciones
<code>matlab/</code>	Motor de navegación, los tres ensayos y la generación de figuras
<code>etapa1/</code>	Los cuatro scripts del primer entrenamiento del detector, realizado en una fase temprana de este mismo trabajo

A.2. Índice de ficheros

Generación del entorno (Python y Blender)

<code>terreno_lunar.py</code>	Genera el mapa de alturas, el mapa de albedo y el catálogo de cráteres con ruido fractal y los perfiles morfométricos de Pike. Solo usa NUMPY: no depende de Blender
<code>blender_construir_terreno.py</code>	Monta la escena: malla, material, iluminación y las dos cámaras
<code>blender_preparar_vuelo.py</code>	Renderiza el mapa de referencia y define la trayectoria de crucero
<code>blender_preparar_vuelo_f4.py</code>	Ídem para el descenso
<code>blender_camara_descenso.py</code>	Añade el perfil de guiñada al descenso
<code>blender_exportar_etiquetas.py</code>	Proyecta el catálogo sobre cada fotograma y exporta las etiquetas en formato YOLO, el mapa embarcado y la trayectoria real
<code>blender_exportar_descenso.py</code>	Ídem para la secuencia de descenso
<code>blender_fig_trayectoria3d.py</code>	Genera la vista tridimensional de la Figura 5.3

Primer entrenamiento del detector

Corresponden a la etapa 1 de la Tabla 3.1 y se realizaron en una fase temprana de este mismo trabajo.

<code>descargar_datasets.py</code>	Descarga de Kaggle los conjuntos públicos de cráteres
<code>preparar_dataset.py</code>	Combina dos de ellos en un único conjunto de una sola clase y lo reparte
<code>ampliar_dataset.py</code>	Añade el tercer conjunto, LU3M6TGT, ya en formato YOLO
<code>entrenar_yolo.py</code>	Entrenamiento de la etapa 1 sobre el conjunto combinado

Detector (Python)

<code>01.verificar.etiquetas.py</code>	Comprueba la coherencia de las etiquetas exportadas de Blender antes de entrenar con ellas
<code>02.preparar.dataset.py</code>	Reparte los fotogramas en entrenamiento y validación con el hueco geométrico de la Sección 3.5.1
<code>03.entrenar.py</code>	Ajuste fino sobre los renderizados: la etapa 2
<code>04.evaluar.py</code>	Métricas de detección sobre el conjunto de validación
<code>05.metrica.navegacion.py</code>	Mide lo que le importa al navegador: cráteres del mapa recuperados y fotogramas con al menos tres
<code>06.diagnostico.deteccion.py</code>	Diagnóstico del fallo descrito en la Sección 3.7
<code>07.deteccion.limpia.py</code>	Barrido del umbral de confianza sobre los cráteres del mapa
<code>08.exportar.detecciones.py</code>	Vuelca las detecciones del crucero a <code>detecciones.csv</code>
<code>09.barrido.conjunto.py</code>	Barrido conjunto del umbral de mapa y del número K de detecciones retenidas
<code>10.robustez.rotacion.py</code>	Mide la cobertura frente al giro de la imagen
<code>11.detecciones.descenso.py</code>	Vuelca las detecciones del descenso
<code>12.reentrenar.rotacion.py</code>	Reentrenamiento con aumento por rotación: la etapa 3

Motor de navegación (MATLAB)

Los cuatro primeros constituyen el motor propiamente dicho; el resto son los ensayos y las utilidades.

<code>cargar_mapa.m</code>	Motor: lee el mapa embarcado y construye la tabla de tripletas con su descriptor y su poda
<code>buscar_tripletas.m</code>	Motor: busca en la tabla por descriptor, con tolerancias, mediante búsqueda binaria
<code>leer_detecciones.m</code>	Motor: carga las detecciones y marca las cajas que tocan el borde del fotograma
<code>resolver_frame.m</code>	Motor: resuelve un fotograma completo, de detecciones a vector de estados, incluida la verificación
<code>fase3a_offline.m</code>	Ensayo 1, navegación en diferido
<code>fase3b_directo.m</code>	Ensayo 2, navegación en tiempo real
<code>fase4_directo.m</code>	Ensayo 3, descenso con rotación
<code>fase4_cascada.m</code>	Barrido de densidad del mapa
<code>detector_yolo.py</code>	Puente con MATLAB: mantiene el modelo cargado y devuelve las detecciones del fotograma solicitado
<code>guardar_pasada.m</code>	Archiva los ficheros de resultados de cada repetición, para el ensayo de repetibilidad
<code>memoria_fig*.m</code>	Generación de las figuras de la memoria
<code>fase3b_crear_video.m, fase4_crear_video.m</code>	Componen el vídeo de entrada a partir de los fotogramas renderizados

A.3. El puente entre MATLAB y Python

Merece una nota aparte porque es la pieza que hace posible el ensayo en tiempo real y no es evidente cómo se monta. MATLAB dirige el bucle y llama a Python a través de

la interfaz `py.*`, con el intérprete lanzado como proceso independiente (`ExecutionMode = OutOfProcess`) para que un fallo de la biblioteca de inferencia no arrastre consigo a MATLAB. El módulo de Python mantiene el modelo cargado entre llamadas, de modo que el coste de inicialización se paga una sola vez, y devuelve las detecciones del fotograma solicitado junto con su número, lo que permite al bucle comprobar que recibe el fotograma que pidió y no otro.

Anexo B

Tablas de resultados completas

El Capítulo 5 presenta, por legibilidad, extractos de los barridos realizados. Aquí se recogen completos.

B.1. Barrido conjunto de umbral de mapa y número de detecciones

Cada fila es una combinación del diámetro mínimo de los cráteres que entran en el mapa y del número K de detecciones más grandes que se retienen por fotograma. La precisión es la fracción de detecciones retenidas que corresponden realmente a un cráter del mapa; la exhaustividad, la fracción de cráteres del mapa presentes en la imagen que quedan retenidos. Las dos últimas columnas son el porcentaje de fotogramas con al menos tres y al menos cuatro cráteres del mapa retenidos.

$D_{\min}$	K	Cráteres	Tripletas	Precisión	Exhaust.	≥ 3 (%)	≥ 4 (%)
8	4	322	5 512 640	0,898	0,550	93,0	66,2
8	5	322	5 512 640	0,871	0,667	100,0	86,0
8	6	322	5 512 640	0,863	0,793	100,0	99,4
8	7	322	5 512 640	0,836	0,896	100,0	100,0
8	8	322	5 512 640	0,785	0,958	100,0	100,0
8	10	322	5 512 640	0,678	0,981	100,0	100,0
10	4	258	2 829 056	0,898	0,605	93,0	66,2
10	5	258	2 829 056	0,869	0,732	100,0	85,4
10	6	258	2 829 056	0,855	0,864	100,0	98,1
10	7	258	2 829 056	0,812	0,957	100,0	98,7
10	8	258	2 829 056	0,736	0,988	100,0	98,7
10	10	258	2 829 056	0,624	0,994	100,0	98,7
12	4	235	2 135 445	0,896	0,641	93,0	65,6
12	5	235	2 135 445	0,866	0,774	100,0	84,1
12	6	235	2 135 445	0,841	0,902	100,0	96,8
12	7	235	2 135 445	0,780	0,975	100,0	97,5
12	8	235	2 135 445	0,697	0,993	100,0	97,5
12	10	235	2 135 445	0,589	0,995	100,0	97,5
14	4	216	1 656 360	0,893	0,658	93,0	64,3
14	5	216	1 656 360	0,864	0,795	100,0	82,8
14	6	216	1 656 360	0,827	0,913	100,0	95,5
14	7	216	1 656 360	0,760	0,979	100,0	96,2
14	8	216	1 656 360	0,678	0,994	100,0	96,2
14	10	216	1 656 360	0,573	0,996	100,0	96,2
16	4	211	1 543 465	0,874	0,807	87,9	61,8
16	5	211	1 543 465	0,797	0,921	94,3	77,7
16	6	211	1 543 465	0,710	0,984	94,3	88,5
16	7	211	1 543 465	0,618	0,999	94,3	89,2
16	8	211	1 543 465	0,544	1,000	94,3	89,2
16	10	211	1 543 465	0,459	1,000	94,3	89,2

La combinación adoptada es $D_{\min} = 10$ km y $K = 7$.

B.2. Robustez del detector frente al giro de la imagen

Las mismas imágenes de validación, giradas. *Recall* es la fracción de cráteres del mapa recuperados; *ancho* es la razón entre el ancho de la caja devuelta y el de la caja verdadera, que es la medida del inflado.

Modelo	Giro (°)	Recall	≥ 3 (%)	≥ 4 (%)	Ancho
crucero	0	1,000	43,4	5,7	1,014
crucero	15	0,951	39,6	0,0	1,010
crucero	30	0,748	13,2	0,0	1,015
crucero	45	0,276	0,0	0,0	1,019
crucero	60	0,081	0,0	0,0	1,000
crucero	90	0,187	0,0	0,0	1,007
crucero	135	0,024	0,0	0,0	1,065
crucero	180	0,000	0,0	0,0	—
reentrenado	0	1,000	43,4	5,7	1,158
reentrenado	15	0,951	34,0	5,7	1,279
reentrenado	30	0,854	22,6	1,9	1,380
reentrenado	45	0,829	22,6	1,9	1,427
reentrenado	60	0,846	26,4	0,0	1,382
reentrenado	90	0,967	37,7	3,8	1,102
reentrenado	135	0,780	15,1	0,0	1,449
reentrenado	180	0,976	39,6	5,7	1,145

B.3. Barrido del umbral de confianza

Medido sobre los cráteres del mapa embarcado, con los tres modelos ensayados. *Cajas* es el número total de detecciones por fotograma, *vacíos* el porcentaje de fotogramas sin ninguna detección, y *s/frame* el tiempo de inferencia.

Modelo	Conf.	Cajas	Recall	≥ 3 (%)	Vacíos (%)	s/frame
reales	0,05	131,9	0,871	99,4	0,0	0,136
reales	0,25	69,7	0,701	84,1	0,0	0,091
reales	0,5	41,0	0,463	36,3	1,3	0,071
reales	0,7	20,4	0,265	5,7	5,7	0,066
partida	0,05	12,6	0,769	99,4	0,0	0,052
partida	0,25	5,0	0,540	80,3	0,0	0,045
partida	0,5	2,5	0,337	10,8	2,5	0,043
partida	0,7	1,3	0,182	0,0	31,2	0,042
Blender	0,05	583,4	0,994	98,7	0,0	0,439
Blender	0,25	393,9	0,994	98,1	0,0	0,306
Blender	0,5	315,7	0,994	97,5	0,0	0,260
Blender	0,7	251,1	0,994	96,8	0,0	0,273

B.4. Cobertura por franja de altitud

Porcentaje de fotogramas resueltos en cada franja de 10 km. Cada ensayo se agrupa por su *propia* altitud, de modo que los ensayos 1 y 2, que recorren de 170 a 120 km, solo ocupan las cinco primeras franjas.

Franja (km)	<i>n</i>	Ensayo 1	Ensayo 2
170–160	200	100,0	100,0
160–150	102	100,0	100,0
150–140	94	100,0	100,0
140–130	102	97,1	97,1
130–120	201	93,5	92,0

El descenso recorre un intervalo mucho mayor. La tabla siguiente recoge el ensayo 3 y los tres mapas del barrido de densidad, todos ellos sobre la misma secuencia y con las mismas detecciones: lo único que cambia entre las tres últimas columnas es el mapa embarcado.

Franja (km)	n	Ensayo 3	$D \geq 10$	$D \geq 6$	$D \geq 4$
170–160	206	100,0	100,0	100,0	100,0
160–150	85	100,0	100,0	100,0	100,0
150–140	66	100,0	100,0	100,0	100,0
140–130	55	100,0	92,7	92,7	92,7
130–120	48	91,7	91,7	100,0	100,0
120–110	44	22,7	25,0	27,3	38,6
110–100	41	0,0	9,8	22,0	26,8
100–90	38	0,0	0,0	28,9	86,8
90–80	35	0,0	0,0	2,9	91,4
80–70	33	0,0	0,0	0,0	33,3
70–60	32	0,0	0,0	0,0	0,0

Las columnas del ensayo 3 y de $D \geq 10$ usan el mismo mapa embarcado y difieren ligeramente porque no comparten la fuente de detecciones: el ensayo 3 ejecuta el detector dentro del bucle, mientras que el barrido de densidad parte de las detecciones exportadas previamente a `detecciones_descenso.csv`, de modo que los tres mapas se comparen sobre exactamente la misma entrada. El resto del capítulo emplea la columna del ensayo 3.

Anexo C

Parámetros del sistema

Todas las constantes que gobiernan el comportamiento del sistema, con el valor adoptado, el fichero donde se fija y la razón de la elección. Salvo indicación en contra, el valor es el mismo en los tres ensayos.

C.1. Cámara y escena

Parámetro	Valor	Razón
Resolución	1920×1080	Resolución del renderizado. Con los 640×480 iniciales los cráteres pequeños no llegaban al detector
Campo de visión horizontal	60°	Compromiso entre huella cubierta y resolución sobre el suelo
Focal equivalente	1663 px	Derivada: $W/(2 \tan(\alpha/2))$
Unidad de escena	1 u = 1 km	Evita los artefactos de precisión del <i>z-buffer</i> con distancias de cientos de kilómetros
Terreno	840×630 km	Suficiente para 700 fotogramas de crucero sin salirse

C.2. Terreno y catálogo

Parámetro	Valor	Razón
Semilla del generador	20260818	Fija todo el terreno. Con la misma semilla se obtiene el mismo mapa de alturas, el mismo catálogo y las mismas etiquetas
Resolución del mapa de alturas	8192 px	102 m por píxel, que es la resolución que ve la cámara a la altitud de crucero
Malla del terreno	3000×2250	Por debajo del muestreo del mapa de alturas; más vértices no añaden detalle
Cráteres en el catálogo	41 702	De los 580 000 sorteados, los que superan 0,8 km de diámetro. Densidad de 0,079 por km ²
Cuencas de mar	12	Escaladas con el área del terreno
Amplitud del relieve base	2200 m	7 octavas de ruido fractal
Frescura mínima para el mapa	0,30	Por debajo, el cráter no proyecta sombra suficiente y el detector no lo encuentra de forma fiable
Diámetro mínimo del mapa	10 km	Del barrido del Anexo B: es donde la exhaustividad se mantiene sin disparar el número de tripletas

C.3. Detector

Parámetro	Valor	Razón
<i>Etapa 1: conjuntos públicos</i>		
Épocas	50	
Resolución de entrenamiento	640 px	
Tamaño de lote	8	
<i>Etapas 2 y 3: renderizados</i>		
Traslación / escala	0,10 / 0,40	Aumento de datos; la escala simula variación de altura
Variación de brillo	0,25	Las imágenes son monocromas, así que el tono y la saturación van a cero
Mosaico	1,0	Se desactiva en las 10 últimas épocas
<i>Inferencia</i>		
Umbral de confianza	0,70	Del barrido: la recuperación de los cráteres del mapa no se degrada entre 0,05 y 0,70, y el umbral alto reduce a la mitad las cajas que pasan al emparejador
Resolución de inferencia	1024 px	Compromiso entre detalle y tiempo
Tamaño de lote	1	<i>Obligatorio:</i> con lotes mayores se agota el límite de tiempo interno del filtrado de cajas solapadas y se pierden fotogramas enteros sin aviso
Tamaño de lote (entrenamiento)	4	Limitado por la memoria de la tarjeta
Épocas	60 (tope)	Parada temprana en la 43 con paciencia de 20
Aumento por rotación	0° / ±180°	Sin rotación en crucero; con ella en descenso. Son dos modelos distintos

C.4. Emparejamiento

Parámetro	Valor	Razón
Detecciones retenidas K	7	Codo del barrido: exhaustividad 0,957 con precisión 0,812. Subir a 10 apenas gana exhaustividad y hunde la precisión
Lado máximo de tripleta	200 km	Huella de la cámara a la altitud máxima. Poda la tabla de 2829 056 a 65 773 entradas
Tolerancia de forma	0,02	Sobre los cocientes de lados, adimensionales
Tolerancia de diámetro	0,15	Sobre los cocientes de diámetros, más holgada porque el diámetro estimado es más ruidoso que la posición

C.5. Verificación y filtros físicos

Parámetro	Valor	Razón
Rango de altitud (crucero)	50–400 km	Rango plausible conocido a priori. Junto con el tamaño, es lo que fija la escala: el descriptor no la fija
Rango de altitud (descenso)	40–200 km	Ajustado al perfil de descenso
Tolerancia de verificación	55 px	Distancia máxima entre una detección y el cráter del mapa que se le asigna
Tolerancia de diámetro	30 %	Margen sobre el diámetro del catálogo
Corroboraciones mínimas	4	Tres las aporta la propia tripleta: exigir cuatro obliga a que haya al menos un cráter <i>independiente</i> que confirme
Agrupamiento de hipótesis	5 km	Radio para elegir entre hipótesis empatadas en corroboración
Inflado admisible	0,85–1,70	Solo en descenso. El factor medido va de 1,10 a 1,45; fuera de este rango la hipótesis es imposible

C.6. Continuidad entre fotogramas

El sistema no lleva filtro de navegación: la pose que entrega es la solución geométrica del propio fotograma, sin suavizar. Sí emplea, en cambio, la última posición resuelta como

ventana de búsqueda y de rechazo. Con las dos últimas soluciones extrapola linealmente la posición esperada, descarta las hipótesis que caen fuera del radio, y si ninguna sobrevive repite la búsqueda contra el catálogo completo. En el primer fotograma, y tras una racha de fallos lo bastante larga como para que el radio alcance su tope, el sistema resuelve sin ninguna información previa.

Parámetro	Valor	Razón
Salto máximo	45 km	Desplazamiento máximo plausible entre fotogramas consecutivos
Margen por hueco	12 km	Cuánto crece el radio de búsqueda por cada fotograma no resuelto
Radio máximo	400 km	Por encima, la restricción deja de aportar y se resuelve globalmente

Anexo D

Guía de reproducción

Este anexo recoge lo necesario para reconstruir los resultados desde cero, partiendo del código del Anexo A. El sistema es determinista: repetido con las mismas entradas produce ficheros de resultados idénticos byte a byte, según se comprueba en la Sección 5.9.

Dos valores que hay que fijar antes de empezar. La semilla del generador de terreno debe ser 20260818, y el diámetro mínimo con el que se construye el mapa embarcado, `D_MIN_MAPA_KM` en `blender_exportar_etiquetas.py`, debe valer 10. Con otra semilla el terreno es distinto; con otro umbral el mapa deja de tener 258 cráteres y ninguna de las cifras de la memoria se reproduce.

D.1. Entorno

Blender	Generación del terreno y renderizado de las secuencias
Python	Detector y utilidades de evaluación; requiere la biblioteca de inferencia, PyTorch con soporte para tarjeta gráfica, NumPy y OpenCV
MATLAB	Motor de navegación y representación gráfica. El ensayo en tiempo real requiere además que MATLAB pueda lanzar el intérprete de Python

<code>blender/</code>	Scripts de generación, mapa de alturas, catálogo, trayectorias reales y secuencias renderizadas
<code>ia/</code>	Conjunto de datos, scripts de entrenamiento y evaluación, pesos de los modelos y detecciones exportadas
<code>matlab/</code>	Motor de navegación, ensayos y ficheros de resultados
<code>latex/</code>	Memoria

D.2. Orden de ejecución

Cada paso consume lo que produce el anterior. Los pasos 1 a 3 solo hay que repetirlos si se cambia el terreno o el detector.

1. Generar el mundo. Ejecutar `blender_construir_terreno.py` dentro de Blender. Produce el mapa de alturas, las texturas, el catálogo de cráteres y la malla del terreno. Es el paso más lento, del orden de varios minutos, y la mayor parte se va en generar el mapa de alturas.

2. Preparar las trayectorias y renderizar. `blender_preparar_vuelo.py` para el crucero y `blender_preparar_vuelo.f4.py` seguido de `blender_camara_descenso.py` para el descenso, que es el que añade el perfil de guiñada. A continuación `blender_exportar_etiquetas.py` y `blender_exportar_descenso.py`, que proyectan el catálogo sobre cada fotograma y producen las etiquetas en formato YOLO, la trayectoria real y el mapa embarcado en kilómetros.

Comprobación. El propio exportador contrasta su proyección vectorizada contra la función de Blender en unos cuantos puntos de cada secuencia y muestra la desviación máxima, y avisa si supera 10^{-4} en coordenadas normalizadas. Si salta ese aviso, la matriz de cámara o el campo de visión están mal y todo lo que venga después heredará el error sin dar ningún aviso.

3. Entrenar el detector. `01_verificar_etiquetas.py` comprueba que las etiquetas exportadas son coherentes con los fotogramas, `02_preparar_dataset.py` reparte las imágenes dejando el hueco geométrico entre entrenamiento y validación, y `03_entrenar.py` hace el ajuste fino. Para el detector de descenso, `12_reentrenar_rotacion.py` parte de los pesos del anterior y añade el aumento por rotación.

4. Exportar detecciones. `08_exportar_detecciones.py` vuelca las detecciones a `detecciones.csv`, que es lo que consume el ensayo en diferido. **El tamaño de lote debe ser 1;** con lotes mayores se pierden fotogramas enteros de forma silenciosa.

5. Ensayos. Desde MATLAB, con la carpeta `matlab/` como directorio de trabajo:

<code>fase3a_offline</code>	Ensayo 1. No necesita Python
<code>fase3b_directo</code>	Ensayo 2. Lanza el intérprete de Python. No cerrar la ventana hasta que termine: el script guarda solo los fotogramas procesados
<code>fase4_directo</code>	Ensayo 3, descenso con rotación
<code>fase4_cascada</code>	Barrido de densidad del mapa, los tres a la vez

6. Figuras. `memoria_fig_resultados` genera las cuatro figuras del capítulo de resultados a partir de los ficheros de salida, y `memoria_fig_perfiles` y `memoria_fig_mapa` las del Capítulo 2 y del Capítulo 4. La vista tridimensional de la trayectoria se genera aparte, con `blender_fig_trayectoria3d.py` dentro de Blender.

D.3. Salidas esperadas

Fichero	Filas	Contenido
<code>fase3a_resultados.csv</code>	700	Pose estimada, corroborados, hipótesis y error por fotograma
<code>fase3b_resultados.csv</code>	700	Ídem, sin la columna de hipótesis
<code>fase4_directo_resultados.csv</code>	700	Ídem, con altitud y guiñada reales
<code>fase4_cascada.csv</code>	700	Validez, error, altitud y guiñada para cada uno de los tres mapas

D.4. Comprobación de que la reproducción es correcta

Los cuatro ficheros deben coincidir byte a byte con los de referencia. Si se quiere comprobar sin compararlos enteros, basta con verificar tres cifras: el ensayo 1 resuelve 684 de 700 fotogramas con error mediano de 0,150 km; el ensayo 2, 681 con 0,146 km; y el ensayo 3, 467 con 0,314 km sobre los correctos.

Si los números no salen. Un resultado notablemente peor con el mismo código apunta casi siempre al tamaño de lote de la inferencia, que es el fallo descrito en la Sección 3.7: el síntoma es un porcentaje alto de fotogramas sin ninguna detección, no un error grande. Conviene comprobarlo antes de buscar el problema en el emparejador.

Anexo E

Deducciones

E.1. El ajuste de la transformación de semejanza

En el Sección 4.3.2 se enuncian las expresiones de los coeficientes de la semejanza sin desarrollarlas, porque el desarrollo rompería el hilo del capítulo. Se recoge aquí.

Planteamiento

Sea un conjunto de $n \geq 3$ correspondencias entre puntos del mapa s_i y detecciones en la imagen d_i , ambos escritos como números complejos igual que en el Sección 4.3.2: $s_i = x_i + j y_i$ en kilómetros y $d_i = u_i + j v_i$ en píxeles. Se busca la semejanza directa

$$d = a s + b, \quad a, b \in \mathbb{C} \quad (\text{E.1})$$

que mejor los relaciona. El módulo $|a|$ es el factor de escala, su argumento $\arg(a)$ el ángulo de giro, y b la traslación.

Nótese que Ecuación (E.1) es **lineal** en a y en b . Ésa es toda la ventaja de la formulación compleja: escrita con una matriz de rotación y un factor de escala por separado, la ortogonalidad de la matriz introduce una restricción no lineal.

Se minimiza el error cuadrático

$$J(a, b) = \sum_{i=1}^n |a s_i + b - d_i|^2 \quad (\text{E.2})$$

Eliminación de la traslación

Derivando Ecuación (E.2) respecto de $\bar{b}$ e igualando a cero:

$$\frac{\partial J}{\partial \bar{b}} = \sum_i (a s_i + b - d_i) = 0 \implies b = m_d - a m_s$$

con $m_s = \frac{1}{n} \sum_i s_i$ y $m_d = \frac{1}{n} \sum_i d_i$ los centroides.

Es decir: **la semejanza óptima lleva el centroide de la tripleta del mapa al centroide de la tripleta detectada**, sea cual sea a . Sustituyendo, el problema queda en las coordenadas centradas $\sigma_i = s_i - m_s$ y $\delta_i = d_i - m_d$:

$$J(a) = \sum_i |a \sigma_i - \delta_i|^2$$

Solución para la escala y el giro

Desarrollando el módulo al cuadrado como producto por el conjugado,

$$J(a) = \sum_i (a\sigma_i - \delta_i) \overline{(a\sigma_i - \delta_i)} = |a|^2 \sum_i |\sigma_i|^2 - a \sum_i \sigma_i \bar{\delta}_i - \bar{a} \sum_i \bar{\sigma}_i \delta_i + \sum_i |\delta_i|^2$$

De los cuatro términos, solo el primero y el tercero dependen de $\bar{a}$; el segundo depende de a y el cuarto de ninguno de los dos. Derivando respecto de $\bar{a}$:

$$\frac{\partial J}{\partial \bar{a}} = a \sum_i |\sigma_i|^2 - \sum_i \bar{\sigma}_i \delta_i = 0$$

de donde

$$a = \frac{\sum_i \overline{(s_i - m_s)} (d_i - m_d)}{\sum_i |s_i - m_s|^2} \quad b = m_d - a m_s \quad (\text{E.3})$$

Observaciones

El denominador es real y positivo salvo que todos los puntos observados coincidan, caso que el código descarta explícitamente. La solución es **cerrada**: no hay iteración ni valor inicial, lo que explica que el coste por hipótesis sea tan bajo y que el resultado sea reproducible bit a bit.

Con exactamente tres correspondencias el ajuste es exacto y el residuo es nulo, de modo que *el residuo no sirve para decidir si la hipótesis es buena*. Ésa es la razón de fondo de que la verificación tenga que apoyarse en cráteres que no participaron en el ajuste, y no en el error de ajuste.

E.2. El inflado de la caja envolvente bajo rotación

En las Sección 3.9 y Sección 5.3.4 se afirma que la caja alineada con los ejes que envuelve a un objeto crece con $|\cos \theta| + |\sin \theta|$ al girarlo. Se demuestra aquí, porque es la explicación del sesgo de tamaño del detector reentrenado.

Enunciado

Sea un rectángulo de anchura w y altura h centrado en el origen, girado un ángulo θ . Su caja envolvente alineada con los ejes tiene anchura

$$W(\theta) = w |\cos \theta| + h |\sin \theta| \quad (\text{E.4})$$

Demostración

Los cuatro vértices del rectángulo sin girar son $(\pm w/2, \pm h/2)$. Al girar θ , la abscisa de cada uno pasa a ser

$$x' = \pm \frac{w}{2} \cos \theta \mp \frac{h}{2} \sin \theta$$

La anchura de la caja envolvente es la diferencia entre la mayor y la menor de esas cuatro abscisas. El máximo se alcanza eligiendo los signos que hacen que los dos términos sumen en valor absoluto, es decir $x'_{\text{máx}} = \frac{w}{2} |\cos \theta| + \frac{h}{2} |\sin \theta|$, y por simetría $x'_{\text{mín}} = -x'_{\text{máx}}$. Restando se obtiene Ecuación (E.4).

Consecuencia para un cráter

Un cráter se proyecta aproximadamente como un cuadrado envolvente, $w = h = D$, y entonces

$$\frac{W(\theta)}{D} = |\cos \theta| + |\sin \theta|$$

que vale 1 a 0° y 90° , y alcanza su máximo $\sqrt{2} \approx 1,414$ a 45° .

Contraste con lo medido. El detector reentrenado devuelve cajas con una razón de anchura de 1,158 a 0° y 1,427 a 45° . La forma de la curva sigue Ecuación (E.4) con correlación 0,99, lo que confirma el mecanismo: el aumento por rotación presenta a la red las cajas envolventes de las imágenes giradas, y la red aprende a reproducir ese inflado leyendo la dirección de la sombra, que es lo que le indica el ángulo.

El valor de 1,158 a 0° , donde la fórmula predice exactamente 1, mide la parte del inflado que la red ha *promediado* sobre todos los ángulos vistos en el entrenamiento y aplica indiscriminadamente. Es esa componente la que obliga a que la ventana de verificación de tamaño del descenso sea tan ancha (0,85 a 1,70) en lugar del $\pm 30\%$ del crucero.